UNIVERSIDADE DA CORUÑA

Escola Politécnica Superior

Trabajo Fin de Grado

CURSO 2020/21

EVALUACIÓN EXPERIMENTAL DE UN MODELO DE SIMULACIÓN DE UN ROBOT MÓVIL PARA MEJORAR LA FIDELIDAD DE COMPORTAMIENTO ENTRE EL MODELO SIMULADO Y EL REAL

Grado en Ingeniería en Tecnologías Industriales

ALUMNO

Cristina Sánchez Prado

TUTORES

Alma María Mallo Casdelo

Alejandro Paz López

FECHA

Diciembre 2020

Título

Evaluación experimental de un modelo de simulación de un robot móvil para mejorar la fidelidad de comportamiento entre el modelo simulado y el real.

Resumen

El presente Trabajo de Fin de Grado (TFG) se enmarca en las líneas de investigación en robótica autónoma y robótica educativa del Grupo Integrado de Ingeniería (GII). En este contexto el uso de simuladores facilita y complementa la enseñanza de conceptos de robótica autónoma e inteligencia artificial al eliminar la dificultad que supone adquirir gran número de robots para su uso en las aulas, a la vez que permite impartir materias de forma no presencial cuando sea necesario.

Uno de los principales problemas que aparecen con los modelos simulados es la dificultad que supone adecuar el comportamiento del modelo del robot simulado con el del robot real (lo que se conoce como “reality-gap”) de forma que se pueda desarrollar un programa en simulación y que ese mismo programa pueda ser ejecutado en el robot real sin que existan diferencias importantes en su comportamiento e interacción con el entorno. Así, el objetivo central de este trabajo será validar de forma experimental la fidelidad en el comportamiento de un robot móvil en un entorno de simulación sobre la plataforma Gazebo frente al comportamiento en un entorno real.

Título

Avaliación experimental dun modelo de simulación dun robot móbil para mellorar a fidelidade do comportamento entre o modelo simulado e o real.

Resumo

Este Traballo Fin de Grao (TFG) forma parte das liñas de investigación en robótica autónoma e robótica educativa do Grupo de Enxeñaría Integrada (GII). Neste contexto, o uso de simuladores facilita e complementa o ensino da robótica autónoma e conceptos de intelixencia artificial eliminando a dificultade de adquirir un gran número de robots para o seu uso na aula, ao mesmo tempo que permite ensinar materias de forma non presencial cando sexa necesario.

Un dos principais problemas que aparecen cos modelos simulados é a dificultade de adaptar o comportamento do modelo do robot simulado co do robot real (coñecido como "reality-gap") de maneira que se poida desenvolver un programa en simulación e que o mesmo programa poida ser executado no robot real sen que haxa diferenzas importantes no seu comportamento e interacción co ambiente. Así, o principal obxectivo deste traballo será validar experimentalmente a fidelidade no comportamento dun robot móbil nun entorno de simulación na plataforma Gazebo en comparación co comportamento nun entorno real.

Título

Experimental evaluation of a mobile robot's simulation model to improve the behavior's fidelity between the simulated and the real model.

Summary

This Final Degree Project (TFG) is part of the research lines in autonomous robotics and educational robotics of the Integrated Group for Engineering Research (GII). In this context, the use of simulators facilitates and complements the teaching of autonomous robotics and artificial intelligence concepts by eliminating the difficulty of acquiring a large number of robots for use in the classroom, at the same time that it allows teaching subjects in a non-presential way when be necessary.

One of the main problems that appear with simulated models is the difficulty of adapting the behavior of the simulated robot model with that of the real robot (which is known as "reality-gap") so that a program can be developed in simulation and that the same program can be executed in the real robot without significant differences in its behavior and interaction with the environment. Thus, the main objective of this work will be to experimentally validate the fidelity in the behavior of a mobile robot in a simulation environment on the Gazebo platform compared to the behavior in a real environment.

UNIVERSIDADE DA CORUÑA

Escola Politécnica Superior

TRABAJO FIN DE GRADO

CURSO 2020/21

EVALUACIÓN EXPERIMENTAL DE UN MODELO DE SIMULACIÓN DE UN ROBOT MÓVIL PARA MEJORAR LA FIDELIDAD DE COMPORTAMIENTO ENTRE EL MODELO SIMULADO Y EL REAL

Grado en Ingeniería en Tecnologías Industriales

Documento

MEMORIA

Índice de contenido

1 Introducción 14
1.1 Motivación 14
1.2 Antecedentes 14
1.2.1 Robótica educativa 15
1.2.2 Conducción autónoma 20
1.3 Objetivos 22
2 Metodología 23
3 Planificación 26
4 Fundamentos tecnológicos 29
4.1 Robobo 29
4.1.1 Hardware 30
4.1.2 Software 30
4.1.3 Detección de marcadores ArUco 32
4.2 Python 33
4.3 Gazebo 34
4.4 ROS 35
5 Análisis global y especificación inicial de requisitos 37
6 Diseño e implementación 39
6.1 Entorno real 40
6.2 Entorno simulado 43
6.3 Pruebas básicas 45
6.3.1 Motores 45
6.3.2 Cámara 51
6.3.3 Infrarrojos 62
6.4 Pruebas en entornos no controlados 66
6.4.1 Motores 66
6.4.2 Cámara 76
6.4.3 Infrarrojos 82
6.5 Pruebas en escenarios específicos 89
6.6 Pruebas multi-robot 96
6.6.1 Implementación en Gazebo de multi-robot 96
6.6.2 Ejemplo multi-robot 98
7 Conclusiones y trabajo futuro 101
8 Referencias 103

AENXOS.....105
ANEXO I: Presupuesto.....105

Índice de figuras

Figura 1. LEGO Mindstorms EV3. Fuente: [3]. 16
Figura 2. Bloque EV3. Fuente: [4]. 16
Figura 3. Programación de LEGO Mindstreams EV3. Fuente: 17
Figura 4. Modelos oficiales de TurtleBot 3. Fuente: [5]. 17
Figura 5. Vista superior del Kephera IV. Fuente: [6]. 18
Figura 6. Vista inferior del Kephera IV. Fuente: [6]. 18
Figura 7. Robot humanoide $\boldsymbol{NAO6}$. Fuente: [7]. 19
Figura 8. Robot ROMO. Fuente: [8]. 20
Figura 9. Detalle del modelo de ciudad propuesta por Bel Geddes. Fuente: [9]. 20
Figura 10. Niveles de autonomía según el SAE. Fuente: Elaboración propia a partir de [10]. 21
Figura 11. Diagrama general de flujo seguido para el presente TFG. Fuente: Elaboración propia. 24
Figura 12. Diagrama de flujo seguido para las pruebas realizadas en el presente TFG. Fuente: Elaboración propia. 25
Figura 13. Distribución temporal de las tareas a realizar para llevar a cabo el proyecto. Fuente: Elaboración propia 26
Figura 14. Robobo. Fuente: [12]. 29
Figura 15. Esquema de la base Robobo. Fuente: [13]. 30
Figura 16. Diagrama arquitectura Robobo framework. Fuente: [13]. 31
Figura 17. Framework del Robobo. Fuente: [13]. 32
Figura 18. Marcador ArUco. Fuente: [15]. 32
Figura 19. Código en Python para conectarse al Robobo. Fuente: Elaboración propia. 33
Figura 20. Simulación del Robobo y de TurtleBot en Gazebo. Fuente: [17], [18]. 34
Figura 21. Modelos de comunicación soportados por ROS. Fuente: A partir de [20]. 36
Figura 22. Laboratorio del Grupo Integral de Ingeniería. Fuente: Elaboración propia. 40
Figura 23. Construcción ciudad. Fuente: Elaboración propia. 40
Figura 24. Ciudad acabada. Fuente: Elaboración propia. 41
Figura 25. Plano maqueta ciudad. Fuente: Elaboración propia. 42
Figura 26. Mundo 1 simulado con Gazebo. Fuente: Elaboración propia. 43
Figura 27. Elaboración del modelo simulado de la maqueta de la ciudad en SolidWorks. Fuente: Elaboración propia. 44
Figura 28. Mundo 3 simulado con Gazebo. Fuente: Elaboración propia. 44
Figura 29. Trayectoria que debe reccorer el Robobo. Fuente: Elaboración propia. 46
Figura 30. Posición del Robobo antes y después de realizar el circuito (vista en planta) en entorno real. Fuente: Elaboración propia. 47

Figura 31. Posición del Robobo antes y después de realizar el circuito (vista frontal). Fuente: Elaboración propia. 47

Figura 32. Posición del Robobo antes de realizar el circuito en entorno simulado (Mundo 1). Fuente: Elaboración propia. 48

Figura 33. Posición del Robobo después de realizar el circuito en entorno simulado (Mundo 1). Fuente: Elaboración propia. 48

Figura 34. Posición del Robobo después de realizar el circuito en entorno simulado modificando el tiempo de giro (Mundo 1). Fuente: Elaboración propia. 49

Figura 35. Posición del Robobo antes de realizar el circuito en el entorno simulado tras la actualización del simulador. Fuente: Elaboración propia. 49

Figura 36. Posición del Robobo después de realizar el circuito en el entorno simulado tras la actualización del simulador. Fuente: Elaboración propia. 50

Figura 37. Polinomios de interpolación con tendencia exponencial, logarítmico, potencial y cuadrática para las pruebas realizadas en el entorno real. Fuente: Elaboración propia.....52

Figura 38. Esquema del circuito para las pruebas dinámicas de detección del ArUco. Arriba: Vista en planta. Abajo: Vista frontal. Fuente: Elaboración propia.....53

Figura 39. Representación de la velocidad frente al espacio respecto a la coordenada cero en las pruebas obtenidas con el robot real. Fuente: Elaboración propia. 56

Figura 40. Toma de datos a diferentes distancias. Arriba de izquierda a derecha: 30 cm y 80 cm. Abajo: 120 cm. Fuente: Elaboración propia. 56

Figura 41. Polinomios de interpolación con tendencia exponencial, logarítmico, potencial y cuadrática para las pruebas realizadas en el entorno simulado. Fuente: Elaboración propia. 57

Figura 42. Simulación de la prueba de detección de ArUcos a distintas velocidades en el simulador. Fuente: Elaboración propia. 58

Figura 43. Representación de la velocidad frente al espacio respecto a la coordenada cero en las pruebas obtenidas con el simulador. Fuente: Elaboración propia. 60

Figura 44. Comparación de los resultados obtenidos en los entornos para las pruebas realizadas con el robot en movimiento. Fuente: Elaboración propia. 61

Figura 45. Pruebas estáticas básicas para la validación de los infrarrojos en el entorno real. Fuente: Elaboración propia. 62

Figura 46. Obtención de datos con el sensor de infrarrojos de Gazebo en el entorno simulado. Fuente: Elaboración propia. 64

Figura 47. Validación de los motores en entornos no controlados en superficie de madera (vista en planta). Fuente: Elaboración propia. 67

Figura 48. Validación de los motores en entornos no controlados en superficie de madera (vista frontal). Fuente: Elaboración propia. 67

Figura 49. Validación de los motores en entornos no controlados en maqueta de la ciudad (vista en planta). Fuente: Elaboración propia. 68

Figura 50. Validación de los motores en entornos no controlados en maqueta de la ciudad (vista frontal). Fuente: Elaboración propia. 68

Figura 51. Pruebas realizadas en una superficie inclinada de madera con diferentes ángulos de inclinación (4º, 8º y 11º respectivamente) Fuente: Elaboración propia.....69

Figura 52. Pruebas realizadas en una superficie inclinada de cartón plastificado con diferentes ángulos de inclinación (4º y 11º respectivamente) Fuente: Elaboración propia. ...69

Figura 53. Pasos para la modificación del coeficiente de rozamiento de la superficie. Fuente: Elaboración propia. 70

Figura 54. Posición del Robobo antes de realizar el circuito en entorno simulado. Fuente: Elaboración propia. 70

Figura 55. Posición del Robobo tras realizar el circuito en entorno simulado. Fuente: Elaboración propia. 71

Figura 56. Posición del Robobo tras modificar el tiempo en el que debe realizar cada giro y tras terminar el circuito en entorno simulado. Fuente: Elaboración propia. 71

Figura 57. Posición del Robobo antes de realizar el circuito en el entorno simulado tras la actualización del simulador para la superficie de madera. Fuente: Elaboración propia.....72

Figura 58. Posición del Robobo después de realizar el circuito en el entorno simulado tras la actualización del simulador para la superficie de madera. Fuente: Elaboración propia.....72

Figura 59. Posición del Robobo antes de realizar el circuito en entorno simulado (Mundo 2). Fuente: Elaboración propia. 73

Figura 60. Posición del Robobo tras realizar el circuito en entorno simulado (Mundo 2). Fuente: Elaboración propia. 73

Figura 61. Posición del Robobo tras modificar el tiempo en el que debe realizar cada giro y tras terminar el circuito en entorno simulado. Fuente: Elaboración propia. 74

Figura 62. Posición del Robobo antes de realizar el circuito en el entorno simulado tras la actualización del simulador para la superficie de vinilo. Fuente: Elaboración propia. 74

Figura 63. Posición del Robobo después de realizar el circuito en el entorno simulado tras la actualización del simulador para la superficie de vinilo. Fuente: Elaboración propia. 75

Figura 64. Polinomios de interpolación con tendencia exponencial, logarítmico, potencial y cuadrática para las pruebas realizadas en el entorno real a 390 lux. Fuente: Elaboración propia. 77

Figura 65. Polinomios de interpolación con tendencia exponencial, logarítmico, potencial y cuadrática para las pruebas realizadas en el entorno real a 41 lux. Fuente: Elaboración propia. 78

Figura 66. Comparación de los valores obtenidos para diferentes lux. Fuente: Elaboración propia. 80

Figura 67. Simulación de la prueba de detección de ArUcos a distintas velocidades y distintas distancias en el simulador. Fuente: Elaboración propia. 80

Figura 68. Representación de la velocidad frente al espacio respecto a la coordenada cero en las pruebas obtenidas con el simulador. Fuente: Elaboración propia. 82

Figura 69. Pruebas estáticas en entornos no concretos para la validación de los infrarrojos en el entorno real. Fuente: Elaboración propia. 83

Figura 70. Polinomios de interpolación obtenidas para cada superficie para las pruebas realizadas en el entorno real. Fuente: Elaboración propia. 84

Figura 71. Obtención de datos con el sensor de infrarrojos de Gazebo en el entorno simulado. Fuente: Elaboración propia. 86

Figura 72. Esquema del circuito que debe realizar el Robobo en la ciudad. Fuente: Elaboración propia. 89

Figura 73. Inicio de la trayectoria del Robobo en la maqueta de la ciudad en el entorno real. Fuente: Elaboración propia. 90

Figura 74. Circuito del Robobo en el entorno real en la maqueta de la ciudad. Fuente: Elaboración propia. 91

Figura 75. Circuito que realiza el Robobo en el entorno real en la maqueta de la ciudad. Fuente: Elaboración propia. 91

Figura 76. Inicio de la trayectoria del Robobo en la maqueta de la ciudad en el entorno real. Fuente: Elaboración propia. 93

Figura 77. Circuito del Robobo en el entorno real en la maqueta de la ciudad. Fuente: Elaboración propia. 93

Figura 78. Circuito del Robobo en el entorno real en la maqueta de la ciudad. Fuente: Elaboración propia. 94

Figura 79. Ejemplo de jerarquía de name spaces en el fichero de lanzamiento (launch) adaptado para ser implementado en Gazebo. Fuente: [21]. 97

Figura 80. Modificación del plugin de los encoders para la simulación en Gazebo. Fuente: [22] 97

Figura 81. Circuito planteado para llevar a cabo las pruebas multi-robot. Fuente: Elaboración propia. 98

Figura 82. El Robobo1 detecta un objeto en el trayecto. Fuente: Elaboración propia. 99

Figura 83. El Robobo1 comunica al Robobo2 que hay un objeto cortando la calle. Fuente: Elaboración propia. 99

Índice de tablas

Tabla 1. Comparación entre los parámetros de ambos entornos para el mismo movimiento para la superficie blanca. Fuente: Elaboración propia. 50

Tabla 2. Área que devuelve el ArUco en las pruebas en entorno real. Fuente: Elaboración propia. 52

Tabla 3. Distancia obtenida después de la interpolación en las pruebas en entorno real. Fuente: Elaboración propia. 53

Tabla 4. Distancia a la que se detiene el Robobo respecto a la coordenada 0 al leer la señal de STOP en el entorno real. Fuente: Elaboración propia....55

Tabla 5. Área que devuelve el ArUco en las pruebas en entorno simulado. Fuente: Elaboración propia. 57

Tabla 6. Distancia obtenida después de la interpolación en las pruebas en entorno simulado. Fuente: Elaboración propia. 58

Tabla 7. Distancia a la que se detiene el Robobo respecto a la coordenada 0 al leer la señal de STOP en el simulador. Fuente: Elaboración propia. 59

Tabla 8. Comparativa de errores absolutos y relativos entre entorno real y simulado. Fuente: Elaboración propia. 61

Tabla 9. Valores del sensor de infrarrojos obtenidos en pruebas estáticas en el entorno real. Fuente: Elaboración propia. 63

Tabla 10. Valores del sensor de infrarrojos obtenidos en pruebas dinámicas para una superficie blanca. Fuente: Elaboración propia. 63

Tabla 11. Valores del sensor de infrarrojos obtenidos en pruebas estáticas en el entorno simulado. Fuente: Elaboración propia. 64

Tabla 12. Comparación de los valores del sensor de infrarrojos obtenidos en el entorno real frente al simulado. Fuente: Elaboración propia....65

Tabla 13. Comparación entre los parámetros de ambos entornos para el mismo movimiento para la superficie de madera. Fuente: Elaboración propia. 75

Tabla 14. Comparación entre los parámetros de ambos entornos para el mismo movimiento para la superficie de vinilo. Fuente: Elaboración propia. 76

Tabla 15. Área que devuelve el ArUco en las pruebas en entorno real con 390 lux y 41 lux. Fuente: Elaboración propia. 77

Tabla 16. Distancia obtenida después de la interpolación en las pruebas en entorno real para 390 lux y 41 lux. Fuente: Elaboración propia. 78

Tabla 17. Distancia a la que se detiene el Robobo respecto a la coordenada 0 al leer la señal de STOP para 390, 310 y 55 lux. Fuente: Elaboración propia. 79

Tabla 18. Comparativa de los valores obtenidos a una distancia de 40 cm y otra de 80 cm en el simulador. Fuente: Elaboración propia....81

Tabla 19. Valores del sensor de infrarrojos obtenidos en pruebas estáticas para distintas superficies en un entorno real. Fuente: Elaboración propia. 83

Tabla 20. Valores del sensor de infrarrojos obtenidos en pruebas dinámicas para una superficie blanca. Fuente: Elaboración propia. 85

Tabla 21. Valores del sensor de infrarrojos obtenidos en pruebas dinámicas para una superficie de madera. Fuente: Elaboración propia.....85

Tabla 22. Valores del sensor de infrarrojos obtenidos en pruebas dinámicas para una superficie negra. Fuente: Elaboración propia. 86

Tabla 23. Valores del sensor de infrarrojos obtenidos en pruebas estáticas para distintas superficies en un entorno simulado. Fuente: Elaboración propia. 87

Tabla 24. Valores del sensor de infrarrojos obtenidos para la caja blanca en el simulador. Fuente: Elaboración propia. 87

Tabla 25. Valores del sensor de infrarrojos obtenidos para la caja marrón en el simulador. Fuente: Elaboración propia. 88

Tabla 26. Valores del sensor de infrarrojos obtenidos para la caja negra en el simulador. Fuente: Elaboración propia. 88

Tabla 27. Comparación de los parámetros que intervienen en el movimiento del Robobo en los distintos entornos. Fuente: Elaboración propia. 95

1 INTRODUCCIÓN

1.1 Motivación

La digitalización, las nuevas tecnologías de comunicación, los avances en el campo de la computación y, en definitiva, la revolución tecnológica que se está produciendo en nuestra sociedad, están influyendo significativamente en la enseñanza de conceptos de robótica e inteligencia artificial. Estos conceptos contribuyen al aprendizaje de programación con modelos de robots reales, asequibles para cualquier persona interesada en este campo, y al uso de simuladores que faciliten su formación.

Todos estos avances permiten su aplicación no solo en fábricas y cadenas de montaje, sino también en escenarios como la conducción autónoma dónde la conectividad, la inteligencia artificial y la automatización traen constantemente nuevos modelos de negocio.

Por otro lado, las nuevas generaciones han crecido de la mano de todas estas tecnologías y es importante desarrollar en ellos curiosidad e interés en el mundo que les rodea de una forma sencilla, didáctica y divertida para que puedan seguir con nuestro legado. El uso de robots educativos y métodos de programación didáctica junto con la utilización de simuladores que ayuden a entender su comportamiento pueden fomentar de una manera directa este desarrollo.

Además, es importante también poder enseñarles que existen problemas que deben tenerse en cuenta a la hora de trabajar con robots, como la luz ambiental o la superficie en la que se deslice el robot y que, mediante sistemas de programación sencillos e intuitivos, sean ellos los que se enfrenten a estos impedimentos.

También puede ser interesante, que ellos mismos sean los que aprendan a configurar el modelo de simulación para tener en cuenta estas variables y que puedan aprender de un modo práctico estos inconvenientes que son inevitables que aparezcan. Por estas razones también es importante caracterizar las diferencias que se puedan encontrar entre modelos reales y simulados para poder enseñarles a trabajar con estos obstáculos.

Conforme a estas premisas, surge la necesidad hoy en día de encontrar una fidelidad entre realidad y simulación que ayude a respaldar los estudios sobre robótica autónoma y apoye a su crecimiento para poder seguir avanzando en este mundo. De esta manera, es importante estudiar y analizar las diferencias y similitudes entre el comportamiento de un robot real y el mismo robot simulado.

1.2 Antecedentes

Este Trabajo Fin de Grado (TFG) se enmarca dentro de un proyecto de investigación en robótica autónoma y robótica educativa del Grupo Integrado de Ingeniería (GII) de la Universidade da Coruña (UDC) con el que se pretende mejorar la fidelidad del comportamiento de un robot en un modelo real con el de un modelo simulado.

Los experimentos en un escenario real se realizarán con una plataforma robótica móvil controlada mediante un smartphone desarrollada por la spin-off de la UDC, Manufactura de Ingenios Tecnológicos (MINT). Para dicho escenario se ha llevado a cabo el montaje de una maqueta de una ciudad de 16 m^2 en el que se realizarán las diferentes pruebas incrementales. La participación en el proceso de montaje de esta ciudad reside en la importancia de conocer

en detalle el entorno para que a la hora de realizar el modelo simulado sea lo más fidedigno posible a la realidad.

Uno de los objetivos de este trabajo es comparar el funcionamiento de los sensores que tiene el robot real frente al robot simulado mediante un programa de simulación como Gazebo, para que en trabajos futuros pueda ser corregido y pueda ser usado como herramienta en diversos proyectos de robótica autónoma. Es por ello por lo que se ha planteado la idea de recrear un escenario en dos entornos diferentes, el real y el simulado, pero que compartan similitudes para concluir un análisis que pueda servir de ayuda.

Ambos entornos estarán dotados con diferentes objetos que un vehículo pueda encontrarse en la realidad, para poder detectar las dificultades que puedan aparecer y encontrar una solución. Para modelar el escenario que se usará en la simulación del que hablaremos más adelante, se ha empleado el software CAD.

Además, este proyecto también tiene el planteamiento de ser abordado desde un punto de vista educativo. De esta forma, se podrán realizar mejoras en el programa de simulación para que estudiantes que estén trabajando con este tipo de herramientas puedan seguir desarrollando sus capacidades sin ningún impedimento.

Robobo ha demostrado ser una herramienta educativa muy importante para el desarrollo de conceptos de robótica en ámbitos de estudio, gracias a los diferentes lenguajes con los que puede ser programado. Esto permite que estudiantes de diferentes edades puedan dar uso de este robot en función de sus conocimientos y que pueda seguir siendo de utilidad a medida que el alumno vaya desarrollando diferentes capacidades. De esta forma, si se tiene un buen programa de simulación que sea fidedigno con la realidad, puede ayudar al alumno a entender y conocer más en detalle al robot de forma autónoma fuera de las aulas, así como realizar pruebas y desarrollos de forma más sencilla que posteriormente puedan ser aplicados a un entorno real.

1.2.1 Robótica educativa

Es un hecho que hoy en día podemos encontrar robots por todas partes. Miremos allá a dónde miremos existen robots con los que interactuamos en el día a día para hacer nuestras vidas más sencillas. Sin embargo, los robots no han existido siempre. En 1948 aparecen los primeros robots autónomos creados por W. Grey Walter que recordaban a unas tortugas por su lento movimiento y una especie de caparazón que recibían el nombre de *máquina especulatrix* [1].

En la década de los noventa, la robótica fue ganando protagonismo y comenzó a expandirse en diversas áreas, llegando así a desarrollarse dentro del ámbito educativo, con el uso de dispositivos elaborados localmente y diseñados concretamente con fines educativos. El crecimiento en este campo viene de la mano del MIT (Massachussets Institute of Technology) y la colaboración de la compañía Lego, creando una serie de dispositivos con un lenguaje de programación dirigido a un público de edades muy tempranas [2].

A continuación, se hablará de distintos robots autónomos que han sido usados para fines educativos y de investigación.

1.2.1.1 LEGO Mindstorms EV3

Lego Mindstorms Ev3 (Figura 1) es el robot de tercera generación fabricado por la empresa danesa Lego en su línea de robótica para niños Lego Mindstorms y que comenzó a comercializarse en 2013 .



Figura 1. LEGO Mindstorms EV3. Fuente: [3]**.**

El bloque EV3 (Figura 2) actúa como centro de control y fuente de potencia del robot. Además, cuenta con un procesador AMR9 con sistema operativo basado en Linux, un puerto mini USB para PC, un puerto de host USB, un lector de tarjetas Micro SD, un altavoz integrado, receptor de señales infrarrojas, receptor Bluetooth y Wi-Fi y 4 puertos de entrada y 4 puertos de salida, a los que se pueden conectar los sensores y actuadores incluidos [3].



Figura 2. Bloque EV3. Fuente: [4]**.**

El bloque EV3 cuenta con una interfaz de programación propia, un software desarrollado por LEGO Education basado en bloques de programación cuyo objetivo es que sea un programa sencillo, visual e intuitivo (Figura 3). No obstante, este tipo de programación está enfocado para estudiantes de primaria y secundaria, y aunque es posible actualizar el firmware para que el robot pueda ser programado en Python y así ser usado por estudiantes de grado universitario, no posee sensores avanzados como cámaras o láser ni soporta algoritmos complejos ya que no cuenta con ninguna capacidad de cómputo para ello.

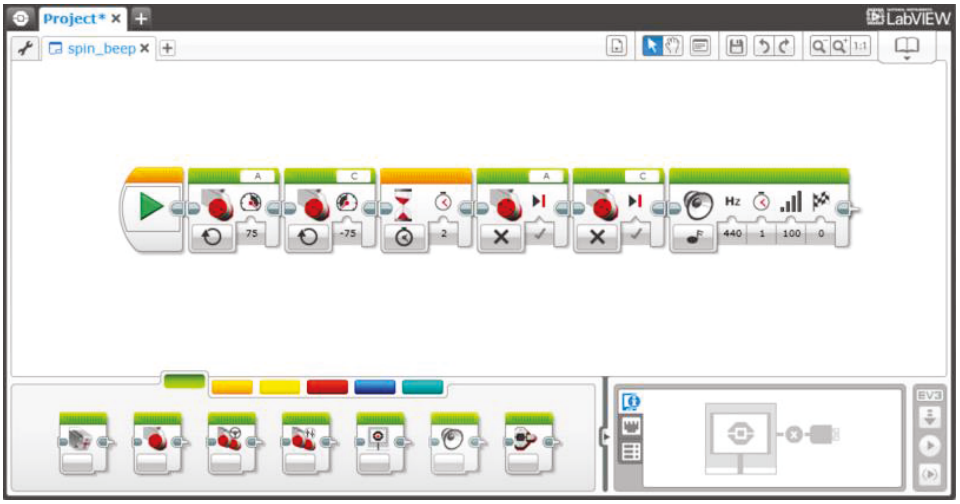


Figura 3. Programación de LEGO Mindstreams EV3. Fuente: [4].

1.2.1.2 TurtleBot3

TurtleBot es un kit robótico de bajo coste que cuenta con un software de código abierto y que puede ser programado en ROS (Robot Operating System). Su tercera versión, TurtleBot3, fue lanzada en 2017 por Open Robotics y ROBOTIS basándose en el modelo Turtle de Logo. Actualmente cuenta con 3 versiones: Waffle, Waffle Pi y Burguer (Figura 4).

TurtleBot3 cuenta con un sensor de distancia de 360 grados (LIDAR) y la tecnología central que utiliza es SLAM (localización y mapeo simultáneo) gracias a la cual es capaz de construir un mapa y conducir el robot por interiores, lo que permite que este tipo de robots puedan ser usados en entornos domésticos. Además, cuenta con la capacidad de control remoto desde portátil o *Smartphone* basado en Android.

Además del LIDAR Y SLAM, los kits de TurtleBot3 incluyen: una base con controlador OpenCR, dos ruedas con motor propio, batería de 1800 mAh y una placa SBC que puede ser una Raspberry Pi o Intel Joule.

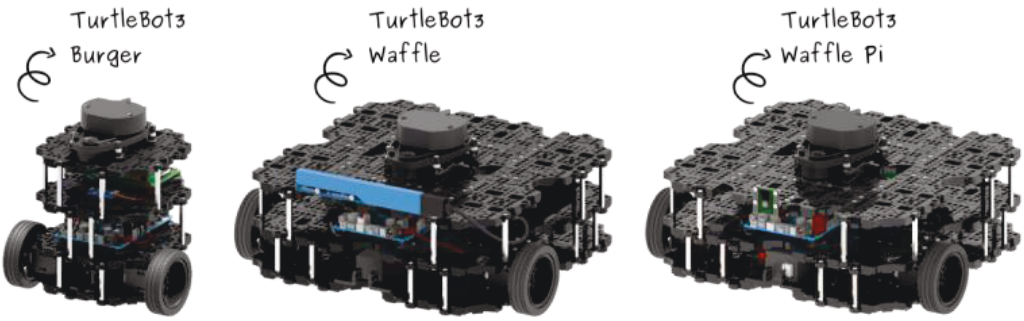


Figura 4. Modelos oficiales de TurtleBot 3. Fuente: [5].

1.2.1.3 Khepera IV

El Khepera IV es un robot compacto diseñado por la empresa suiza K-Team Corporation que desarrolla, fabrica y comercializa robots móviles destinados a la investigación y la educación [6].



Figura 5. Vista superior del Kephera IV. Fuente: [6]**.**

El robot incluye una serie de 8 sensores infrarrojos para la detección de obstáculos, 4 en la parte inferior para evitar caídas o seguimiento de líneas, 5 sensores ultrasónicos para la detección de objetos de largo alcance, cámara a color, WIFI, conectividad Bluetooth, acelerómetro, giroscopio y sistema de odometría

.

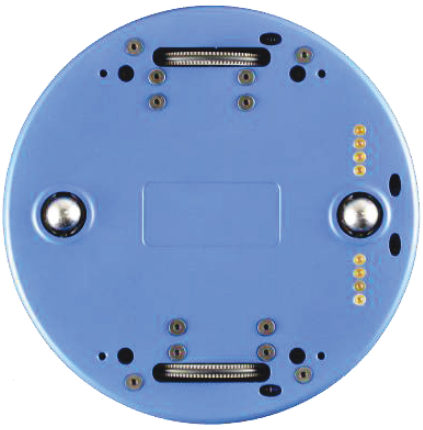


Figura 6. Vista inferior del Kephera IV. Fuente: [6]**.**

Esta nueva generación cuenta con un sistema operativo Linux completo y proporciona un entorno C/C++ para el desarrollo de aplicaciones. Su diseño está pensado para ser usado en laboratorios de investigación interiores, que requieren de poco espacio y que sirvan para llevar a cabo diferentes aplicaciones como navegación, inteligencia artificial, demostración, etc.

1.2.1.4 NAO

NAO (Figura 7) es un robot humanoide totalmente programable famoso en todo el mundo y creado por SoftBank Robotics. Constituye una herramienta de programación usada para fines académicos y de investigación, aunque también se usa como asistentes en compañías para dar la bienvenida, informar y entretener a los visitantes.

El desarrollo del robot comenzó en 2006 con el lanzamiento del Proyecto Nao y actualmente ya van por la sexta versión, NAO^6, lanzada en 2018. Este robot de 58 cm de altura cuenta con 25 grados de libertad que le permiten moverse y adaptarse al entorno; 7 sensores táctiles localizados en la cabeza, manos y pies; 4 micrófonos direccionales y altavoces para interactuar con el público, reconocimiento de voz y posibilidad de hablar en 20 idiomas, 2 cámaras 2D que le permiten reconocer formas, objetos e incluso personas y una plataforma de programación completamente abierta [7].



Figura 7. Robot humanoide $\mathbf{NAO^6}$. Fuente: [7]**.**

Nao cuenta con un sistema multimedia basado en Linux incluyendo un software gráfico de programación llamado Choreographe, compatible con Windows, Linux y macOS, que permite programarlo sin tener conocimientos de un lenguaje de programación. Para usuarios más avanzados permite usar C++, Python, Java y Matlab.

1.2.1.5 ROMO

Existen grandes ventajas a la hora de combinar una base móvil y un smartphone ya que la constante actualización del hardware y el software de estos permite una mayor permanencia del robot. Así fue como nació Romo, uno de los primeros minirobot formado por una base y un dispositivo iOS como cerebro (Figura 8). Fue creado por Romotive, una empresa estadounidense fundada por Keller Rinaudo, Phu Nguyen y Peter Seid. El objetivo de esta empresa era hacer que la gente, y sobre todo los niños se interesaran por la informática y la tecnología.



Figura 8. Robot ROMO. Fuente: [8]**.**

Romo funcionaba de manera intuitiva y no requería de una enseñanza previa, eso era lo que lo hacía tan atractivo, pero uno de sus inconvenientes era el elevado coste de esta gama de smartphones y la poca accesibilidad que un niño podía tener a estos dispositivos. Sin embargo, el gran problema de este robot es que fue un proyecto financiado en una web de micro financiación que dejó de ser actualizada en 2013, por lo que actualmente la aplicación se encuentra desfasada y sin soporte oficial [8].

Otro de los inconvenientes es que Apple desde la quinta generación comenzó a usar un nuevo conector Lightning en sus dispositivos. Es por esto que, si tu Romo cuenta con conector de 30 pines, no será compatible con iPhone 5.

Estas carencias provocaron que está y otras plataformas móviles llevaran al Grupo Integrado de Ingeniería de la UDC junto con la spin-off, Mint, a desarrollar su propia plataforma móvil con smartphone, el Robobo, que será el robot empleado para la realización del presente TFG.

1.2.2 Conducción autónoma

El término conducción autónoma comenzó a ser utilizado por primera vez en 1939, en la Exposición Universal de Nueva York con la presentación de Futurama (Figura 9). Una maqueta creada por el diseñador industrial Norman Bel Geddes en la que se presentaba la ciudad del futuro con automóviles moviéndose propulsados por campos electromagnéticos generados por circuitos insertados en el pavimento [9].



Figura 9. Detalle del modelo de ciudad propuesta por Bel Geddes. Fuente: [9]**.**

Sin embargo, no fue hasta 2012 cuando el primer coche con categoría de conducción autónoma con nivel 2 del SAE apareció en el mercado. Este lanzamiento se le atribuye a la marca de automóviles Tesla, pero no es la única que consiguió dicho objetivo. En 2018, Mercedes Benz lanza el nuevo modelo clase E con la misma categoría de conducción autónoma.

Pero ¿Qué es la conducción autónoma? La conducción autónoma es una modalidad de conducción que reúne los sensores, procesadores, actuadores y software necesarios para conducirse de forma autónoma, es decir, sin el control activo del conductor. Existe una escala propuesta por la SAE (*Society of Autonomous Engineers*), donde se clasifican los distintos niveles de autonomía del vehículo en seis categorías (Figura 10). En esta escala se muestran de menor a mayor el nivel de autonomía, donde el nivel 0 es la autonomía nula, y el nivel 5 la autonomía total.

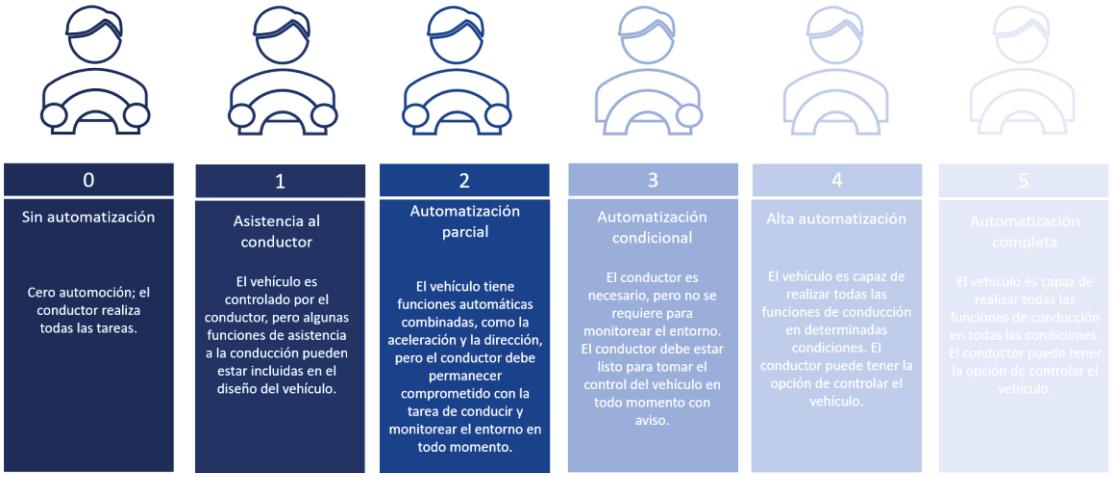


Figura 10. Niveles de autonomía según el SAE. Fuente: Elaboración propia a partir de [10]**.**

En la actualidad no existe ningún vehículo en el mercado con conducción autónoma completa, aunque fabricantes como Tesla ofrece automóviles con un sistema de navegación con esta condición. Las razones por las que la conducción autónoma completa no forma parte de nuestro presente es la dificultad de adaptarse a los distintos tipos de carreteras, la coordinación con otros conductores, los diferentes estilos de conducción de cada usuario, la climatología y lo más importante, la infinidad de sucesos poco probables que pueden llegar a suceder durante la conducción y para los que nuestro vehículo autónomo no está preparado [11].

Hoy en día la tecnología necesaria para hacer realidad la conducción autónoma combina cámaras con el uso del LIDAR (*Laser Imaging Detection and Ranging*). El LIDAR es un dispositivo que permite determinar la distancia desde un emisor láser a un objeto o superficie utilizando un haz de láser pulsado.

Llevar este tipo de sistemas al mercado requiere de grandes esfuerzos de validación tanto en entornos controlados (pistas de pruebas especializadas), como entornos de validación digital y de simulación . Una de las aproximaciones que también puede aportar avances en el campo de la conducción autónoma es la realización de pruebas con maquetas a escala, y el uso de robots en estos casos resulta de gran utilidad.

1.3 Objetivos

Este Trabajo de Fin de Grado tiene como objetivo principal validar de forma experimental la fidelidad en el comportamiento de un robot móvil en un entorno de simulación frente a un entorno real.

Para ello se han planteado una serie de subobjetivos:

- Participar en la definición y construcción de un escenario real y simulado adecuado para la realización de múltiples experimentos que permitan introducir conceptos de robótica autónoma e inteligencia artificial.

- Analizar las capacidades y limitaciones actuales del modelo simulado del robot en Gazebo en relación con su utilización en un escenario como el propuesto. Se tendrán en cuenta las condicionantes externos del uso del robot en contextos educativos.

- Conceptualizar y desarrollar aplicaciones experimentales que permiten evaluar el grado de fidelidad entre el comportamiento en el escenario simulado y en el escenario real. Dichas aplicaciones implicarán la programación del control autónomo del robot utilizando las plataformas software ROS y Python.

- Proponer medidas correctoras en el entorno simulado y/o en la programación/ hardware del robot real.

2 METODOLOGÍA

El establecimiento de una metodología que guíe el desarrollo del proyecto implica analizar cuál es la que podría adecuarse mejor a este en base a sus características. En este caso se ha decidido emplear una metodología iterativa e incremental con un enfoque ágil, por los siguientes motivos principales:

- Los actores que desempeñen el rol de cliente podrán involucrarse durante todo el ciclo de desarrollo. En este caso, tanto los directores del TFG como otros investigadores responsables de la línea de robótica educativa del GII ejercerán este papel.

- Se prevé que sea posible entregar versiones incrementales del producto en periodos de tiempo cortos (por lo general 2 o 3 semanas). Así, tras cada ciclo de desarrollo se analizarán los resultados y se podrá obtener realimentación de los posibles problemas detectados.

- La realimentación será constante desde los primeros ciclos de desarrollo.

- Facilidad para la adaptación o incluso replanificación en caso de que surjan cambios significativos en los requisitos entre cada iteración.

- Para el diseño y desarrollo de los escenarios de prueba se utilizarán tecnologías desconocidas o poco conocidas, por lo que un enfoque ágil con ciclos cortos e hitos periódicos resulta más adecuado para el manejo de la incertidumbre y los riesgos.

El presente proyecto consiste en un trabajo de validación y comparación de sensores y actuadores entre un entorno real y uno simulado en el marco de las metodologías ágiles. Como tal, es necesario establecer las diferentes fases de trabajo que se utilizarán para la realización de las pruebas, la obtención de datos y la redacción de conclusiones. Los pasos que se seguirán se muestran en el diagrama de flujo de la Figura 11 y se detallarán a continuación.

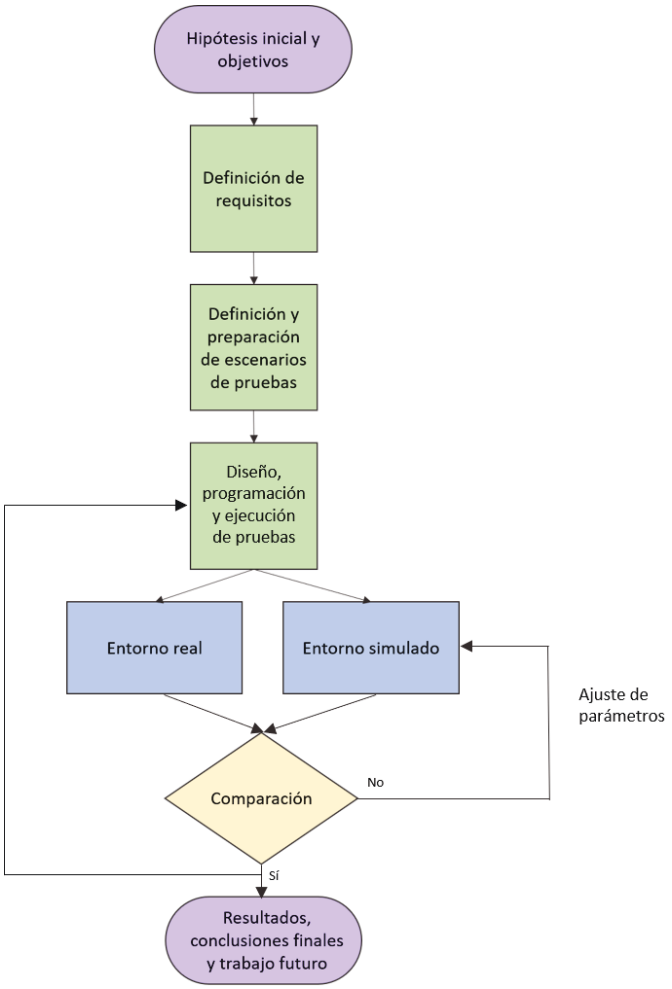


Figura 11. Diagrama general de flujo seguido para el presente TFG. Fuente: Elaboración propia.

Se plantean unos ciclos iniciales de desarrollo secuenciales en los que el objetivo es establecer de forma más detallada los objetivos y los requisitos, hacer una definición preliminar de los escenarios de prueba, familiarizarte con las tecnologías e instalar los entornos software principales que se necesitarán. Al final de cada uno de estos pasos habrá un hito para revisar los avances y detectar posibles problemas.

Después de estas fases iniciales se plantea un ciclo iterativo en el que se repetirán las mismas fases para cada conjunto de pruebas que se ha diseñado inicialmente. De forma resumida, para cada ciclo de desarrollo de cada uno de los conjuntos de pruebas se abordarán las siguientes tareas (de forma que al final de cada uno se obtendrá un entregable incremental que podrá ser evaluado antes de comenzar con el siguiente ciclo de desarrollo):

1. Análisis en detalle del conjunto de pruebas.
2. Diseño de los escenarios necesarios para el conjunto de pruebas.
3. Construcción de escenarios físicos y virtuales, si es necesario.
4. Programación de programas de prueba.

5. Ejecución de los casos de prueba.

6. Análisis de resultados.

7. Documentación y escritura de la memoria.

Por tanto, este mismo esquema se repetirá en cada conjunto de pruebas planteado (Figura 12), buscando, como se ha mencionado, ciclos cortos de desarrollo. En concreto, se repetirá para los siguientes grupos de pruebas:

- Pruebas básicas pensadas para la evaluación de los sensores de manera individual.
- Pruebas en entornos no controlados dónde se repetirán las pruebas básicas y se le añadirán factores que puedan influenciar en su evaluación.
- Pruebas en escenarios específicos en los que los sensores y actuadores trabajarán en común para analizar su funcionamiento en grupo.
- Pruebas multi-robot dónde dos o más robots estarán en contacto y trabajarán con los sensores evaluados anteriormente.

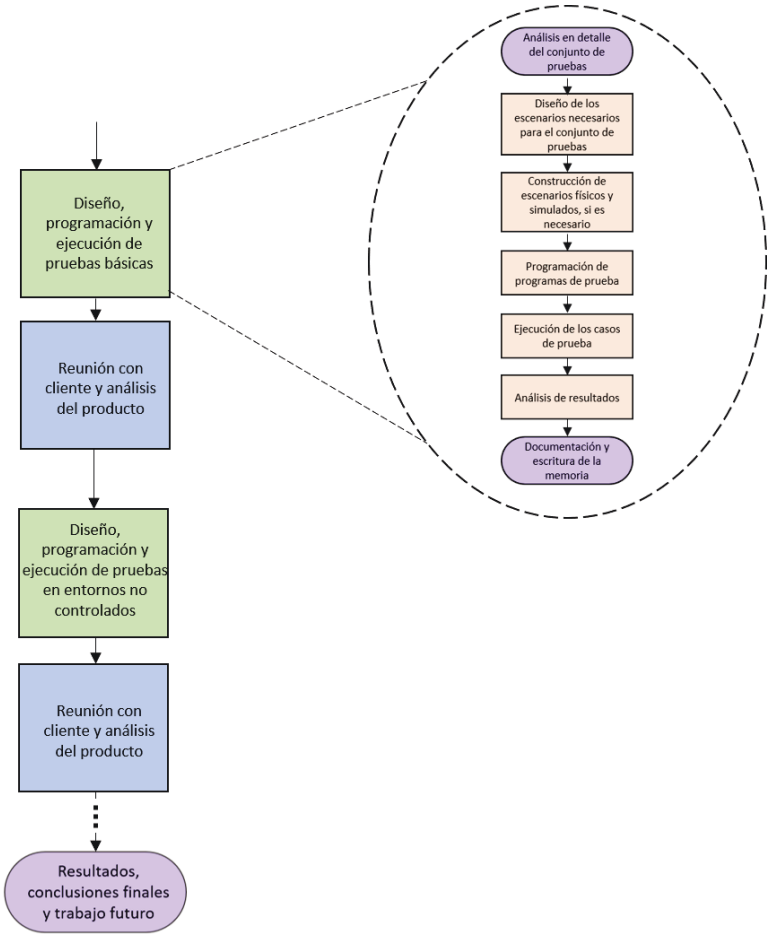


Figura 12. Diagrama de flujo seguido para las pruebas realizadas en el presente TFG. Fuente: Elaboración propia.

3 PLANIFICACIÓN

Para cumplir con los análisis iniciales de los requisitos del presente trabajo, se ha elaborado una lista de tareas con una planificación para realizar las actividades durante la duración del proyecto. En la Figura 13, se muestra una distribución de estas tareas, así como las fechas de entrega para su corrección.

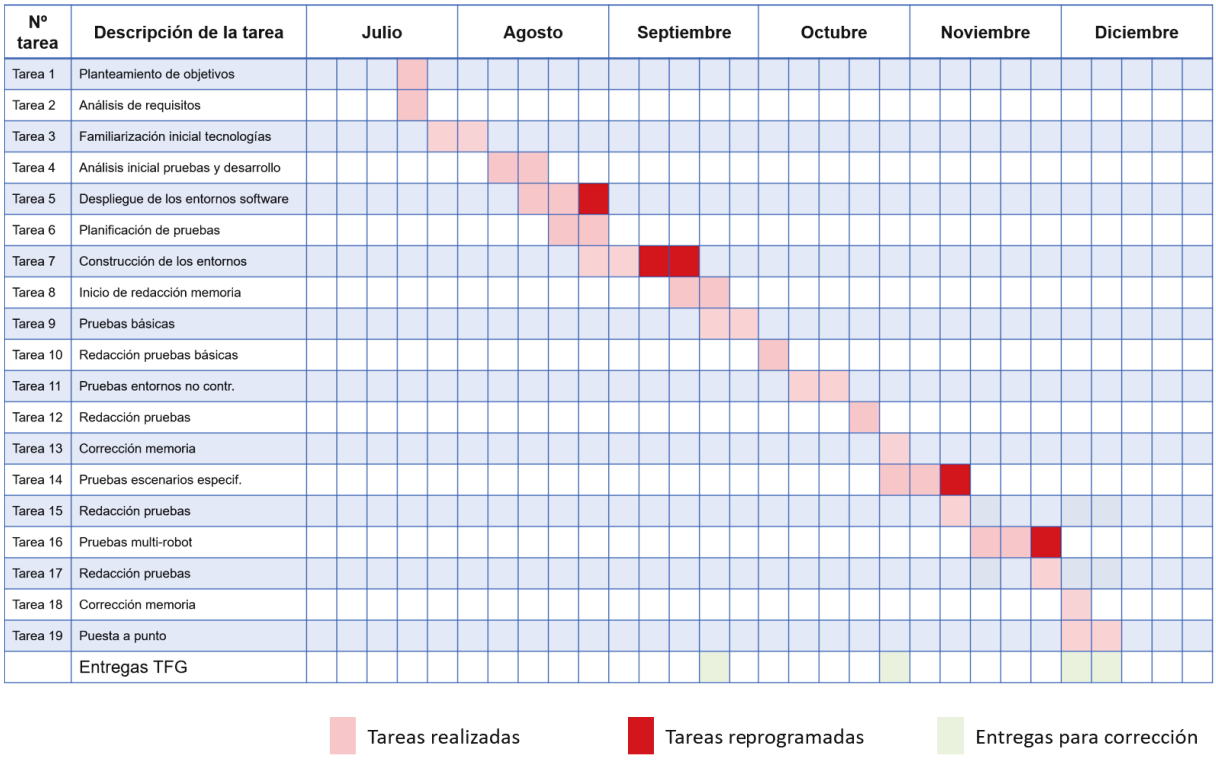


Figura 13. Distribución temporal de las tareas a realizar para llevar a cabo el proyecto. Fuente: Elaboración propia

A continuación, se detallarán las tareas abordadas:

- **Tarea 1**: Planteamiento de unos objetivos iniciales
- **Tarea 2**: Análisis de requisitos iniciales
- **Tarea 3**: Familiarización inicial con tecnologías necesarias.
- **Tarea 4**: Análisis inicial de pruebas y desarrollos necesarios (incluyendo escenarios y programas a implementar)
- **Tarea 5**: Despliegue de los entornos software necesarios para el desarrollo y la realización de las pruebas.
- **Tarea 6**: Determinación y planificación de las pruebas necesarias para lograr los objetivos planteados.

- **Tarea 7**: Determinación y construcción de los entornos necesarios para abordar las distintas pruebas.

- **Tarea 8**: Comienzo de redacción de la memoria del TFG.

- **Tarea 9**: Realización de las pruebas básicas tanto en entorno real como simulado y comparación de los resultados obtenidos.

- **Tarea 10**: Redacción de las pruebas básicas en la memoria con sus conclusiones pertinentes.

- **Tarea 11**: Realización de las pruebas en entornos no controlados tanto en escenario real como en simulado y comparación de los resultados obtenidos.

- **Tarea 12**: Redacción de las pruebas en entornos no controlados en la memoria con sus conclusiones pertinentes.

- **Tarea 13**: Modificación de la memoria tras las correcciones de los tutores.

- **Tarea 14**: Realización de las pruebas en escenarios específicos tanto en escenario real como en simulado y comparación de los resultados obtenidos.

- **Tarea 15**: Redacción de las pruebas en escenarios específicos en la memoria con sus conclusiones pertinentes.

- **Tarea 16**: Realización de las pruebas multi-robot tanto en escenario real como en simulado y comparación de los resultados obtenidos.

- **Tarea 17**: Redacción de las pruebas multi-robot en la memoria con sus conclusiones pertinentes.

- **Tarea 18**: Modificación de la memoria tras las correcciones de los tutores.

- **Tarea 19**: Puesta a punto de la memoria para su entrega.

Como se ha visto en la metodología (Figura 12), para cada conjunto de pruebas que se van a realizar (tarea 9, tarea 11, tarea 14 y tarea 16) se han planificado una secuencia de actividades que incluyen:

1. Análisis en detalle del conjunto de pruebas.

2. Diseño de los escenarios necesarios para el conjunto de pruebas.

3. Construcción de escenarios físicos y virtuales, si es necesario.

4. Programación de programas de prueba.

5. Ejecución de los casos de prueba.

6. Análisis de resultados.

Se puede observar en la planificación que ha habido ciertas tareas que han sido reprogramadas a medida que se realizaba su ejecución. En el caso de la tarea 5, hubo ciertas complicaciones a la hora del despliegue de los entornos software y para la tarea 7, la construcción de uno de los entornos ha llevado más tiempo del esperado, pues hubo aplazamiento en la entrega de los materiales para su construcción.

Por otro lado, en el caso de las tareas 14 y 16, el escenario utilizado para la realización de las pruebas ha sido de uso compartido y por ello no se ha podido disponer de él en cualquier momento.

4 FUNDAMENTOS TECNOLÓGICOS

En este capítulo se explicarán las distintas herramientas que han sido necesarias para abordar el presente proyecto. Al encontrarnos dos distribuciones diferentes, ha sido necesario el uso de entornos de programación distintos, así como la ayuda de un robot real que pueda permitirnos realizar los diversos ensayos para llegar a una serie de conclusiones. Es por ello por lo que las tecnologías de Robobo, Gazebo, Python y ROS serán explicadas en las próximas líneas.

4.1 Robobo

Robobo fue creado en el año 2016, por MINT S.L., una spin-off de la UDC. Se trata de un robot educativo móvil que utiliza una base móvil y un soporte para colocar el smartphone que actúa como centro de procesamiento.

Este robot está diseñado para que estudiantes de secundaria y bachillerato aprendan las claves de programación, inteligencia artificial y robótica autónoma. El elemento clave de Robobo es el smartphone, ya que los teléfonos actuales están equipados con sensores de alta calidad y gran capacidad de cómputo, de los que hablaremos a continuación.

Para analizar al Robobo tenemos que hablar tanto de su hardware como de su software para entender qué es lo que le diferencia del resto de plataformas móviles con smartphone.



Figura 14. Robobo. Fuente: [12].

4.1.1 Hardware

La base del Robobo tiene conectividad bluetooth, lo que permite conectarse a cualquier smartphone que tenga este tipo de conectividad. La base también presenta dos ruedas que permiten el movimiento del robot y una unidad PAN / TILT en la que se posiciona el smartphone para orientar la cámara a la posición deseada. Además, dispone de 8 leds RGB que se pueden programar de forma independiente.

Todos los motores tienen codificadores para poder conocer así la posición de los mismos, de esta forma se puede evitar el bloqueo del motor y detenerlo para que no se produzcan daños. El Robobo tiene 8 sensores de infrarrojos, 5 delante y 3 en la parte de atrás, que le permiten moverse y detectar objetos para poder sortearlos. Entre estos sensores, hay 4 que están orientados al suelo para evitar que el robot se caiga de un lugar alto.

En Figura 15, se puede observar un esquema de dónde están situados los diferentes actuadores y sensores que contiene la base.

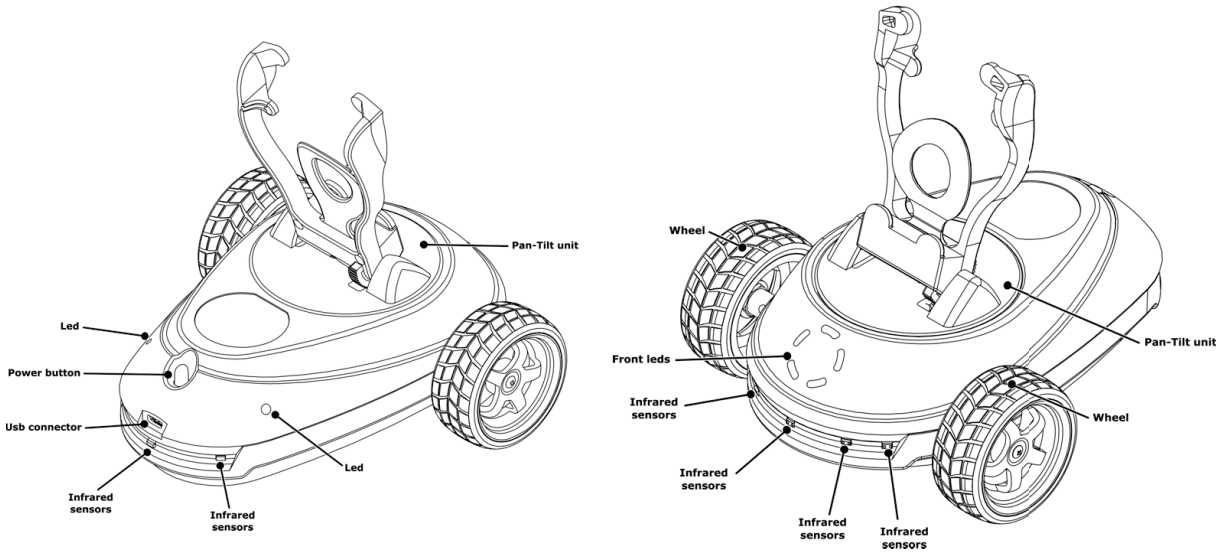


Figura 15. Esquema de la base Robobo. Fuente: [13]**.**

4.1.2 Software

En cuanto al Software, Robobo utiliza un *framework* que permite que el usuario pueda programar al robot desde cualquier dispositivo Windows, Linux o MAC, siempre y cuando estén conectados a la misma red Wifi que el smartphone.

Una de las ventajas de este *framework* es que está basado en una arquitectura modular que le permite escalar en el tiempo y dar soporte a los diferentes lenguajes de programación de manera sencilla. Los módulos son organizados en bibliotecas que cubren diferentes áreas del robot, como: visión, sonido, control remoto, velocidad, emociones, tacto, sensores e interfaz del Robobo.

Se puede observar en la Figura 16, que todas las funciones del robot se encuentran en sus módulos encapsulados. Estos, se pueden clasificar en tres categorías:

- Funciones de actuación.
- Funciones de comunicación.
- Funciones de detección.

Una de las ventajas de esta arquitectura modular es que permite la incorporación de nuevas características y actualizar la lista de módulos sin necesidad de adaptar todo el código básico del robot. Además, se puede activar y desactivar cada uno de los módulos por si alguno de ellos no es necesario en ese momento.

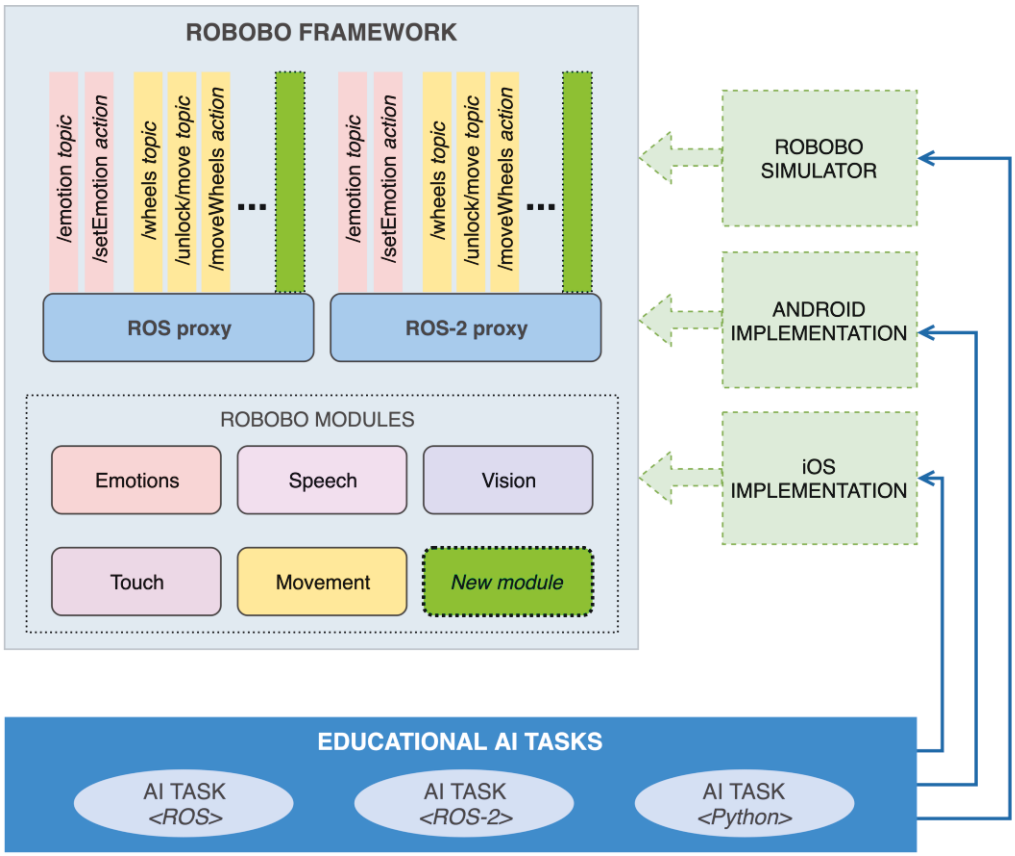


Figura 16. Diagrama arquitectura Robobo *framework*. Fuente: [13]**.**

Esta arquitectura facilita su implementación en diferentes plataformas como Android, iOS y versiones de simulación, compartiendo la misma API de comunicación y permitiendo así que el mismo programa pueda ser ejecutado en cualquiera de los entornos comentados.

Existen alternativas diferentes para programar el Robobo: podemos escribir programas que sean ejecutados directamente en el smartphone usando el Robobo *framework* (podemos escribir programas en Java para dispositivos Android y podemos utilizar Swift para dispositivos iOS). Estas opciones permiten una mayor flexibilidad y un mayor rendimiento, aunque sean más complejas. También existe la posibilidad de programar el robot de manera remota. Hay dos soluciones basadas en *websocket* y ROS/ROS2 que permiten controlar el Robobo mediante bibliotecas que soportan Python, Javascript y Scratch [14].

Las distintas alternativas con las que puede ser programado el Robobo permiten que pueda ser usado por diferentes perfiles de estudiantes. Es por ello que el *framework* está orientado en tres niveles dependiendo del nivel del programador:

- Nivel principiante: Dirigido a estudiantes que se encuentran en la escuela y tienen un interés en la robótica. En este nivel el alumno puede usar Scratch para interactuar con el Robobo y un simulador propio basado en Unity.
- Nivel intermedio: Dirigido para estudiantes de secundaria con ciertos conocimientos de programación. Pueden programar a su robot mediante herramientas como Python o Javascript y simularlos con V-REP.

- Nivel avanzado: Dirigido a estudiantes universitarios con un conocimiento más avanzado de la robótica. En este punto, el programador ya puede utilizar ROS para poder programar al robot y usar Gazebo como simulador.



Figura 17. Framework del Robobo. Fuente: [13].

4.1.3 Detección de marcadores ArUco

Existen un tipo de marcadores artificiales que sirven para orientar a nuestro robot con independencia de su posición. Uno de los tipos más habituales son los marcadores cuadrados binarios, ya que gracias a sus cuatro esquinas es posible estimar de forma correcta la posición de la cámara, y al encontrarse codificados de forma binaria, es más sencillo aplicar técnicas de detección de objetos y corrección de errores.

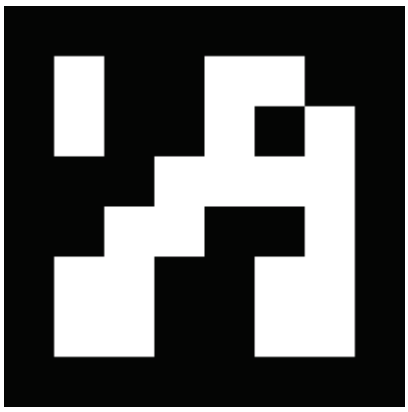


Figura 18. Marcador ArUco. Fuente: [15].

Con esta idea, Sergio Garrido y Rafael Muñoz crean en 2014 la biblioteca ArUco [15], una biblioteca popular para la detección de marcadores fiduciales cuadrados. Un marcador ArUco (Figura 18) es un marcador cuadrado compuesto por un borde negro ancho y una matriz binaria interna que determina su identificador (ID).

La idea de uso de estos marcadores es imprimirlos y colocarlos en escenas en el mundo real, para que posteriormente sean detectados de forma única en la imagen. Su uso principal en robótica estriba en su capacidad de servir como sistema de localización del robot en un entorno. Si el robot pierde su orientación en algún momento, gracias a la detección de algún marcador ArUco puede saber fácilmente donde se encuentra y reorientarse.

Sin embargo, en este TFG los marcadores ArUcos van a ser utilizados como señales de tráfico que indiquen al Robobo la acción que debe realizar durante su movimiento en las pruebas planteadas. La biblioteca ArUco cuenta con su librería en OpenCV, y los investigadores de MINT han implementado una variante que se ejecuta de forma nativa en el smartphone, para poder utilizar las funcionalidades de la detección de ArUcos en Robobo.

4.2 Python

Python es un lenguaje de programación que soporta orientación a objetos, programación imperativa y programación funcional. Es por ello por lo que se dice que es un lenguaje multiparadigma. La filosofía de Python hace hincapié en la legibilidad de su código para que sea un lenguaje interpretado, dinámico y multiplataforma.

Está administrado por la *Python Software Foundation* y posee una licencia de código abierto, denominada Python *Software Foundation License*.

En el presente proyecto, Python ha sido utilizado como herramienta de programación para el desarrollo de casos de prueba y aplicaciones en el escenario real. Como se comentó anteriormente Robobo posee un *framework* para ser programado en este lenguaje gracias a la librería “Robobo.py” creada por MINT. Como se puede observar en la Figura 19, a modo de ejemplo, se puede visualizar el código necesario en este lenguaje para conectarse al Robobo y que este pueda realizar las funciones de las que se dispone.

```
#IMPORTAMOS RUTAS
generalpath = "/home/cristina/TFG/Librerias/"
robobopath = generalpath + "robobo.py-master"
streamingpath = generalpath + "robobo-python-video-stream-master/robobo_video"

import sys
sys.path.append(robobopath)
from Robobo import Robobo

sys.path.append(streamingpath)
from robobo_video import RoboboVideo

IP = "192.168.0.15"

rob = Robobo(IP)
rob.connect()
```

Figura 19. Código en Python para conectarse al Robobo. Fuente: Elaboración propia.

4.3 Gazebo

La simulación de robots es una herramienta esencial a la hora de tratar con robots reales. Un simulador bien diseñado hace posible probar rápidamente algoritmos, diseñar robots, realizar pruebas de regresión y entrenar el sistema de inteligencia artificial utilizando escenarios realistas. Gazebo es uno de los mejores simuladores de robots 3D que existe en la actualidad. Es un programa *open source* (código abierto) y está distribuido bajo la licencia APACHE 2.0. Se utiliza en ámbitos de investigación en robótica e inteligencia artificial, ya que permite simular de manera exacta y eficiente robots en entornos complejos de interior y exterior.

Algunas de sus características son:

- Simulación dinámica: Permite acceder a varios motores de física de alto rendimiento.

- Gráficos 3D avanzados: Gazebo proporciona una representación realista de diferentes entornos en los que se puede incluir iluminación, sombras y texturas con una alta calidad.

- Sensores y ruido: El simulador también permite generar datos con ruido como telémetros láser, cámaras 2D y 3D, sensores de estilo Kinect, sensores de contacto, sensores de fuerza y torsión, etc.

- Complementos: Es posible desarrollar complementos personalizados para robots, sensores y control ambiental. Los complementos brindan acceso directo a la API de Gazebo.

- Modelos Robot: En Gazebo se puede trabajar con robots que te proporciona el propio programa de simulación como iRobot o TurtleBot, pero también se puede crear un robot propio con formato SFD (Figura 20).

- Conectividad TCP/IP: Se pueden ejecutar simulaciones en servidores remotos e interactuar con la plataforma a través del paso de mensajes basados en sockets mediante Google Protobufs.

- Simulación en la nube: Otra de sus características es que se puede utilizar CloudSim para ejecutar Gazebo en Amazon AWS y GzWeb para interactuar con la simulación a través de un navegador.

- Herramientas en línea: Estas herramientas que nos proporciona el simulador facilitan la introspección y el control de la simulación.

Gazebo en un programa que se integra bien con ROS, sistema operativo para robots basado en Linux del que se hablará a continuación, lo que le permite ser utilizado en la plataforma Robobo [16].

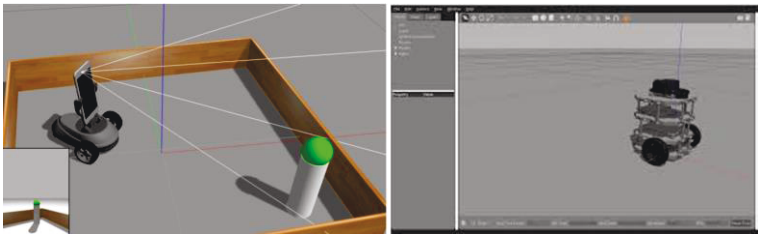


Figura 20. Simulación del Robobo y de TurtleBot en Gazebo. Fuente: [18].

4.4 ROS

Sistema Operativo Robótico (en inglés Robot Operating System, ROS) es una plataforma software, conocida como *framework* , que sirve para el desarrollo de software específico en robótica. ROS fue creado en 2007 por el instituto Willow Garage bajo la licencia BSD. Todos los programas de ROS son de código abierto lo que permite ser usado de manera gratuita tanto en investigación como para fines educativos.

El sistema fue creado teniendo en cuenta otras plataformas software de código abierto, como OpenCv, Gazebo, Orocos/KDL, etc, ya que su filosofía implica que sus programas puedan ser reutilizables, escalables e integrables en todo el software de robótica existente y futuro.

ROS está formado por un conjunto de herramientas de visualización, librerías de programación, aplicaciones y drivers para poder desarrollar aplicaciones que puedan ser usadas en diferentes plataformas robóticas. Por otro lado, ROS debe ser preferiblemente instalado sobre plataformas basadas en UNIX, aunque también es compatible con Linux, Mac Os o Microsoft Windows. Aunque ROS no sea un sistema operativo completo si proporciona ciertos servicios estándares, es por ello que también recibe la denominación de Meta-Sistema Operativo.

Tiene una estructura modular, donde cada programa o aplicación (en ROS conocido como nodo), se comunica con un nodo Master, que funciona como núcleo del sistema y puede realizar la función de plataforma por la que se comuniquen los diversos nodos mediante *topics* o servicios [19].

Sus principales características son las que se citan a continuación [20]:

- Computación distribuida: Actualmente, la mayoría de robots requieren de componentes software que ejecuten diferentes procesos en diferentes ordenadores. ROS proporciona herramientas que facilitan la implementación de software divididos en pequeñas partes independientes que cooperan para lograr un objetivo.

- Reutilización de software: ROS incluye paquetes en los que ya se han implementado y probado versiones estables de algoritmos para robótica.

- Facilidad para la realización de pruebas: Este sistema nos permite distintas alternativas para solucionar los problemas, entre ellos:

    - Separa sistemas de bajo nivel de los de alto nivel, lo que permite que los programas de bajo nivel y el software correspondiente pueda ser reemplazado por un simulador y así poder centrarse en probar la parte de alto nivel.

    - Permite ahorrar tiempo ya que proporciona una manera fácil de grabar y reproducir datos.

- Popularidad: Una de las principales ventajas es que ROS es usado por una gran cantidad de personas, y esto implica que los repositorios sean constantemente actualizados. Además, si existe un error o mal funcionamiento también permite que pueda ser corregido en la plataforma y actualizarla constantemente para poder trabajar con el hardware más avanzado.

A continuación, veremos los distintos elementos que conforman la red de comunicación de ROS:

- Nodos: Los nodos son la unidad de procesamiento, y como se ha dicho anteriormente es un ejecutable dentro de un paquete ROS.

- Topics: Son canales lógicos que identifican el contenido de un mensaje, pero son los nodos los que pueden enviar y recibir mensajes. En la Figura 21 podemos ver el patrón de diseño. Por una parte, tenemos el Publisher que es el que publica mensajes en un determinado *topic*; y por otro lado está el *subscriber*, se subscribe a mensajes de un determinado *topic*. Posibilitan un modo de comunicación muchos -a- muchos en el que los nodos participantes no necesitan conocerse.

- Mensajes: Los mensajes son estructuras de datos simples que se pasan entre los nodos.

- Servicios: Es una implementación de la arquitectura cliente/servidor entre nodos. Utilizan dos tipos de mensajes, uno en la solicitud y otro en la respuesta, de manera que cuando el cliente realiza una solicitud, el servidor realiza un procesamiento y responde directamente al cliente (Figura 21).

- Roscore: Es una colección de nodos y programas que deben ser lanzados antes que el resto de elementos y que implementan el núcleo de ROS.

- ROS Bag: Son archivos para guardar y volver a ejecutar datos de comunicación en Ros. Esto permite memorizar una serie de datos y reutilizarlos repitiéndolos secuencialmente.

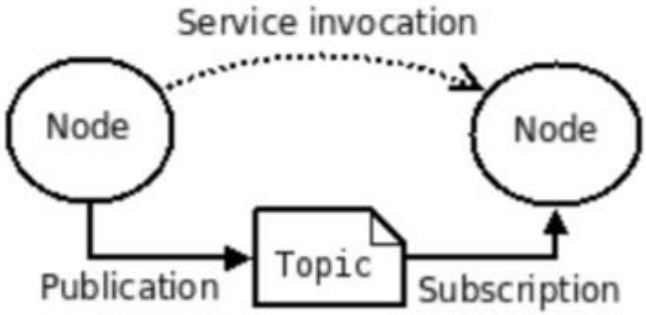


Figura 21. Modelos de comunicación soportados por ROS. Fuente: A partir de [20]**.**

5 ANÁLISIS GLOBAL Y ESPECIFICACIÓN INICIAL DE REQUISITOS

A la hora de trabajar con robots es importante tener un modelo disponible para la simulación con el que poder realizar pruebas cuando no dispones de un entorno físico con el que trabajar, además de constituir una importante herramienta para la investigación de robótica autónoma. No obstante, obtener un modelo simulado con un comportamiento fiel a la realidad, requiere la realización de experimentos que analicen de forma pormenorizada la respuesta de los sensores/actuadores del robot en el entorno simulado y en el entorno real.

Este proyecto pretende realizar estos experimentos con el robot real Robobo y su modelo de simulación en el simulador Gazebo, con el objetivo de detectar y caracterizar las desviaciones de comportamiento del modelo simulado respecto al robot real en distintos escenarios de uso, así como proponer y desarrollar mejoras que sean realizables dentro del alcance de este proyecto.

Actualmente el modelo de simulación en Gazebo se encuentra todavía en desarrollo, es decir, no contiene todavía la implementación de todos los sensores disponibles en el robot real. Esto implica que no se podrán evaluar desviaciones de comportamiento para todos los sensores, pero comenzar a realizar esta evaluación en este momento permitirá no sólo llevar a cabo las medidas correctoras necesarias en los sensores que están ya implementados, sino también establecer un protocolo a seguir para la evaluación continuada durante toda la fase de desarrollo del simulador.

Siguiendo la metodología establecida para el proyecto se realizarán las pruebas y desarrollos incrementales para el análisis del comportamiento de los sensores y actuadores del robot. Para llevar a cabo este análisis se hará uso de herramientas tales como Gazebo, un simulador para robots del que se hablará más adelante. También será necesaria la elaboración de programas en Python y ROS que ayuden a obtener los resultados de cada una de las pruebas.

Tras un análisis inicial del proyecto, se establecen los requisitos necesarios para abordar los objetivos planteados.

- Requisitos funcionales:
    - REF1: Generación de programas de prueba en Python fácilmente reutilizables para evaluar y caracterizar de forma individual los sensores/actuadores relacionados con motores, cámara e infrarrojos.
    - REF2: Analizar y documentar los resultados de evaluación y caracterización de los sensores/actuadores probados en REF1.
    - REF3: Generación de programas de prueba en Python fácilmente reutilizables para evaluar y caracterizar los sensores/actuadores relacionados con motores, cámara e infrarrojos considerado el efecto de condiciones del entorno.
    - REF4: Analizar y documentar los resultados de las pruebas realizadas en REF3.
    - REF5: Generación de programas de prueba para evaluar el comportamiento del robot en un escenario complejo concreto.

  - REF6: Realizar las modificaciones necesarias en el modelo de simulación de Gazebo para posibilitar la implementación de experimentos multi-robot.
  - REF7: Implementar un experimento multi-robot que ejemplifique las posibilidades de desarrollo de comportamientos colaborativos de robótica autónoma en un escenario de conducción autónoma.
  - REF8: Disponer de un escenario de pruebas adecuado en el que poder evaluar individualmente los sensores y actuadores del robot.
  - REF9: Disponer de un escenario de pruebas adecuado en el que poder evaluar conjuntamente los sensores y actuadores del robot.

- Requisitos no funcionales:
  - RENF1: Los programas de control autónomo de los robots deben ser programados en lenguaje Python y ser fácilmente portables entre un entorno real y uno simulado en Gazebo.
  - RENF2: Soportar el lenguaje de programación ROS de forma idéntica al uso en el robot real.

6 DISEÑO E IMPLEMENTACIÓN

Como ya se ha comentado anteriormente, las pruebas se han realizado en dos entornos distintos, uno real y uno simulado. Sin embargo, se ha intentado que ambos entornos tengan la máxima similitud posible para poder realizar las comparaciones necesarias.

De esta manera podemos dividir las pruebas en dos partes: las realizadas con robots en espacios reales, y las realizadas mediante un programa de simulación. Los resultados obtenidos en ambos entornos serán comparados y analizados en cada una de las pruebas realizadas.

Las pruebas, que serán explicadas a continuación, a su vez han sido divididas en cuatro partes:

- Pruebas básicas de comportamiento de sensores y actuadores individuales.
- Pruebas en entornos variables no controlados.
- Pruebas en escenarios específicos.
- Pruebas multi robot.

La idea de estas pruebas es ir incrementando la dificultad, de manera que se pueda observar las diferencias entre los distintos entornos desde el nivel más básico, en el que no se tendrá en cuenta ningún tipo de factor externo, hasta el nivel más alto en el que participará más de un robot. Así se podrá realizar un análisis detallado de las similitudes y las discrepancias del comportamiento del robot entre ambos entornos.

Las pruebas básicas tienen como objetivo validar los principales sensores y actuadores sin tener en cuenta factores externos como la luz, el rozamiento, el problema a resolver, etc. A partir de estas, se realizarán otras pruebas donde entren en juego un conjunto de factores externos dependientes del entorno dónde se realicen.

Estas pruebas básicas y en entornos variables no controlados han sido realizadas con el objetivo de estudiar de manera individual la respuesta de estos sensores y actuadores en ambos contextos para poder finalmente ser usados de forma conjunta en un escenario y con un objetivo concreto como es la ciudad para tareas de conducción autónoma.

Para la obtención de unos resultados que nos ayuden a lograr los objetivos propuestos al principio del trabajo, se han diseñado las pruebas teniendo en cuenta que es lo que se quiere probar en ellas. Mediante programación y la repetición de experimentos se han documentado cada una de ellas para poder ser analizadas y poder determinar si la simulación es fiel a la realidad. Estas serán explicadas en detalle en cada apartado para poder entender el resultado obtenido.

Antes de exponer las pruebas realizadas, es necesario hablar de los escenarios que han sido utilizados. Por ello, en los siguientes apartados se mostrarán los distintos escenarios y se hablará de las aportaciones en su construcción.

6.1 Entorno real

Para las pruebas realizadas en el entorno real, se han utilizado distintos escenarios. Las pruebas básicas y las pruebas en entornos no controlados han sido realizadas en las instalaciones del laboratorio del Grupo Integrado de Ingeniería de la UDC. En dicho laboratorio se ha utilizado una mesa blanca como superficie (Figura 22), así como distintos objetos de colores y señales necesarios para la realización del proyecto.



Figura 22. Laboratorio del Grupo Integral de Ingeniería. Fuente: Elaboración propia.

Para las pruebas en entornos concretos y pruebas multi-robot, ha sido necesario la creación de una maqueta de una ciudad. En la Figura 23 y en la Figura 24 se pueden ver las distintas etapas de la construcción del modelo y su resultado final. A la hora de su construcción se ha tenido en cuenta factores externos como la iluminación y su posible incidencia en el funcionamiento de algunos sensores, como se verá más adelante.



Figura 23. Construcción ciudad. Fuente: Elaboración propia.

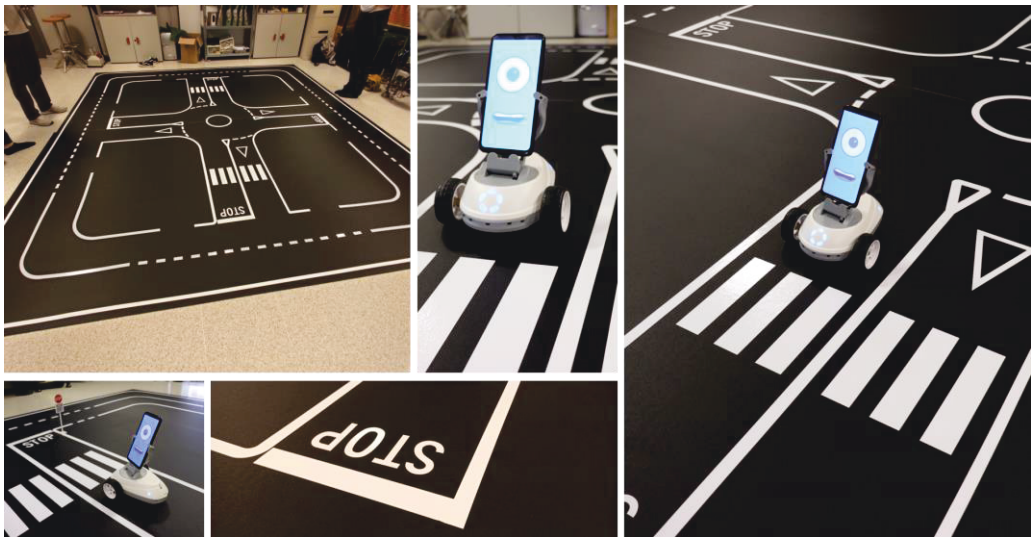


Figura 24. Ciudad acabada. Fuente: Elaboración propia.

El material utilizado para la construcción de la maqueta de la ciudad ha sido el vinilo. Para ello se han dispuesto dos láminas de vinilo unidas para conseguir una superficie de 3,5 metros de ancho por 4 metros de alto. En ella se han colocado cuatro zonas habilitadas para cuatro superficies, dos carriles de 26 cm de ancho para que el robot circule en ambos sentidos, dos pasos de cebra y una rotonda. El objetivo de la elaboración de esta maqueta es poder llevar a cabo estudios del comportamiento que podría tener un robot en una ciudad real. El plano que se ha seguido para su confección corresponde a la Figura 25.

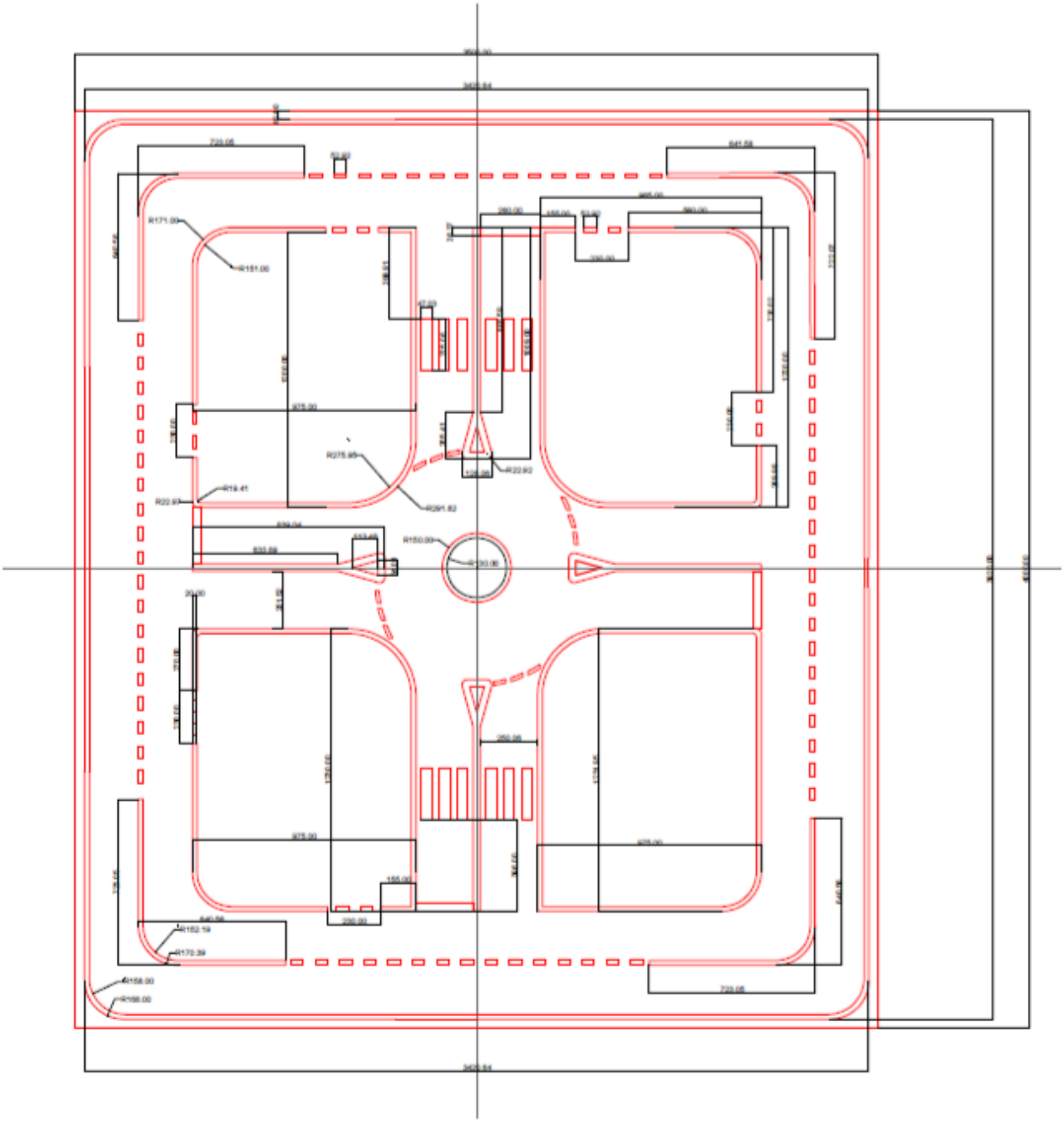


Figura 25. Plano maqueta ciudad. Fuente: Elaboración propia.

6.2 Entorno simulado

Para la realización de las pruebas en el entorno simulado, han sido necesarios diferentes escenarios. Las pruebas básicas y en entornos no controlados se han llevado a cabo en una simulación de una mesa de uno de los laboratorios del Grupo Integrado de Ingeniería, utilizada para el entorno real y que durante la explicación de las pruebas llamaremos “Mundo 1”. En la Figura 26 se puede observar el modelo utilizado.

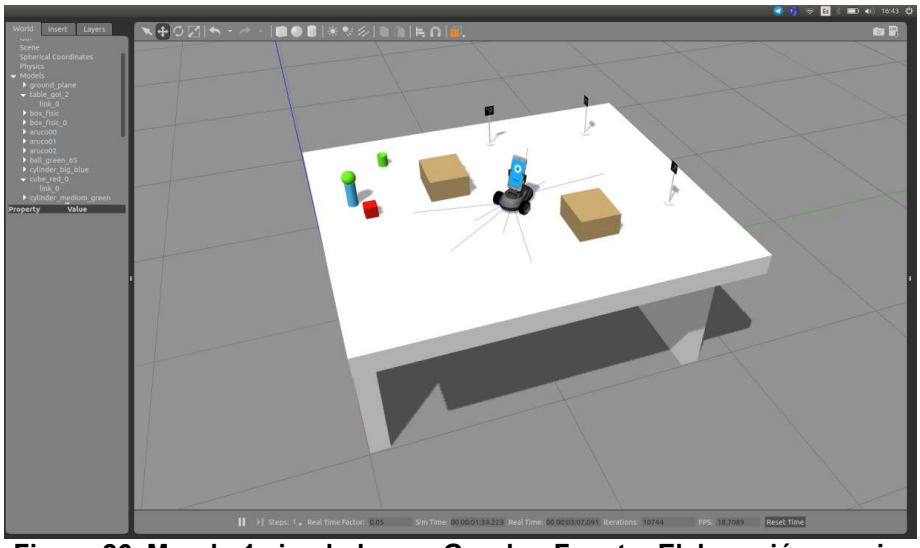


Figura 26. Mundo 1 simulado con Gazebo. Fuente: Elaboración propia.

Este escenario es una recreación de la Figura 22, con el que se pretende validar y caracterizar de forma individual los sensores y actuadores del robot en un entorno simulado.

Por otro lado, se ha llevado a cabo la elaboración de un mundo, al que llamaremos “Mundo 2” para simular la maqueta de la ciudad, mostrada en el apartado de entorno real. Para su realización se ha utilizado el programa de SolidWorks usando el plano de la Figura 25 como base. En la Figura 27 se muestra una imagen con una parte de su proceso de elaboración. Lo primero que se hizo fue realizar un rectángulo con las medidas indicadas en el plano, y darle la textura deseada. A continuación, se le añadieron las líneas blancas que corresponden a las líneas de la carretera y la acera. Una vez creada la pieza, esta fue exportada en formato STL. A continuación, en la Figura 28 se muestra una imagen del “Mundo 3”.

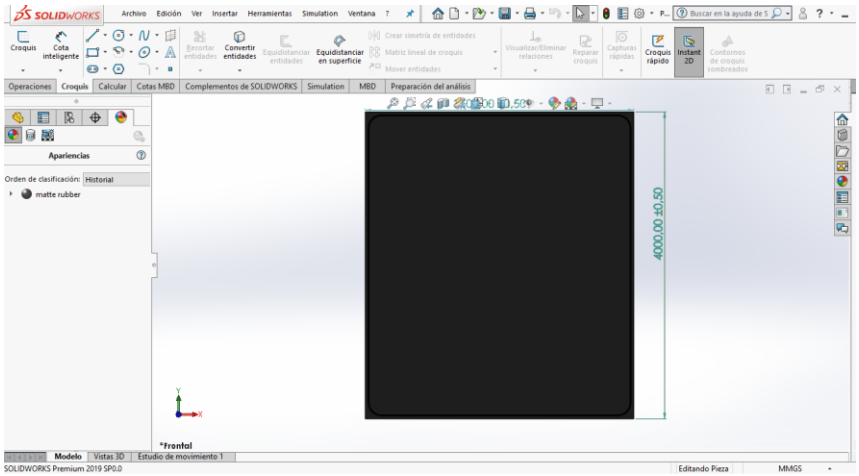


Figura 27. Elaboración del modelo simulado de la maqueta de la ciudad en SolidWorks. Fuente: Elaboración propia.

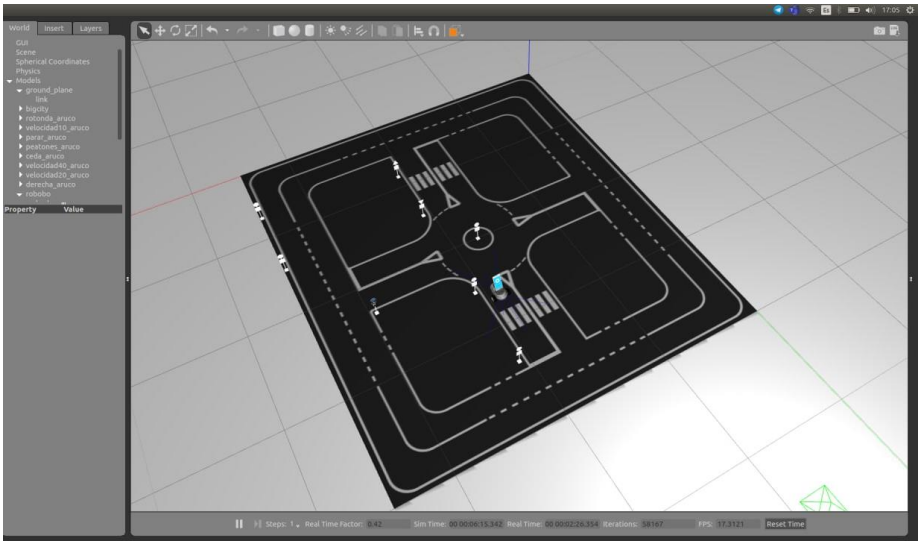


Figura 28. Mundo 3 simulado con Gazebo. Fuente: Elaboración propia.

Con el Mundo 3 se pretende evaluar conjuntamente el funcionamiento de los sensores y actuadores estudiados previamente de forma individual en ambos entornos para proponer mejoras que ayuden a perfeccionar la fidelidad entre escenario real y simulado.

6.3 Pruebas básicas

Como se ha visto en el punto de fundamentos tecnológicos, el Robobo posee una serie de sensores y actuadores que pueden ser evaluados sin tener en cuenta el escenario de aplicación en el que se encuentre.

El objetivo de las pruebas básicas es validar y caracterizar cada uno de los sensores y actuadores principales de forma independiente en el entorno simulado y el real, con el objetivo de compararlos y extraer unas conclusiones acerca de su funcionamiento. Los sensores que se van a caracterizar son: la cámara y concretamente algunas de las funciones que tiene, los motores y los sensores infrarrojos del Robobo. Para ello se han planteado una serie de pruebas que permiten estudiar y comparar el comportamiento en ambos entornos:

- Para los motores, las pruebas propuestas tratan de evaluar como es el funcionamiento de los motores, tanto su velocidad, como la facilidad que tenga para poder girar el robot cuando este sea el objetivo deseado. Para ello se ha dispuesto un circuito cerrado en el que el robot debe volver al mismo punto una vez terminado el trayecto.

- La cámara tiene distintos tipos de funciones que permiten detectar objetos, líneas, colores, etc. Para este proyecto y en concreto estas pruebas ha interesado validar la detección de marcadores ArUco para evaluar como de buena es su detección pues esta función será utilizada en el proceso final de toma de datos de pruebas. Para ello se han planteado unas pruebas en las que el robot permanece estático y otras en las que el robot se encuentra en movimiento.

- Por último, la detección de infrarrojos también es importante a la hora de trabajar con robótica autónoma porque evita que el robot pueda detenerse antes de colisionar. Para caracterizar este sensor, se han realizado dos tipos de pruebas que permiten estudiar su comportamiento en estático y en dinámico.

Estas pruebas serán explicadas más en detalle en cada uno de los siguientes apartados, así como los valores resultantes de su estudio.

6.3.1 Motores

Esta prueba consiste en que el robot realice un circuito cerrado, en este caso, con la forma de un cuadrado. Se han dispuesto para ello cuatro tramos rectos a la misma velocidad y el mismo tiempo y cuatro giros con la misma rueda, la misma velocidad y el mismo tiempo. De esta manera el robot debería de volver al mismo punto de partida. Esto se ha comparado en ambos entornos, real y simulado y a continuación se muestran los resultados obtenidos.

❖ Entorno real

El escenario utilizado en la realización de estas pruebas básicas ha sido el laboratorio del Grupo Integrado de Ingeniería. Se han realizado cinco pruebas en las que se han obtenido resultados semejantes. También se han llevado a cabo unas pruebas cero, para ajustar los tiempos necesarios para que el robot realice el giro deseado.

En los tramos rectos, se le ha asignado una velocidad de 10 a cada rueda y un tiempo de 5 segundos para que realice el movimiento. Para los giros, a la rueda derecha se le asignado una velocidad de 10, mientras que a la izquierda se le ha asignado velocidad nula. En cuanto al tiempo asignado para que realice el giro se han probado distintas medidas: 5, 2,

3, 4, 3.75, 3.65 y 3.7. Finalmente, el tiempo utilizado en los tramos de giro ha sido 3.7, pues es el tiempo con el que mejor realiza un giro de 90º.

A continuación, en la Figura 29, se puede observar un esquema de la trayectoria que debe seguir el Robobo. En este caso hay que tener en cuenta el rozamiento de las ruedas con el suelo, y el ángulo de inicio en el que se coloca el robot.

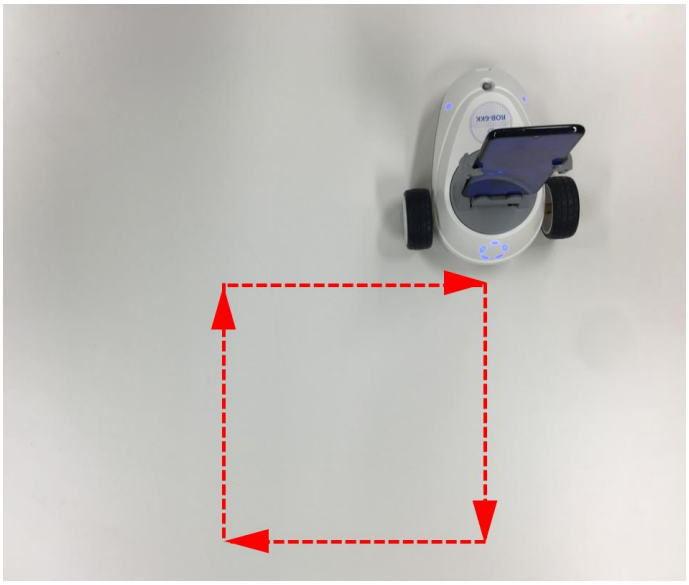


Figura 29. Trayectoria que debe recorrer el Robobo. Fuente: Elaboración propia.

Una vez realizadas las cinco pruebas, los resultados obtenidos fueron los siguientes: Como podemos ver en la Figura 30 y en la Figura 31, se ha realizado una marca previa, para ver la situación exacta desde la que partía el Robobo. En estas figuras podemos ver la posición del robot previa a la trayectoria y su posición tras finalizar la misma. Al finalizar la trayectoria, este no termina exactamente dónde comenzó, pero si bastante cerca. Esto es debido en su mayoría al factor de rozamiento del que hablábamos. Este rozamiento, provoca que el primer giro no se haga de manera completa, es decir sea un giro menor de 90º. Este error se va acumulando en cada giro y provoca que, al terminar el recorrido, el robot no llegue al punto exacto desde el que partió.

Por otro lado, a la hora de realizar el recorrido, también influye la posición de partida del robot al inicio de este. Es importante colocar el dispositivo en la medida que se minimice la desviación frente a la trayectoria para disminuir errores en los giros.

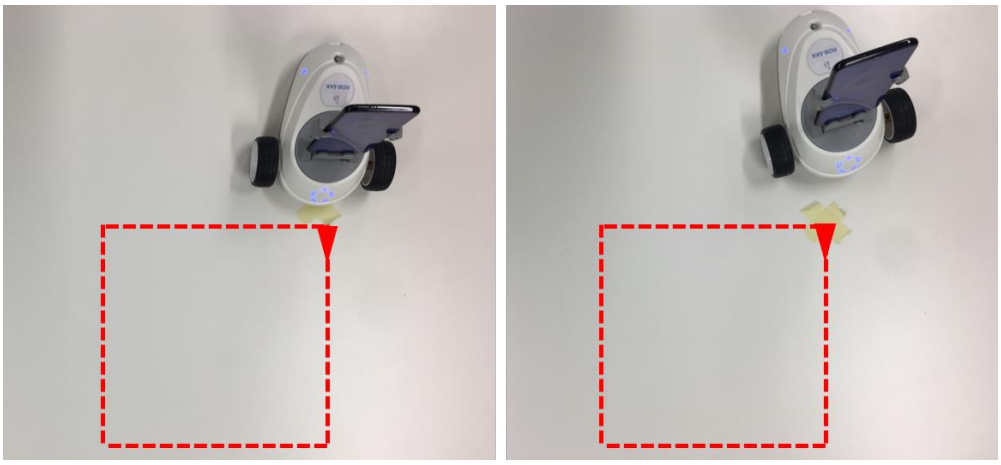


Figura 30. Posición del Robobo antes y después de realizar el circuito (vista en planta) en entorno real. Fuente: Elaboración propia.

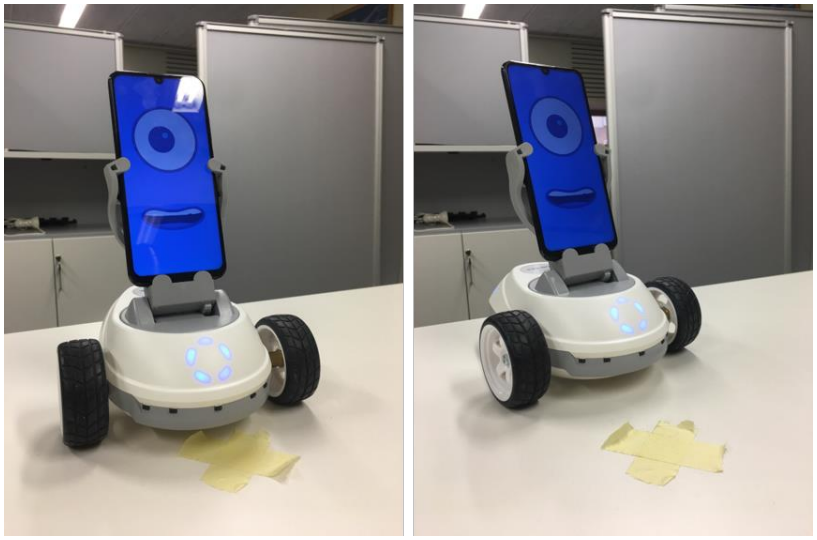


Figura 31. Posición del Robobo antes y después de realizar el circuito (vista frontal). Fuente: Elaboración propia.

❖ Entorno simulado

La misma prueba ha sido realizada para el entorno simulado. En este caso, el mundo utilizado ha sido el Mundo 1. En la Figura 32 y en la Figura 33 se muestra una imagen previa al movimiento del Robobo y una imagen posterior a la trayectoria.

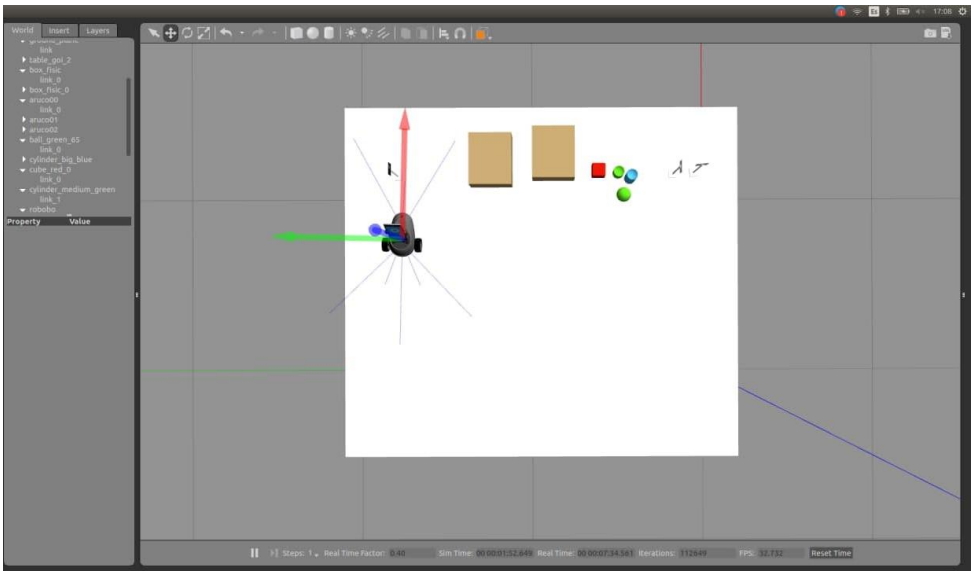


Figura 32. Posición del Robobo antes de realizar el circuito en entorno simulado (Mundo 1). Fuente: Elaboración propia.

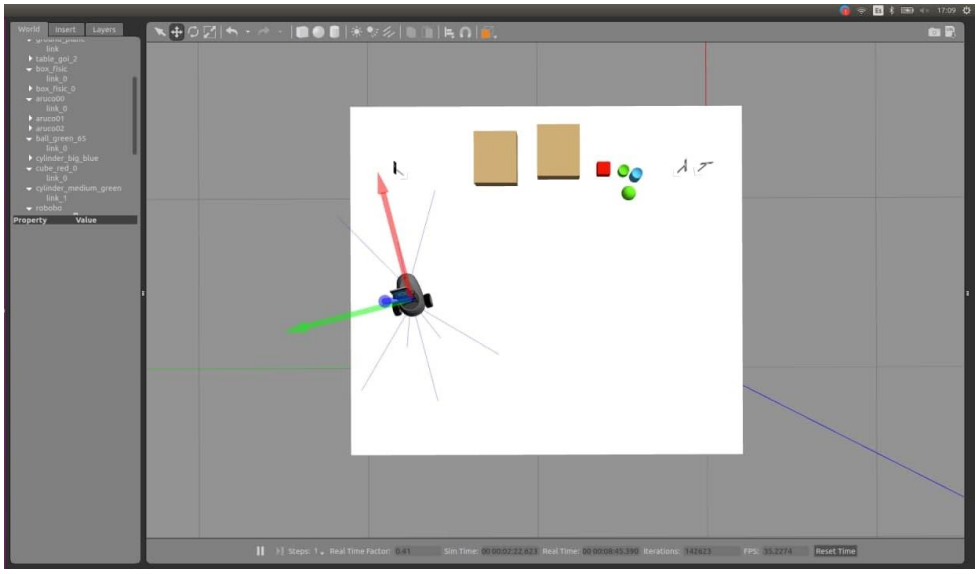


Figura 33. Posición del Robobo después de realizar el circuito en entorno simulado (Mundo 1). Fuente: Elaboración propia.

Como se puede observar, el Robobo no ha logrado hacer la trayectoria marcada. Esto puede deberse al deslizamiento de las ruedas con la superficie de la mesa. El giro no ha sido el deseado en ningún momento de la trayectoria y por ello, se ha modificado el programa inicial, para comprobar si con un tiempo mayor para la realización del giro, el robot terminaba en una posición más cercana a la de inicio. Sin embargo, el resultado no ha sido el esperado. Partiendo de que el tiempo inicial era 3,8 segundos, y las velocidades para cada rueda eran 0 para la izquierda y 10 para la derecha, los tiempos con los que se han realizado las siguientes pruebas, han sido: 5, 7.5, 10, 12.5 y 15. Ninguno de estos tiempos ha resultado exitoso, por ello finalmente se ha probado con un tiempo de 30 segundos en vez de seguir aumentando el tiempo ya que no vale la pena que tarde uno o varios minutos en realizar cada uno de los giros. Su posición final tras modificar el tiempo de cada giro a 30 segundos ha sido la que se muestra en la Figura 34.

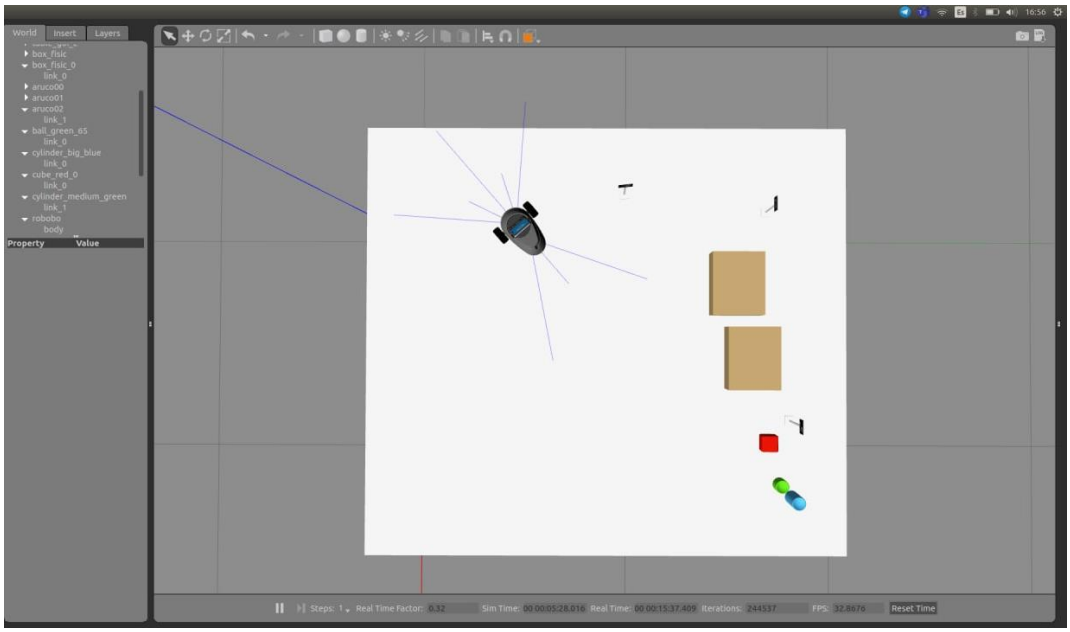


Figura 34. Posición del Robobo después de realizar el circuito en entorno simulado modificando el tiempo de giro (Mundo 1). Fuente: Elaboración propia.

Como podemos observar, su posición final está mucho más cerca de la inicial, pero dista bastante de la posición final del Robobo realizando el mismo recorrido en un entorno real. Tras estos resultados y dado que estos movimientos serán necesarios para futuras pruebas, se ha comunicado el error y este ha sido resuelto. Una vez actualizado el simulador de Gazebo y realizadas las mismas pruebas, el resultado obtenido se muestra en la Figura 35 y en la Figura 36.

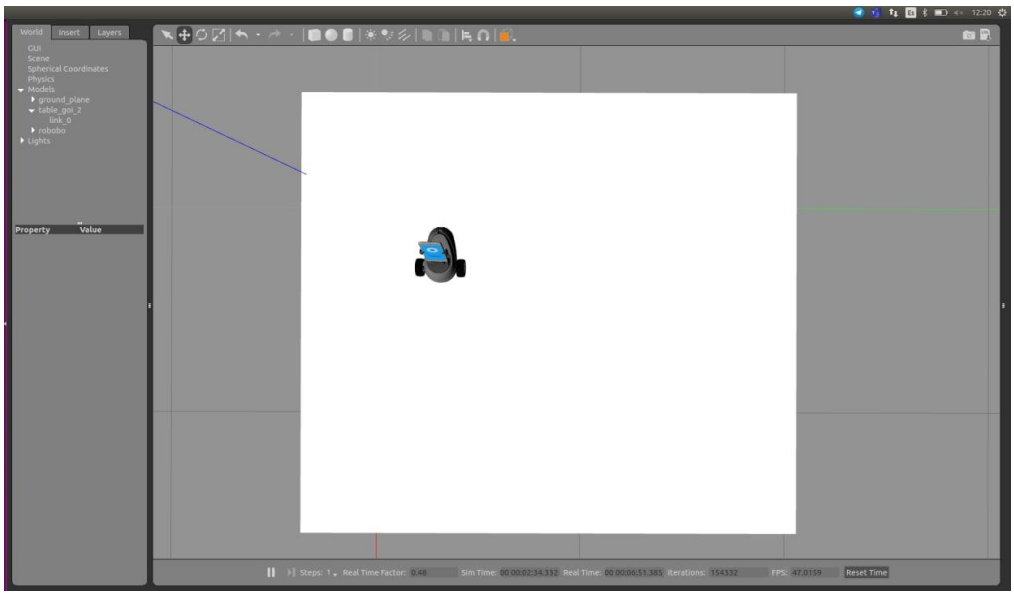


Figura 35. Posición del Robobo antes de realizar el circuito en el entorno simulado tras la actualización del simulador. Fuente: Elaboración propia.

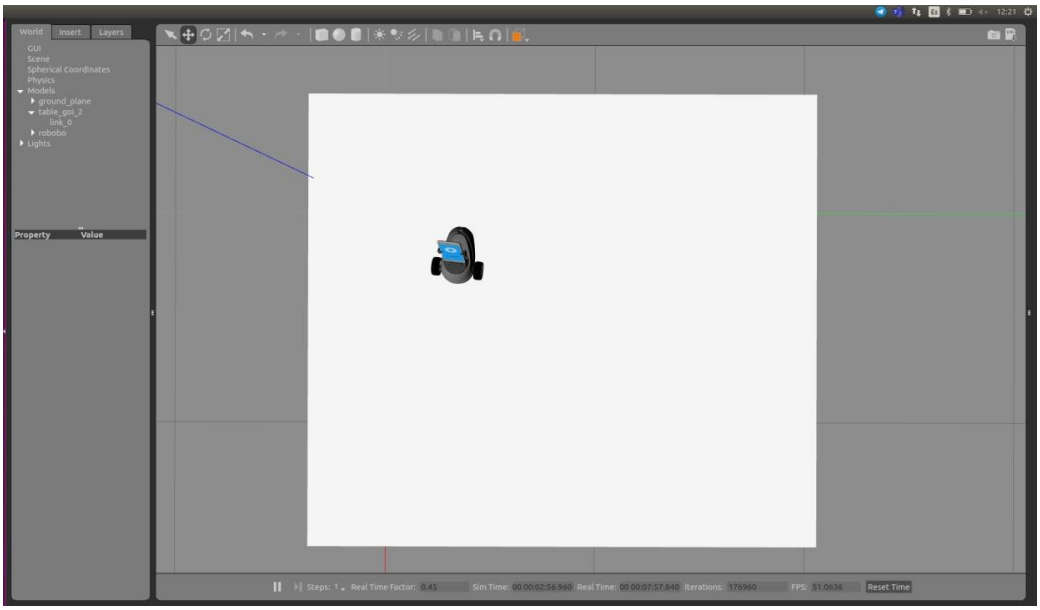


Figura 36. Posición del Robobo después de realizar el circuito en el entorno simulado tras la actualización del simulador. Fuente: Elaboración propia.

Como se observa en las imágenes, el Robobo termina prácticamente en el mismo sitio en el que comenzó, aunque ha sido necesario modificar los parámetros de la velocidad de cada rueda. Estos han sido -20 para la rueda izquierda y 20 para la rueda derecha con un coeficiente de rozamiento de la superficie de 1 y un tiempo para la realización del giro de 3,8 segundos.

❖ Comparación y conclusiones

Una vez arreglados los errores de movimiento encontrados en las primeras pruebas, podemos determinar que los actuadores del Robobo en el entorno simulado trabajan de forma similar a los del robot real, en la Tabla 1 se muestra una comparación de los parámetros que intervienen en este movimiento en ambos escenarios.

Tabla 1. Comparación entre los parámetros de ambos entornos para el mismo movimiento para la superficie blanca. Fuente: Elaboración propia.

Parámetros	Entorno real	Entorno simulador
Velocidad de ambas ruedas en el movimiento recto	10	20
Velocidad de la rueda izquierda en el giro	0	-20
Velocidad de la rueda derecha en el giro	10	20
Tiempo necesario para realizar cada giro	3,8	3,8

Como se muestra en la tabla anterior, la velocidad de ambas ruedas es el doble en el entorno simulado, debido a que el movimiento de este es mucho más lento. Para el giro, la diferencia de la velocidad entre ambas ruedas en el simulador es el cuádruple que en el robot real y el tiempo necesario para realizarlo es el mismo en ambos entornos. Podemos

determinar así que, aunque el simulador consiga realizar el mismo giro que en el entorno real, el movimiento en este es mucho más lento. Algo que se tendrá en cuenta para las próximas pruebas.

Además, este tipo de pruebas son también interesantes para, desde un punto de vista educativo, poder formar a los alumnos con aspectos que deben considerar a la hora de realizar programas para un robot real que puedan pasarse después a un simulador.

6.3.2 Cámara

La cámara es otro de los sensores que va a ser evaluado, ya que se requiere de su participación en futuras pruebas. Por ello es necesario validar su funcionamiento tanto en un entorno real como en un entorno simulado y compararlo con datos reales. Para llevar a cabo su caracterización, han decidido realizarse dos tipos de pruebas diferentes.

Utilizando el sensor de la cámara se pueden usar diferentes funciones de detección, como detección de colores, detección de señales, detección de objetos y, detección de marcadores ArUcos entre otras. Para el desarrollo del presente trabajo ha interesado caracterizar la detección de marcadores ArUcos, pues como se ha comentado anteriormente, esta será usada en pruebas futuras.

La validación de la detección de marcadores ArUcos, se realizará mediante dos pruebas diferentes, en las que se pretende estudiar la distancia a la que pueden ser leídos estos marcadores y cómo afectará la velocidad a la que vaya el robot en su detección. Para ello se han dividido las pruebas en dos tipos:

- Pruebas en las que el robot permanecerá estático, y lo que se variará es la distancia de los ArUcos.

- Pruebas dinámicas en las que el robot mantendrá una trayectoria, cambiando las distintas velocidades, para observar cómo afecta la velocidad en su detección.

A continuación, se muestran los resultados obtenidos.

❖ Entorno real

Para comenzar con la caracterización del sensor de la cámara se han realizado las pruebas en las que el robot permanece estático. Para ello, se ha dispuesto el Robobo en una posición fija, y se ha ido modificando la distancia del ArUco. Previamente a la explicación del proceso y los resultados, hay que valorar que el método de detección de ArUcos del entorno simulado no devuelve una distancia, si no el área del ArUco. Cuanto más cerca esté este mayor será el área. Es por esta razón por la que, aunque el método de detección de ArUcos en el entorno real sí devuelve una distancia, se ha creado un programa para que también devuelva el área. De esta forma se podrán comparar los resultados obtenidos en ambos entornos de una manera más clara y efectiva.

La realización de estas pruebas se ha llevado a cabo de la siguiente forma: se han determinado las distancias que querían ser evaluadas, y mientras el robot permanecía en el mismo punto, el marcador ArUco se ha ido colocando a distintas longitudes. En la Tabla 2 se muestra el valor del área obtenida para cada medida.

Tabla 2. Área que devuelve el ArUco en las pruebas en entorno real. Fuente: Elaboración propia.

Distancia del ArUco (cm)	Área que devuelve (px2)
30 cm	10181,20
40 cm	5873,35
60 cm	2324,70
80 cm	1211,48
100 cm	766,03
120 cm	497,96

Con los valores obtenidos, se han ajustado los puntos a distintos polinomios de interpolación con el objetivo de encontrar una relación entre el área en pixeles en la imagen y la distancia real. Para ello graficamos los datos resultantes. En la Figura 37, analizamos cuatro posibles polinomios de interpolación que se adaptan a nuestro problema: siguiendo una tendencia exponencial, logarítmica, potencial y cuadrática. Comparando los coeficientes de determinación (R^2) de cada una de ellas, podemos observar que la tendencia potencial es la que mejor se adapta a nuestros valores, ya que tiene un coeficiente de 0,9994.

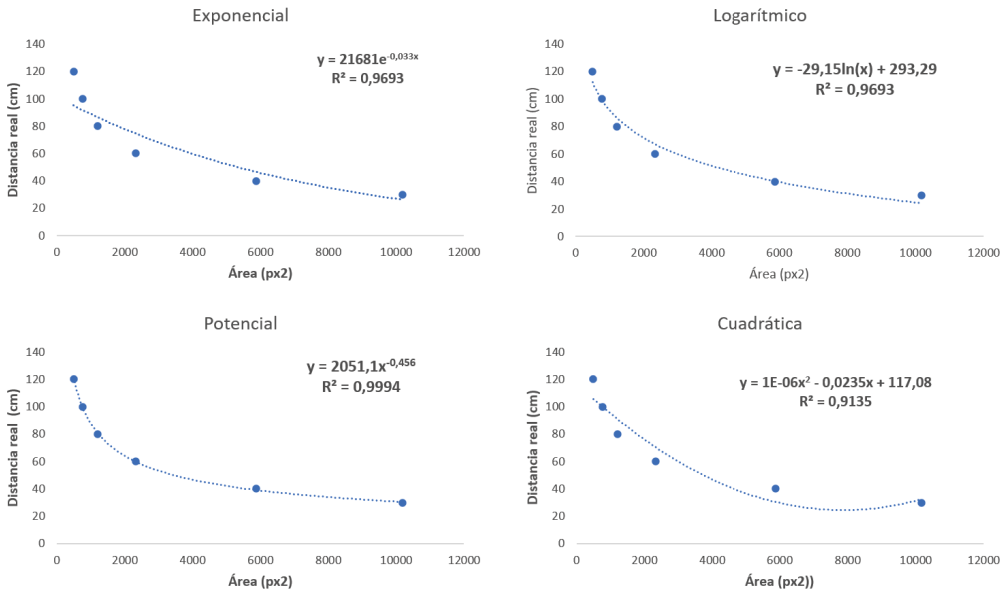


Figura 37. Polinomios de interpolación con tendencia exponencial, logarítmico, potencial y cuadrática para las pruebas realizadas en el entorno real. Fuente: Elaboración propia.

La Ecuación (1) de interpolación que nos da la gráfica potencial es la siguiente:

$$y = 2051{,}1\, x^{(-0456)} \tag{1}$$

De esta manera, podemos calcular la distancia del ArUco, pues x se sustituirá por el área obtenida en cada una de las distancias. En la siguiente tabla, se muestran los resultados

(Tabla 3). Como se puede observar, la distancia obtenida tras la interpolación es prácticamente la misma distancia real a la que se colocó el ArUco.

Tabla 3. Distancia obtenida después de la interpolación en las pruebas en entorno real. Fuente: Elaboración propia.

Distancia del ArUco (cm)	Área que devuelve (px2)	Distancia obtenida (cm)
30 cm	10181,20	30,509178
40 cm	5873,35	39,208017
60 cm	2324,70	59,830622
80 cm	1211,48	80,536847
100 cm	766,03	99,259198
120 cm	497,96	120,79979

Una vez estudiada la distancia a la que puede ser leída el ArUco y como se ajusta al valor obtenido por el sensor, pasamos a analizar como varía esta precisión en función de la velocidad a la que va el Robobo. Para ello, se ha dispuesto un pequeño circuito con tres marcadores ArUcos, uno de giro a la derecha, otro de cambio de velocidad y otro de stop. La señal de giro a la derecha ha sido colocada a una distancia de 56 cm de la de cambio de velocidad y la señal de stop se ha colocado a 12 cm del final de la mesa, ya que tomaremos este como punto de referencia y por lo tanto será nuestra coordenada cero. El Robobo ha sido colocado a una distancia de 40 cm de las señales. En la siguiente figura se muestra un esquema del circuito.

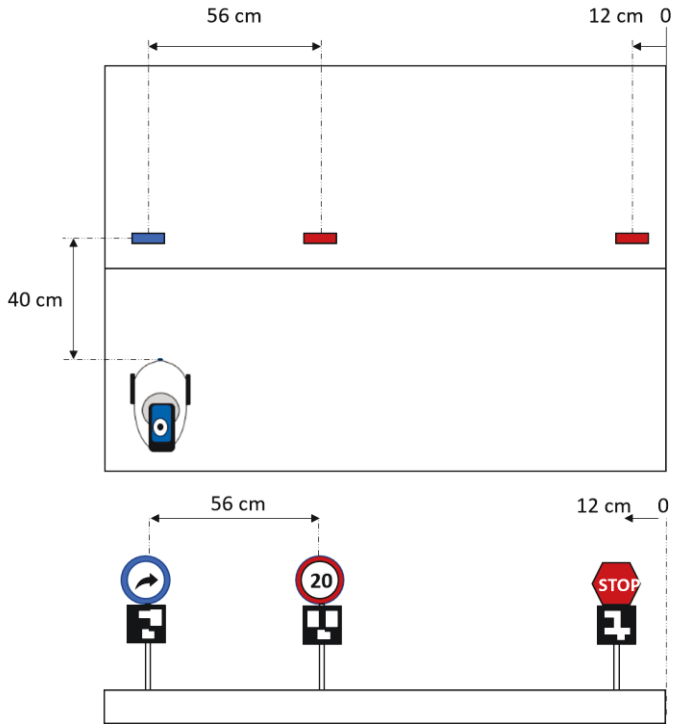


Figura 38. Esquema del circuito para las pruebas dinámicas de detección del ArUco. Arriba: Vista en planta. Abajo: Vista frontal. Fuente: Elaboración propia.

Estas pruebas se han realizado con distintas velocidades: 5, 10, 15, 20, 25 y 30 y para cada una de ellas se han realizado cuatro repeticiones. El objetivo de esta prueba consiste en comprobar si la velocidad influye a la hora de detectar un marcador ArUco y cuanto afecta a esta detección, es decir, se ha comprobado la distancia a la que se detiene el Robobo respecto a la coordenada cero una vez que ha detectado el stop para cada velocidad. La distancia a la que se detiene el Robobo se ha medido desde la coordenada cero hasta el punto de contacto de la rueda con el suelo, pues la cámara del smartphone (que es la que detecta el ArUco) está inclinada con un tilt de 100. Por tanto, si se traza una vertical desde el punto de la cámara hasta el suelo, coincide con el punto de contacto de la rueda con la superficie.

Durante el circuito, el robot tiene que detectar primeramente la señal de giro a la derecha y tras realizar el giro se moverá 5 segundos. Una vez pasados estos 5 segundos, el robot girara la pantalla hacia la izquierda (*pant*) con un ángulo de 100 y a una velocidad de 50. Esto se ha realizado así porque como observamos en la Figura 38 las señales están colocadas en una horizontal, de esta forma, aunque la trayectoria del Robobo sea paralela a esta horizontal, podrá seguir detectando los marcadores ArUcos. El hecho de que el robot se mueva unos segundos tras realizar el giro y se detenga para voltear su pantalla hacia la izquierda, es para que este no detecte de nuevo la señal de giro a la derecha, y vuelva a realizar un giro de 90º debido que a una distancia de 40 cm el espectro visual que capta el robot es más reducido. A continuación, se muestra la tabla de datos obtenida.

Tabla 4. Distancia a la que se detiene el Robobo respecto a la coordenada 0 al leer la señal de STOP en el entorno real. Fuente: Elaboración propia.

Velocidades	Distancia respecto a la coordenada 0 (cm)
5	13,5
	14
	12
	14,5
10	13
	14
	11,5
	11,5
15	11
	13
	9,5
	11,5
20	8,5
	6
	10
	7,5
25	4,5
	6
	4,5
	5,5
30	2
	3,5
	2,5
	4

Para poder analizar mejor los datos obtenidos, se ha hecho una media de las distancias para cada velocidad y se han comparado en una misma gráfica. Como se observa en la Figura 39, a mayor velocidad, más lento es el tiempo de respuesta y por lo tanto el robot tarda más en detener sus motores una vez detectado el ArUco.

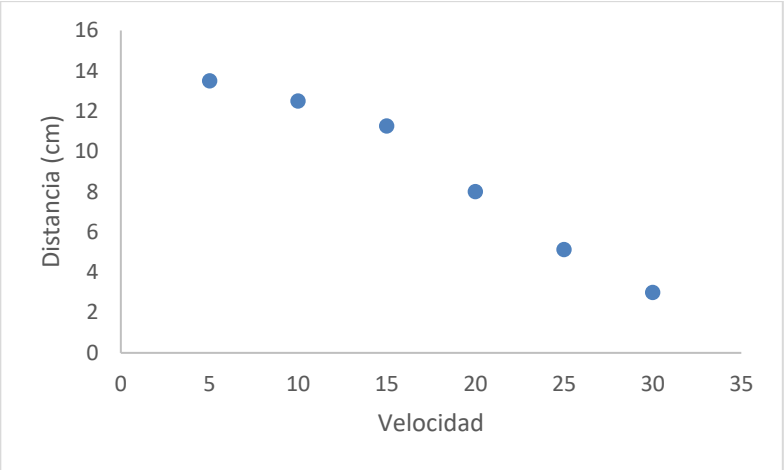


Figura 39. Representación de la velocidad frente al espacio respecto a la coordenada cero en las pruebas obtenidas con el robot real. Fuente: Elaboración propia.

❖ Entorno simulado

Las pruebas realizadas en el entorno simulado han sido abordadas de la misma manera que en el entorno real y el mundo utilizado ha sido el Mundo 1. En la Figura 40 se puede observar la obtención del área del ArUco a distintas distancias: 30, 80 y 120 centímetros. En el extremo superior derecho de cada figura se muestra la imagen que está viendo la cámara frontal del smartphone y en ella el ArUco que detecta.

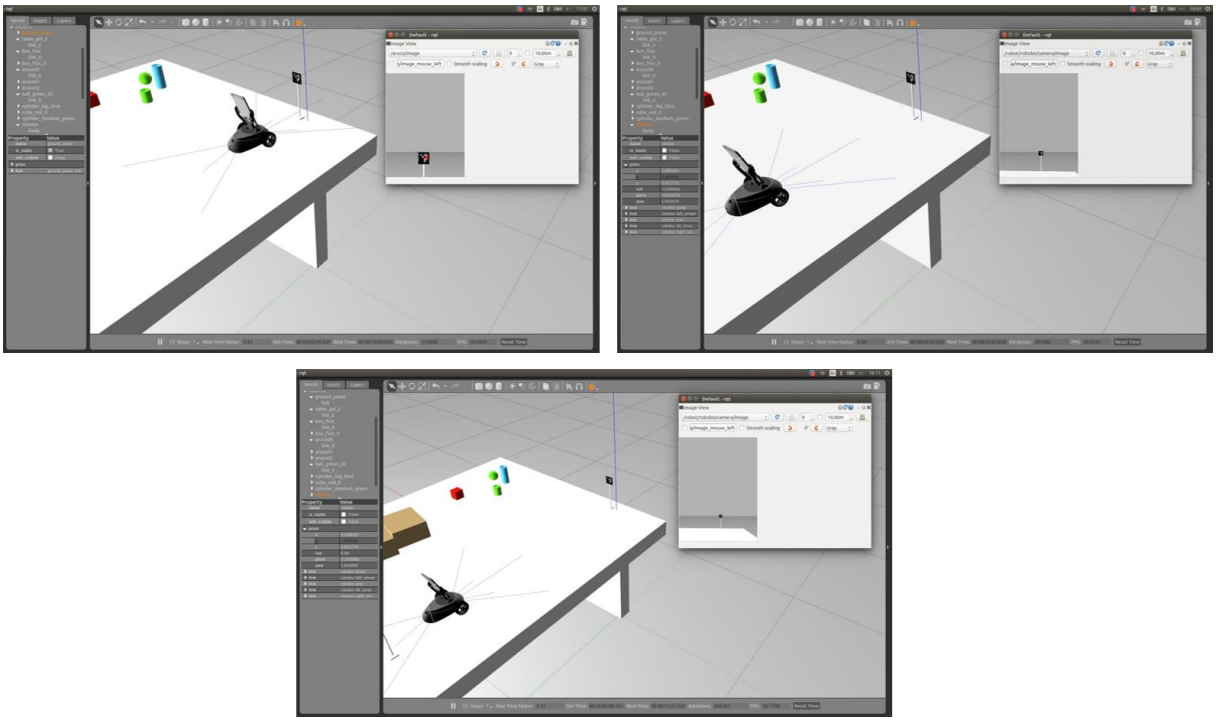


Figura 40. Toma de datos a diferentes distancias. Arriba de izquierda a derecha: 30 cm y 80 cm. Abajo: 120 cm. Fuente: Elaboración propia.

En Gazebo, se puede modificar la situación del objeto variando los parámetros de x, y y z. En este caso, el marcador ArUco ha sido colocado en una posición fija con coordenadas

x=0.5 y=0 y z=0.9 y el Robobo se ha situado con coordenadas x=0.5 y z=0.9, variando la coordenada del eje y, de esta forma el robot se encuentra centrado respecto a la posición del ArUco. Los datos obtenidos se muestran en la Tabla 5. Y a continuación se realiza el mismo procedimiento que para el entorno real.

Tabla 5. Área que devuelve el ArUco en las pruebas en entorno simulado. Fuente: Elaboración propia.

Distancia del ArUco (cm)	Área que devuelve (px2)
30 cm	4746,78048
40 cm	2800,0
60 cm	1528,0
80 cm	728,46413
100 cm	483,0
120 cm	323,44

Con los valores de la Tabla 5, se han ajustado los puntos a distintos polinomios de interpolación con para determinar la relación que existe entre el área en pixeles en la imagen del robot simulado y la distancia real. En la Figura 41 analizamos las cuatro tendencias que nos ofrecen los diferentes polinomios de interpolación. Podemos observar que el polinomio de interpolación con mayor coeficiente de determinación (R^2) es, como en el caso anterior el de potencial dado que ofrece un coeficiente de 0,9942.

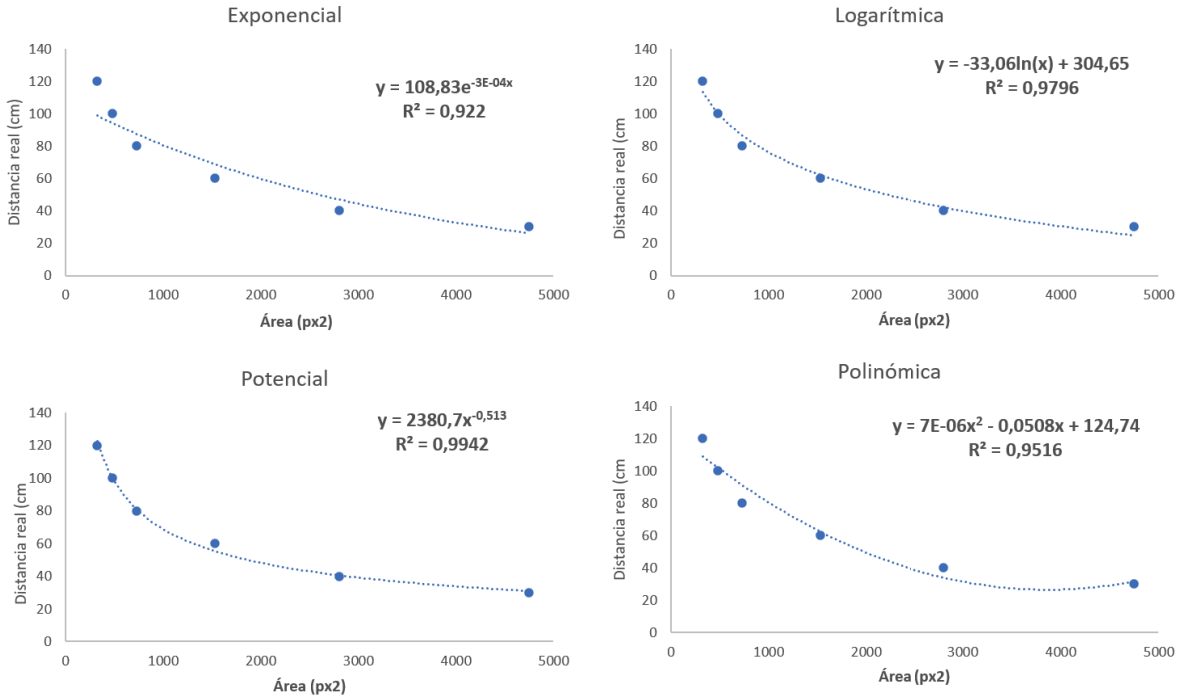


Figura 41. Polinomios de interpolación con tendencia exponencial, logarítmico, potencial y cuadrática para las pruebas realizadas en el entorno simulado. Fuente: Elaboración propia.

La Ecuación (2) de interpolación que nos da este polinomio es:

$$y = 20380{,}7\ x^{(-0{,}513)} \quad (2)$$

Por tanto, en la Tabla 6 comprobamos que la distancia obtenida en el entorno simulado se acerca bastante a la distancia real.

Tabla 6. Distancia obtenida después de la interpolación en las pruebas en entorno simulado. Fuente: Elaboración propia.

Distancia del ArUco (cm)	Área que devuelve (px2)	Distancia obtenida (cm)
30 cm	4746,78048	30,95365296
40 cm	2800,0	40,58005225
60 cm	1528,0	55,36679587
80 cm	728,46413	80,96350106
100 cm	483,0	99,96311798
120 cm	323,44	122,7949451

Una vez calculada la distancia según el área del ArUco que devuelve el sensor del Robobo, describimos las pruebas realizadas para determinar si la velocidad afecta en la lectura del marcador ArUco. Para su realización, se ha tomado como base el esquema de la Figura 38, en la que el Robobo tiene que hacer un pequeño circuito a distintas velocidades siguiendo las indicaciones de cada marcador ArUco, y detenerse en la última señal. En la Figura 42, se muestra la simulación de la prueba realizada en el Gazebo con una imagen de lo que está viendo la cámara del Robobo

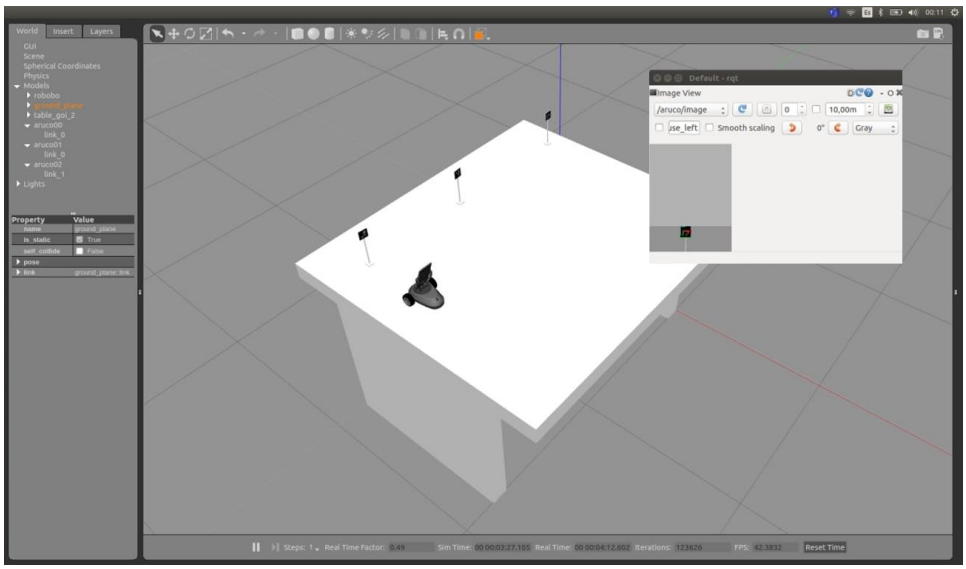


Figura 42. Simulación de la prueba de detección de ArUcos a distintas velocidades en el simulador. Fuente: Elaboración propia.

Para su realización se ha utilizado el Mundo 1, en el que se han colocado tres señales con marcador ArUco. La primera y la segunda señal están separadas 56 cm y la última está colocada a 12 cm del final de la mesa, que tomaremos como coordenada cero. El procedimiento para la toma de datos es el siguiente; el Robobo, que al inicio estará colocado a 40 cm de la primera señal, una vez lea la señal de STOP, debe detenerse y se anotará la distancia que existe entre el Robobo y la coordenada cero. Para su determinación se ha tomado el punto de apoyo de la rueda como coordenada, dado que la cámara está inclinada con un valor de *tilt* de 100 y la vertical que traza hasta el suelo coincide con ese punto. En la siguiente tabla se muestran los resultados obtenidos.

Tabla 7. Distancia a la que se detiene el Robobo respecto a la coordenada 0 al leer la señal de STOP en el simulador. Fuente: Elaboración propia.

Velocidades	Distancia respecto a la coordenada 0 (cm)
5	30,7088
	32,1784
	32,9454
	34,1634
10	30,1362
	30,7785
	32,1286
	30,5479
15	31,0144
	30,4279
	30,1254
	29,0449
20	29,8443
	30,3589
	30,5966
	29,9141
25	29,7202
	28,4072
	28,8587
	30,3384
30	30,7146
	31,2127
	31,1823
	29,5276

Para la toma de datos, se han realizado cuatro pruebas para cada velocidad y las velocidades de estudio han sido: 5,10,15, 20, 25 y 30. En la Figura 43 se muestra una gráfica con la media de los datos obtenidos para cada medida. Como se puede observar, cuanto menor es la velocidad más distancia existe entre el Robobo y la coordenada cero, por lo que si el Robobo va a menor velocidad detecta antes la señal y se detiene más instantáneamente que cuando lleva una velocidad más alta. También hay que sumarle que cuando va a mayores velocidades existe una mayor inercia que hace que el robot se estacione unos centímetros más tarde.

También podemos observar que para la velocidad 30 las distancias son mayores que para la velocidad 25. Esto posiblemente se deba a que el movimiento del robot en el simulador no es recto, pues este tiende a desviarse unos centímetros de la trayectoria, y si la distancia entre el punto de partida y el de llegada es muy alta, esta desviación es más notoria.

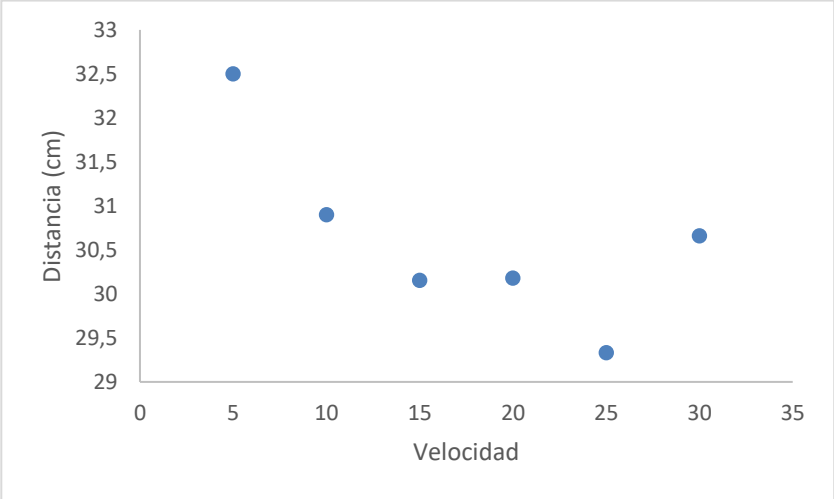


Figura 43. Representación de la velocidad frente al espacio respecto a la coordenada cero en las pruebas obtenidas con el simulador. Fuente: Elaboración propia.

❖ Comparación y conclusiones

A continuación, vamos a comparar las pruebas de detección de marcadores ArUco mientras el robot no se encuentra en movimiento. Para poder llevar a cabo una comparación más exacta de los resultados obtenidos en ambos entornos, se ha realizado una tabla en la que se comparan los errores absoluto y relativo de cada uno de ellos.

Si se observa la Tabla 8, podemos ver que los errores absolutos de los valores conseguidos a partir del entorno real son ligeramente menores que los obtenidos en el simulador. Sin embargo, si comparamos los errores relativos de ambas pruebas se observa una diferencia más evidente entre los valores. Esto quiere decir que para las pruebas realizadas en el simulador el polinomio de detección obtenido se ajusta ligeramente peor a la realidad que el obtenido para el entorno real.

En un principio, se había supuesto que el polinomio obtenido con los datos del simulador se ajustaría mejor que el obtenido en el entorno real, puesto que en el simulador influyen menos variables externas.

Tabla 8. Comparativa de errores absolutos y relativos entre entorno real y simulado. Fuente: Elaboración propia.

Error absoluto de la distancia obtenida en el entorno real	Error relativo de la distancia obtenida en el entorno real (%)	Error absoluto de la distancia obtenida en el entorno simulado	Error relativo de la distancia obtenida en el entorno simulado (%)
0,16	0,55%	0,95	3,18%
0,60	1,51%	0,58	1,45%
0,15	0,26%	4,63	7,72%
0,48	0,61%	0,96	1,20%
0,29	0,30%	0,03	0,04%
0,12	0,11%	2,79	2,33%

Por otro lado, se pueden comparar los resultados obtenidos con el Robobo en movimiento. Esta comparación se muestra en la Figura 44. En ella podemos observar que la distancia a la que se detiene el robot una vez leído el marcador ArUco es mayor en el entorno simulado que en el entorno real. En este aspecto, puede haber cierta influencia del entorno como la luz que exista en el ambiente en ese momento. Este tema se tratará más en profundidad en el siguiente punto de pruebas en entornos no controlados.

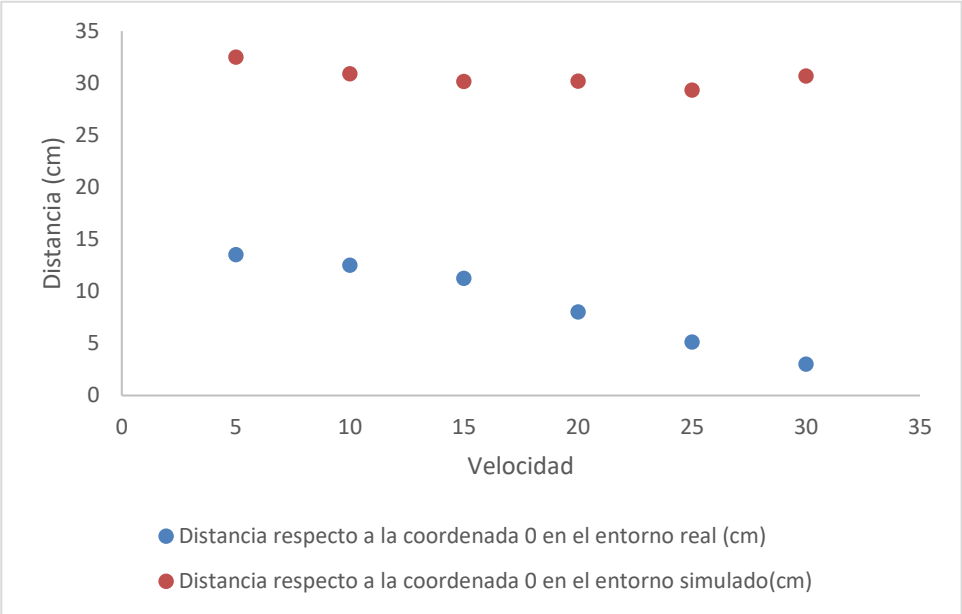


Figura 44. Comparación de los resultados obtenidos en los entornos para las pruebas realizadas con el robot en movimiento. Fuente: Elaboración propia.

6.3.3 Infrarrojos

Los infrarrojos son una parte muy importante de los sensores, ya que nos permiten detectar obstáculos y que el Robobo pueda esquivarlos. Para validar los infrarrojos también se han llevado a cabo dos pruebas: pruebas estáticas y pruebas dinámicas.

En las pruebas estáticas el objetivo es compararla los valores que devuelve el sensor de infrarrojos del robot real, con los valores que devuelve el simulador y a su vez con la distancia a la que se encuentra el objeto. Así se podrá hacer una comparación de la precisión de los infrarrojos en ambos entornos.

En cuanto a las pruebas dinámicas, se le han dado instrucciones al robot de que se detenga cuando el valor que devuelve el sensor de infrarrojos sea inferior a un valor establecido. El objetivo de estas pruebas es el mismo que en las pruebas estáticas, pero además se podrá evaluar la precisión del sensor cuando el robot se encuentre en movimiento, y si este será capaz de pararse a tiempo ante la colisión con un obstáculo.

❖ Entorno real

Para las pruebas estáticas se ha utilizado una pared blanca como fondo (Figura 45) y el Robobo ha sido colocado a distintas distancias con las que se han obtenido los valores que se muestran en la Tabla 9.



Figura 45. Pruebas estáticas básicas para la validación de los infrarrojos en el entorno real. Fuente: Elaboración propia.

Tabla 9. Valores del sensor de infrarrojos obtenidos en pruebas estáticas en el entorno real. Fuente: Elaboración propia.

Distancia del robot (cm)	Valores del sensor de infrarrojos
11 cm	149
14 cm	70
19 cm	41
24 cm	32
34 cm	21
44 cm	17
54 cm	14
69 cm	8

Como podemos observar en la tabla anterior, cuanto más cerca está el objeto mayor es el valor obtenido por el sensor de infrarrojos del robot.

Para las pruebas dinámicas, como se ha explicado anteriormente, el robot ha de permanecer en movimiento hasta que detecte un objeto. Cuando el sensor de infrarrojos devuelva un valor inferior a 30, el Robobo debe de detenerse. Estas pruebas han sido realizadas cinco veces para poder obtener un resultado más concluyente. Después de cada prueba, se ha medido la distancia real a la que se encontraba el Robobo obteniéndose los valores que se pueden ver en la siguiente tabla.

Tabla 10. Valores del sensor de infrarrojos obtenidos en pruebas dinámicas para una superficie blanca. Fuente: Elaboración propia.

Valores del sensor de infrarrojos	Distancia del robot (cm)
32	15 cm
34	17 cm
35	19 cm
34	15 cm
33	17 cm

En la Tabla 10, podemos observar que para la misma distancia a la que se encuentra el robot, los valores que nos devuelve el sensor no son los mismos. Sin embargo, sí que se puede ver una uniformidad en estos valores. El robot suele detenerse a una misma distancia y los valores que arroja el sensor de infrarrojos son bastante próximos.

También podemos observar que, al encontrarse el robot en movimiento, dependiendo de la velocidad que lleve, tendrá una cierta inercia, y por lo tanto es más complicado que este se pare en el momento justo donde se deja de cumplir la condición del bucle.

❖ Entorno simulado

Para la realización de las pruebas en el entorno simulado se ha utilizado el Mundo 1. El objetivo de estas pruebas es el mismo que en el entorno real, obtener los valores que devuelve el sensor de infrarrojos del robot a diferentes distancias y determinar cuál es la tendencia que más se ajusta con los datos experimentales obtenidos mediante el uso de un polinomio de interpolación.

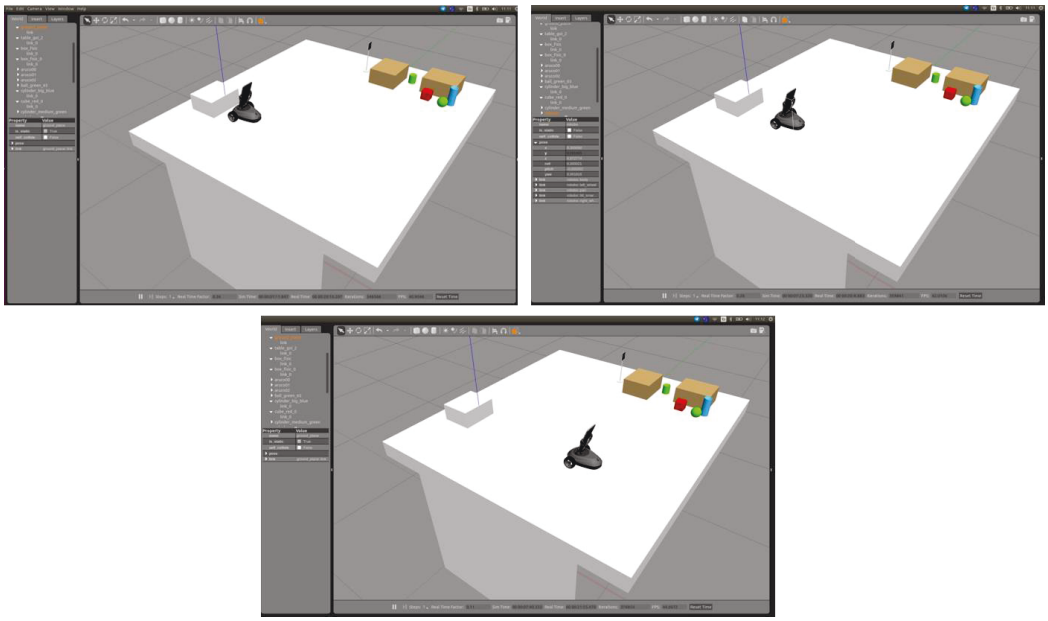


Figura 46. Obtención de datos con el sensor de infrarrojos de Gazebo en el entorno simulado. Fuente: Elaboración propia.

En la Figura 46 se muestra una imagen de la toma de datos en el simulador de Gazebo. Para su obtención, se ha utilizado una caja de color blanco, que se ha colocado en el punto (0, 0, 0,94183) y posteriormente se ha ido modificando la distancia a la que se encontraba el Robobo respecto a la caja. A continuación, se muestra la tabla de datos obtenida.

Tabla 11. Valores del sensor de infrarrojos obtenidos en pruebas estáticas en el entorno simulado. Fuente: Elaboración propia.

Distancia del robot (cm)	Valores del sensor de infrarrojos
11 cm	0
14 cm	89,466827
19 cm	47,295811
24 cm	29,599653
34 cm	17,300091
44 cm	13,627504
54 cm	13,634351
69 cm	13,635187

Como se puede observar en la Tabla 11, cuando el robot se encuentra a 11 cm del objeto, el sensor de infrarrojos no es capaz de detectarlo, y deja de variar el valor cuando este se encuentra a una distancia mayor de 44 cm. En el rango entre 14 y 44 cm los datos arrojados por el sensor sí devuelven un valor usual que puede ser comparando con los conseguidos en el entorno real.

❖ Comparación y conclusiones

A continuación, se muestra una tabla comparando los valores obtenidos en entorno real frente a los obtenidos en el simulado para las mismas distancias (Tabla 12). En ella podemos ver que la información obtenida de las medidas realizadas en el entorno simulado es ligeramente mayor que en el entorno real y a medida que aumenta la distancia al objeto, más se reduce esta diferencia entre ambas medidas. Por su parte en el entorno simulado los infrarrojos parecen tener un rango de funcionamiento óptimo entre 14 y 44 cm. Por debajo de 11 no se detecta ningún valor y por encima de 44 el valor obtenido por el sensor de infrarrojo permanece constante (en torno a 13,6).

Tabla 12. Comparación de los valores del sensor de infrarrojos obtenidos en el entorno real frente al simulado. Fuente: Elaboración propia.

Distancia del robot (cm)	Valores del sensor de infrarrojos en el entorno real	Valores del sensor de infrarrojos en el entorno simulado	Diferencia entre ambas medidas
11 cm	149	0	149
14 cm	70	89,466827	19,466827
19 cm	41	47,295811	6,295811
24 cm	32	29,599653	2,400347
34 cm	21	17,300091	3,699909
44 cm	17	13,627504	3,372496
54 cm	14	13,634351	0,365649
69 cm	8	13,635187	5,635187

6.4 Pruebas en entornos no controlados

Las pruebas llevadas a cabo en este apartado han sido realizadas en distintas instalaciones de la UDC, concretamente en el laboratorio del Grupo Integrado de Ingeniería y el Laboratorio de Robótica. Como se ha comentado anteriormente, en estas pruebas entran en juego factores del entorno que influyen en el comportamiento de los sensores y actuadores como la luminosidad, el rozamiento de la superficie por la que se mueve el Robobo, los colores de los objetos, etc. Es importante tener en cuenta los resultados que se puedan obtener en este punto, ya que nos permitirán mejorar el comportamiento que pueda tener el robot en el simulador para que dichos factores se ajusten lo máximo posible a la realidad.

Para la realización de estas pruebas, se han repetido las pruebas realizadas en el apartado anterior, pero teniendo en cuenta factores externos que puedan influir en su resultado final. Es por ello, que en este apartado también se analizarán de manera individual los mismos sensores y actuadores: motores, cámara e infrarrojos.

6.4.1 Motores

Las pruebas que se han desarrollado para la validación de este actuador son las mismas realizadas en el punto de pruebas básicas, pero en este caso estas han sido realizadas en diferentes superficies, para comparar como afecta el rozamiento de cada una de ellas. Las superficies valoradas en el proyecto han sido:

- Una mesa blanca del laboratorio del Grupo Integrado de Ingeniería.
- Una superficie de madera.
- La superficie de la maqueta elaborada para las pruebas en escenarios concretos.

Además, se ha creído conveniente la realización de unas pruebas en superficies inclinadas, aunque no pueda disponerse de esta función en el simulador. De esta forma podrá valorarse en un futuro la opción de colocar superficies inclinadas en Gazebo. Las superficies inclinadas que serán objeto de estudio son:

- Una superficie de madera.
- Una superficie de cartón plastificado.

❖ Entorno real

La primera superficie de estudio será la de madera. El objetivo de esta prueba es la misma que la del punto de pruebas básicas, es decir, el Robobo debe realizar un circuito de forma cuadrada, y debe llegar a la misma posición. Los valores asignados en cada tramo también son los mismos que los de las pruebas anteriores y también se han realizado cinco repeticiones. En este caso, también ha sido marcada la posición inicial del robot previa a la trayectoria.

Los resultados obtenidos en las cinco pruebas han sido bastante parecidos. En la Figura 47 y en la Figura 48 se puede ver la posición del robot antes de iniciar el circuito y la posición final del mismo. Se puede observar que la posición del Robobo en este caso se acerca mucho más a su posición inicial.

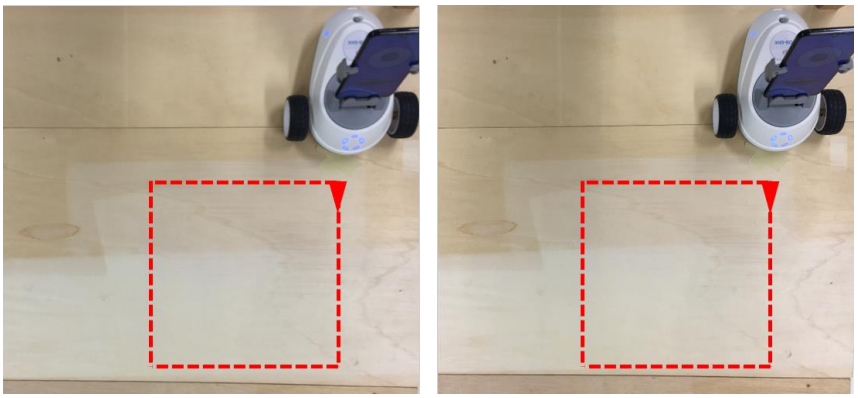


Figura 47. Validación de los motores en entornos no controlados en superficie de madera (vista en planta). Fuente: Elaboración propia.



Figura 48. Validación de los motores en entornos no controlados en superficie de madera (vista frontal). Fuente: Elaboración propia.

Las mismas velocidades y los mismos tiempos han sido utilizados en las pruebas realizadas en la maqueta de la ciudad. También se ha marcado la posición inicial del Robobo y de la misma manera se han realizado cinco repeticiones. En este caso, podemos observar en la Figura 49 y en la Figura 50, que el robot no termina en la posición en la que comenzó. Esto posiblemente se deba al coeficiente de rozamiento con la superficie, aunque también puede haber influido la posición de partida del Robobo previa a la trayectoria, de manera que los errores que se pueden ir acumulando en cada giro que realiza el robot sean mayores.

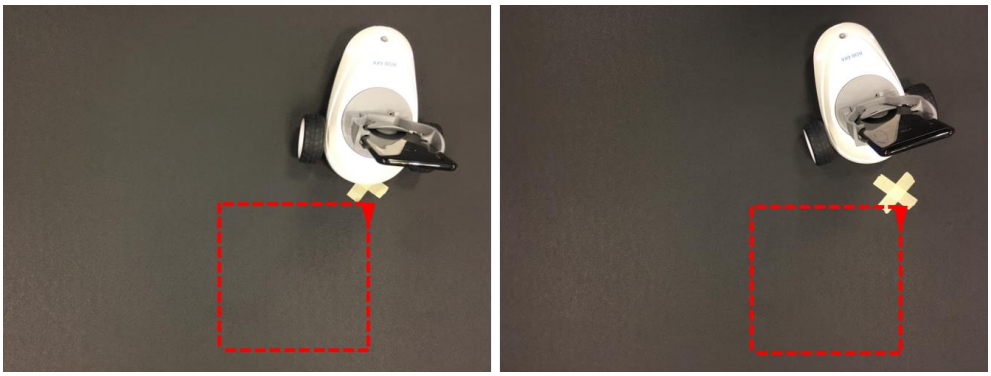


Figura 49. Validación de los motores en entornos no controlados en maqueta de la ciudad (vista en planta). Fuente: Elaboración propia.

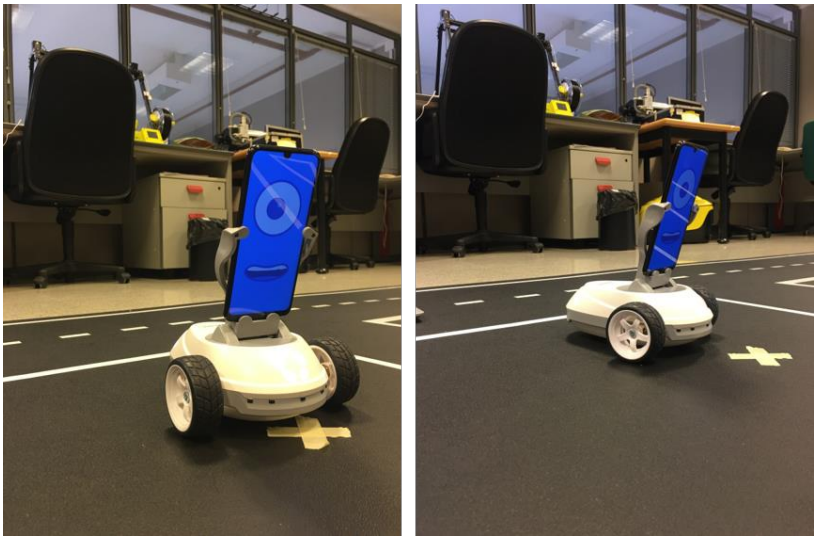


Figura 50. Validación de los motores en entornos no controlados en maqueta de la ciudad (vista frontal). Fuente: Elaboración propia.

Comparando todas las superficies en las que se han realizado las pruebas, podemos determinar que el coeficiente de fricción es clave ya que influye en la trayectoria de todas ellas. Aunque las desviaciones no sean muy grandes, en un circuito en el que el robot debe hacer numerosos giros, esto puede ser un inconveniente a tener en cuenta.

En cuanto a las pruebas realizadas en superficies inclinadas, como se ha comentado anteriormente, se han estudiado dos superficies: una de madera, y otra de cartón plastificado.

Para poder obtener unos resultados más concluyentes, las pruebas en esta superficie han sido realizadas con tres ángulos de inclinación distintos, medidos con el giroscopio del propio smartphone. Estos ángulos son: 4º, 8º y 11º. En la Figura 51, se pueden ver las distintas inclinaciones de la superficie. En la realización de estas pruebas se ha observado que el Robobo es capaz de seguir la trayectoria, pues el coeficiente de fricción de la superficie es lo suficientemente elevado como para que este no se deslice hacia atrás.



Figura 51. Pruebas realizadas en una superficie inclinada de madera con diferentes ángulos de inclinación (4º, 8º y 11º respectivamente) Fuente: Elaboración propia.

Sin embargo, para la superficie inclinada de cartón plastificado (Figura 52) sólo se han realizado pruebas con dos ángulos de inclinación distintos, 4º y 11º, ya que se ha observado que, en ninguno de los casos, el Robobo es capaz de subir la rampa. Esto es debido al material de la superficie, al ser de cartón plastificado el robot desliza y no es capaz de mantener la trayectoria.

Una vez más observamos la importancia de poder trabajar con diversos coeficientes de rozamiento para poder mejorar el comportamiento de los robots tanto en entornos reales como simulados.

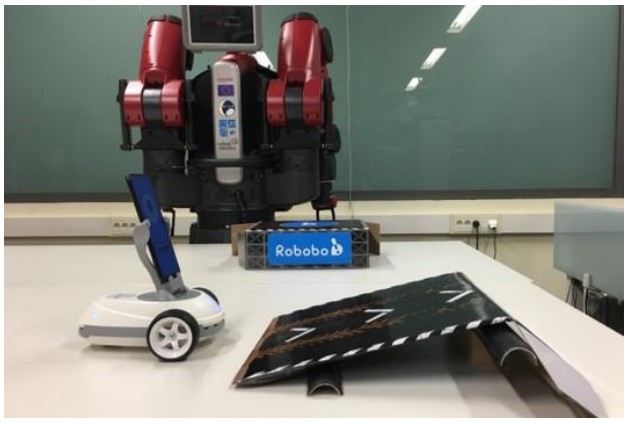


Figura 52. Pruebas realizadas en una superficie inclinada de cartón plastificado con diferentes ángulos de inclinación (4º y 11º respectivamente) Fuente: Elaboración propia.

❖ Entorno simulado

Para la realización de estas pruebas se ha llevado a cabo el mismo planteamiento que en el realizado en el apartado de entorno real. El Robobo debe realizar un circuito de forma cuadrada, en el que se han dispuesto cuatro tramos rectos con un tiempo de 5 segundos, y cuatro giros con un tiempo de 3,8 segundos. Para los tramos rectos, se le ha marcado una velocidad de 10 en ambas ruedas y en los giros, la rueda derecha lleva una velocidad de 10 mientras que la izquierda lleva velocidad nula.

Las pruebas en este apartado han comenzado con el estudio de la superficie de madera. Para su realización, se ha modificado el coeficiente de fricción de las superficies para adecuarlo lo máximo posible a la realidad dado que Gazebo permite esa opción. Para ello previamente se ha buscado dicho valor y una vez abierto el modelo, en este caso el Mundo 1 pues es dónde se dispone de una superficie de madera, se han realizado los cambios pertinentes. Gazebo permite editar el objeto, en este caso la superficie de madera, y en la pestaña de *collision*, modificar el parámetro mu. En la Figura 53 se muestra una imagen del proceso.

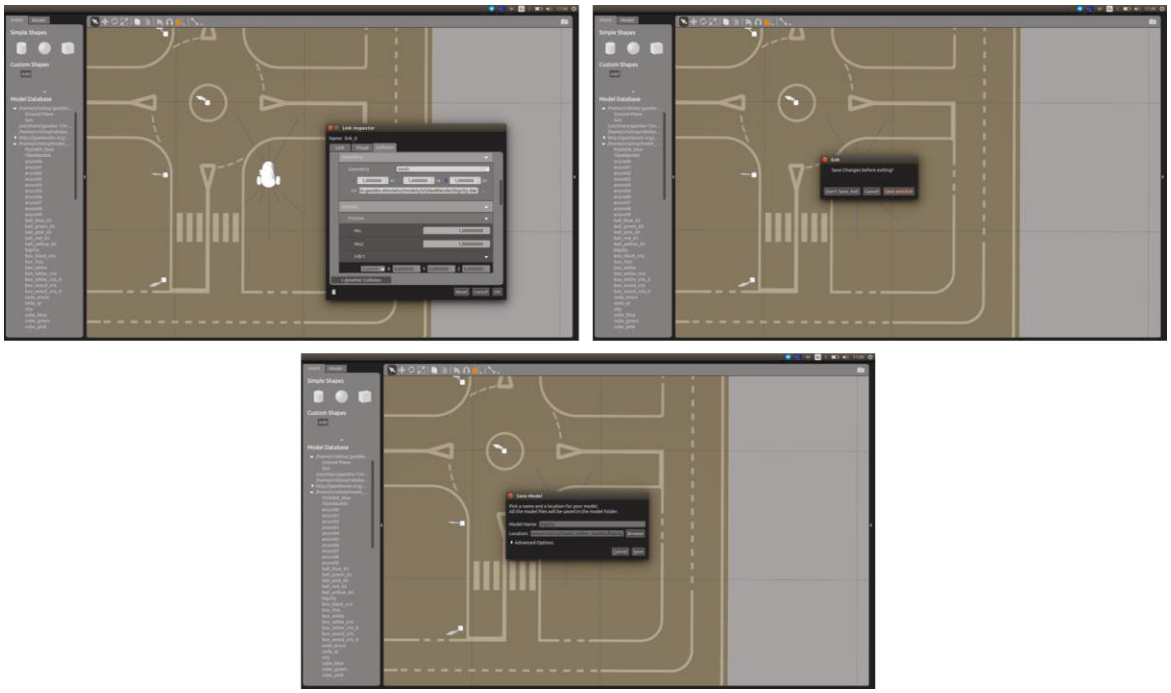


Figura 53. Pasos para la modificación del coeficiente de rozamiento de la superficie. Fuente: Elaboración propia.

Una vez adaptado el coeficiente de fricción, se ha puesto en marcha el movimiento del robot para que este realice el circuito asignado. En la Figura 54 se muestra la posición previa al movimiento del Robobo y su posición final tras realizar el circuito.

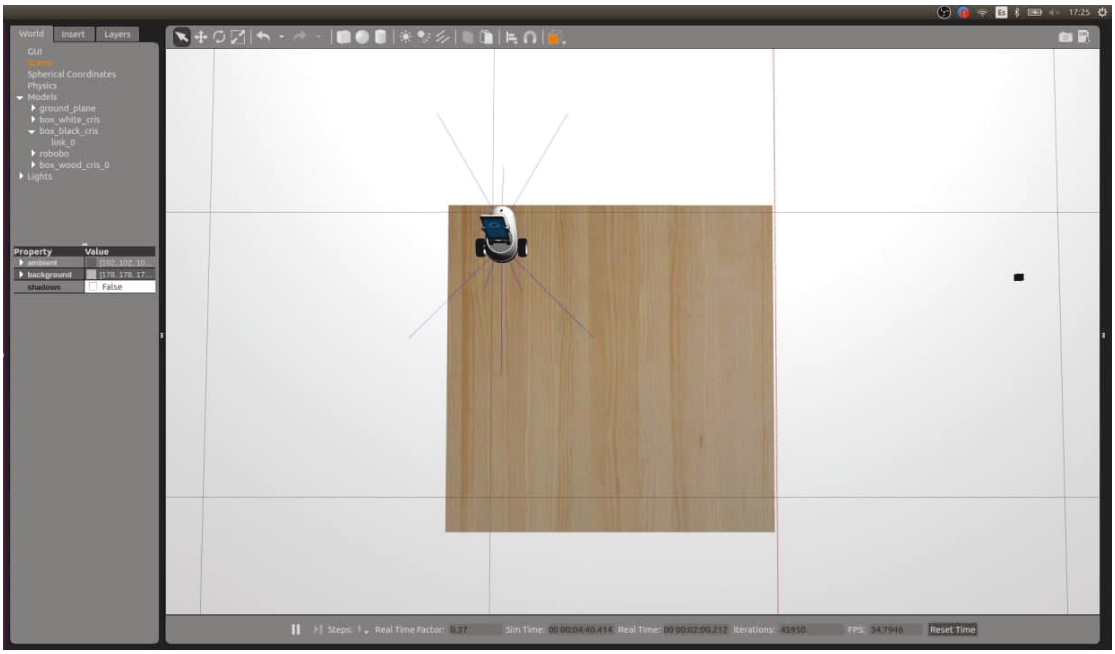


Figura 54. Posición del Robobo antes de realizar el circuito en entorno simulado. Fuente: Elaboración propia.

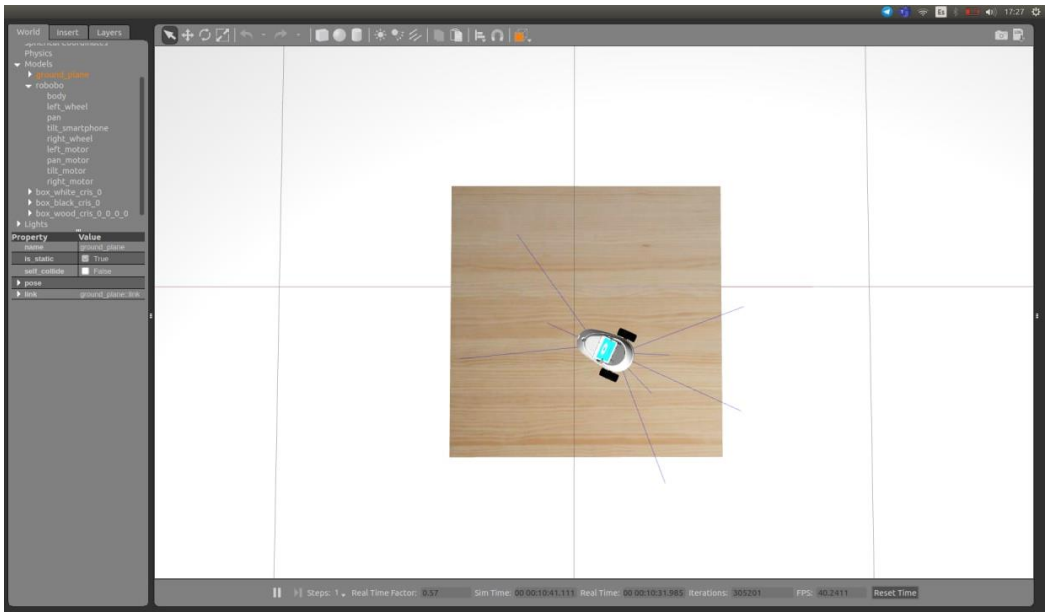


Figura 55. Posición del Robobo tras realizar el circuito en entorno simulado. Fuente: Elaboración propia.

Como se puede observar en la Figura 55, el Robobo termina bastante alejado de la posición inicial, algo similar a lo que ocurría en la superficie blanca, estudiada en el apartado de pruebas básicas (Figura 33). Si nos acordamos, en este mundo fue necesario un estudio posterior para comprobar cuanto tiempo necesitaría el Robobo en el mundo simulado para poder volver a su posición inicial. Este mismo estudio se ha realizado con esta superficie, por lo que se ha probado con los mismos tiempos: 5, 7.5, 10, 12.5 y 15. Sin embargo, como con la superficie blanca, con ninguno de estos tiempos el Robobo logró volver a su posición inicial. Se probó por tanto con un tiempo de 30 segundos para cada giro. La posición final del robot se muestra en la siguiente figura.

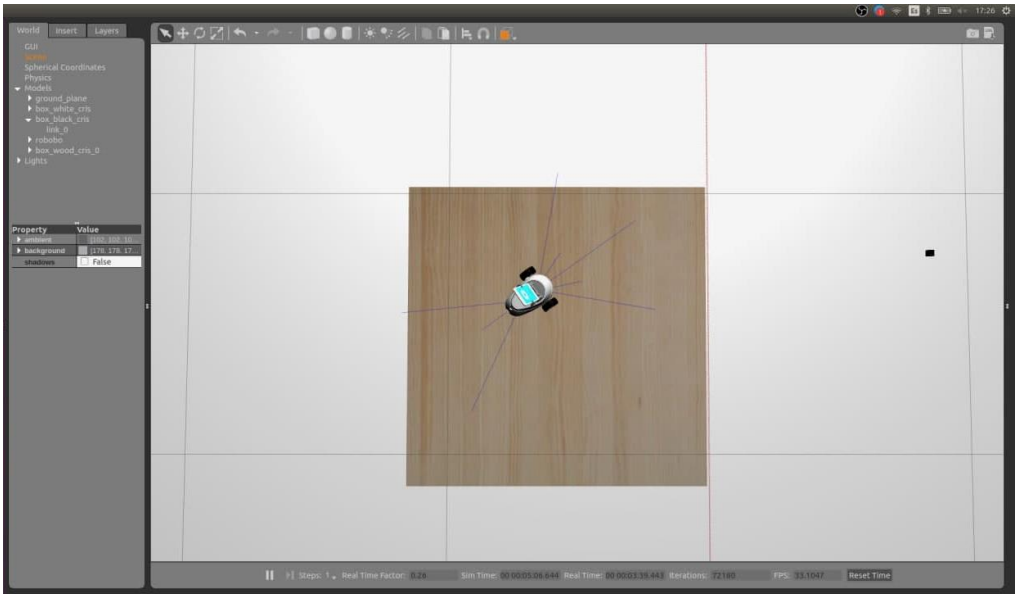


Figura 56. Posición del Robobo tras modificar el tiempo en el que debe realizar cada giro y tras terminar el circuito en entorno simulado. Fuente: Elaboración propia.

Como se muestra en la figura anterior, la posición del robot es más cercana a su punto de partida, pero sigue sin ser la deseada. Tras la actualización del simulador, se realizaron las mismas pruebas, y el resultado obtenido se muestra en las siguientes figuras.



Figura 57. Posición del Robobo antes de realizar el circuito en el entorno simulado tras la actualización del simulador para la superficie de madera. Fuente: Elaboración propia.

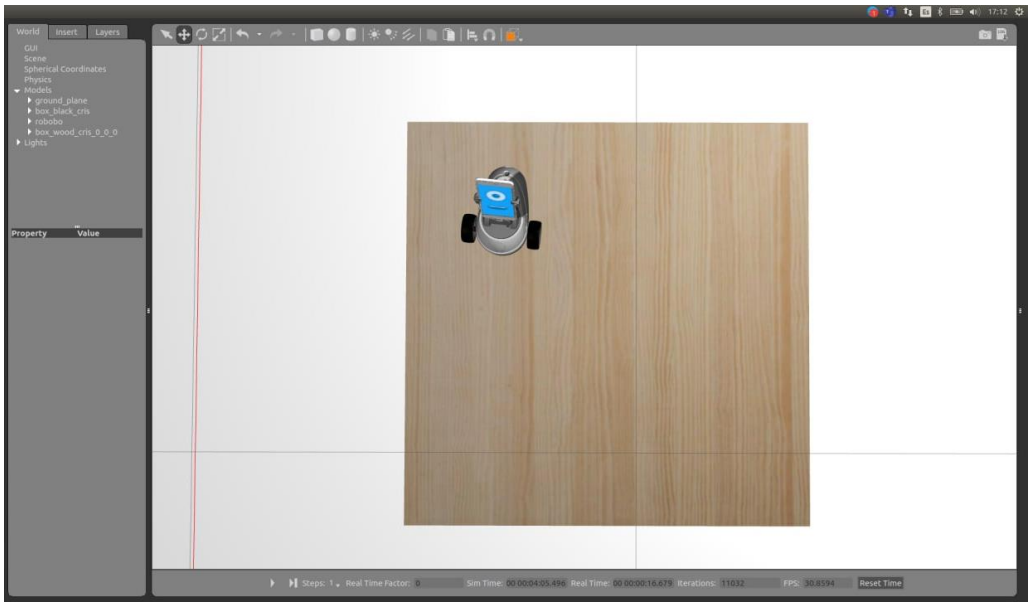


Figura 58. Posición del Robobo después de realizar el circuito en el entorno simulado tras la actualización del simulador para la superficie de madera. Fuente: Elaboración propia.

Como vemos en la Figura 57 y la Figura 58 el robot vuelve a su posición inicial, sin embargo, como el caso anterior, los parámetros de velocidad y tiempo han necesitado una modificación respecto a los reales: -20 para la rueda izquierda y 20 para la rueda derecha con un coeficiente de rozamiento de la superficie de 0,4 y un tiempo para la realización del giro de 3,5 segundos.

A continuación, se han realizado los mismos pasos para el estudio de la superficie de la ciudad en el Mundo 2. En este caso, la superficie con la que se va a trabajar es un vinilo, por

ello se ha buscado el coeficiente de fricción del vinilo, y se han modificado los parámetros de rozamiento en el simulador con los pasos indicados en la Figura 53. En la Figura 59 y Figura 60 se muestra la posición del Robobo previa a la realización del circuito y su posición tras finalizar este.

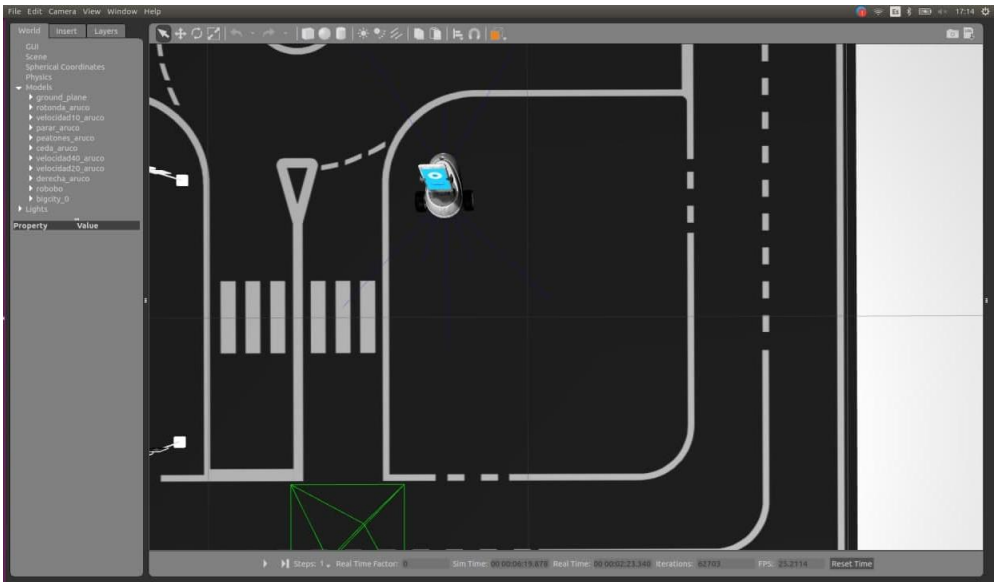


Figura 59. Posición del Robobo antes de realizar el circuito en entorno simulado (Mundo 2). Fuente: Elaboración propia.

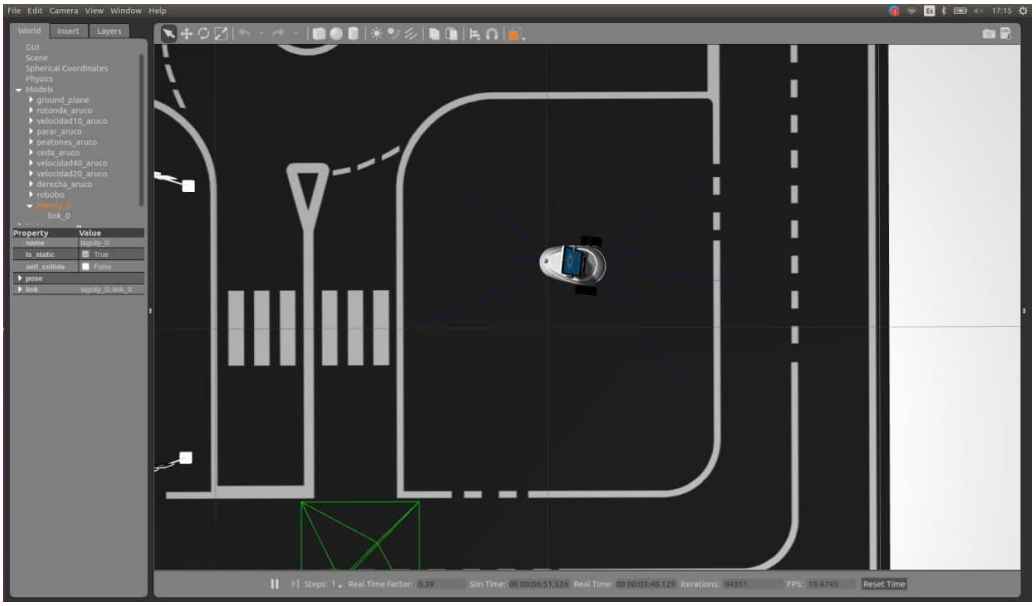


Figura 60. Posición del Robobo tras realizar el circuito en entorno simulado (Mundo 2). Fuente: Elaboración propia.

Como se puede comprobar en las anteriores figuras, el Robobo tampoco ha regresado a su posición inicial, de modo que ha sido necesario realizar el mismo estudio que en las superficies anteriores, para comprobar que tiempo necesita en cada giro para terminar el circuito en la posición en la que lo comenzó. Los tiempos con los que se han realizado las pruebas han sido los mismos que en las pruebas anteriores, 5, 7.5, 10, 12.5 y 15, pero

tampoco se ha llegado a un resultado concluyente. Por ello se ha realizado una última prueba con un tiempo de medio minuto para que el Robobo realice cada giro.

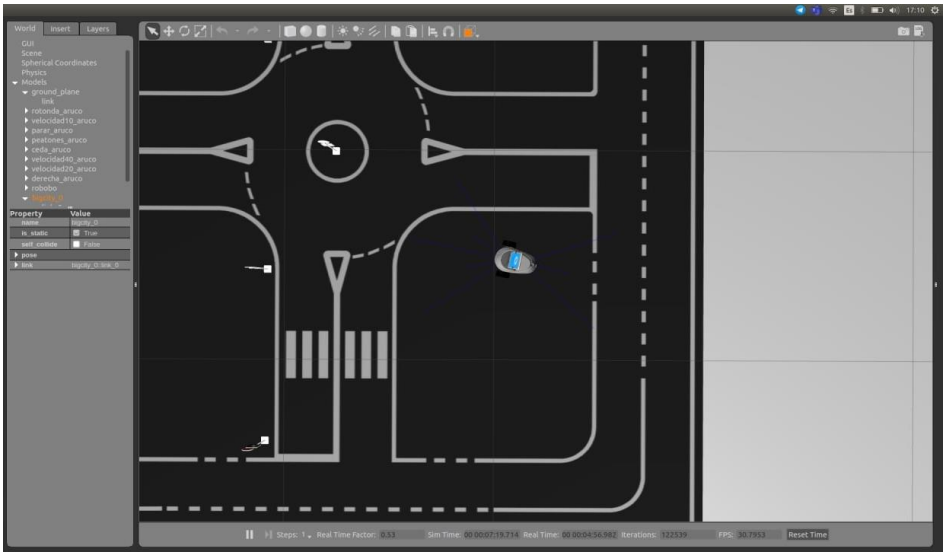


Figura 61. Posición del Robobo tras modificar el tiempo en el que debe realizar cada giro y tras terminar el circuito en entorno simulado. Fuente: Elaboración propia.

Observamos pues que la posición del robot con un tiempo para que realice el giro de 3,8 segundos obtenida en la Figura 60 no dista mucho de la obtenida en la Figura 61 tras haber modificado el tiempo a 30 segundos, aunque se acerca más a su posición al inicio de la trayectoria.

Como en los casos anteriores después de actualizar el Gazebo, la posición final respecto a la inicial es prácticamente la misma, pero los parámetros del movimiento han sido modificados para conseguir estos resultados. En la Figura 62 y Figura 63 se muestran estos resultados y los valores de los parámetros han sido: -20 para la rueda izquierda y 20 para la rueda derecha con un coeficiente de rozamiento de la superficie de 0,09 y un tiempo para la realización del giro de 2,7 segundos.

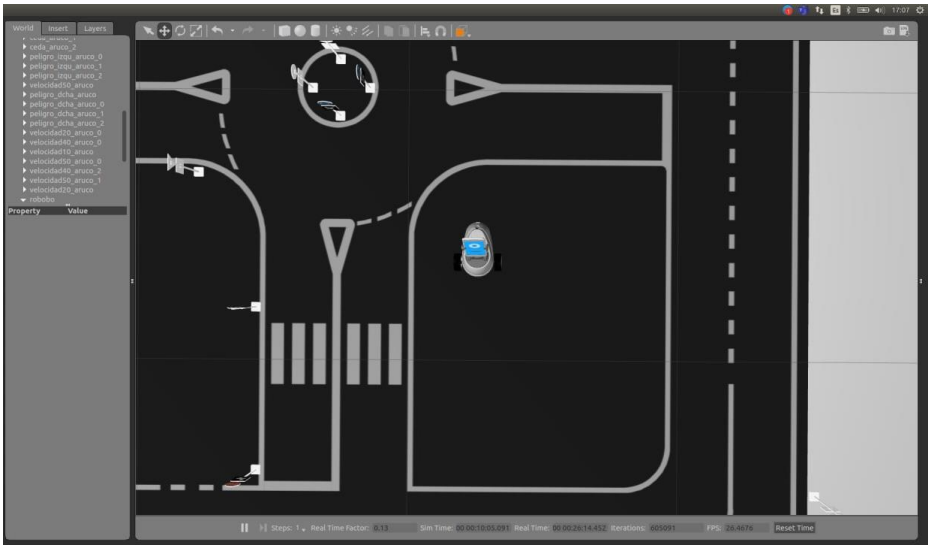


Figura 62. Posición del Robobo antes de realizar el circuito en el entorno simulado tras la actualización del simulador para la superficie de vinilo. Fuente: Elaboración propia.

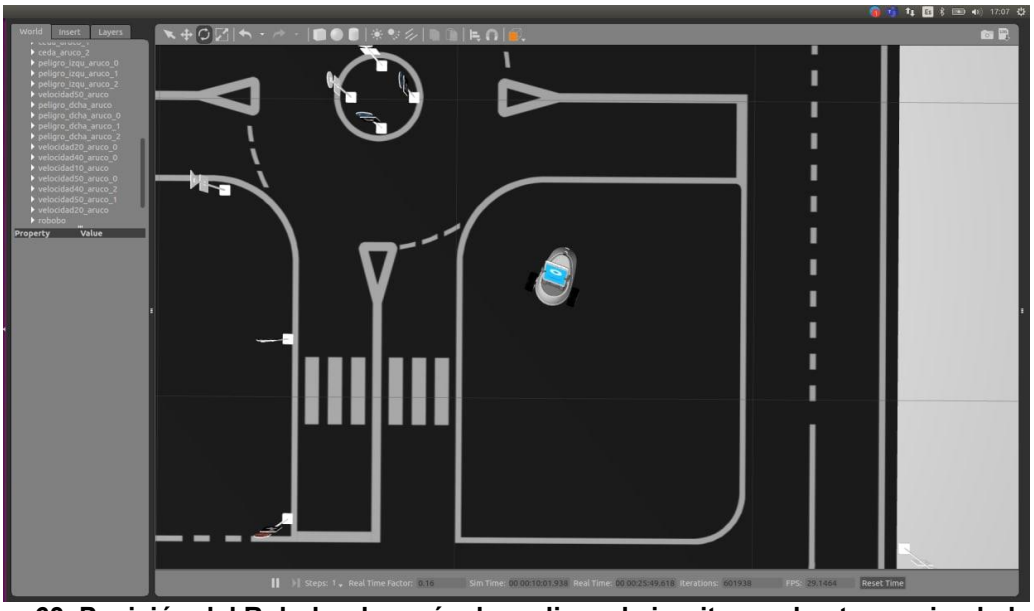


Figura 63. Posición del Robobo después de realizar el circuito en el entorno simulado tras la actualización del simulador para la superficie de vinilo. Fuente: Elaboración propia.

❖ Comparación y conclusiones

Comparando los movimientos realizados en ambos escenarios, se puede determinar que la posición final tras realizar los giros asignados es prácticamente la misma que la inicial en ambos entornos. En las siguientes tablas se muestra una comparación de los valores de los parámetros implicados en este movimiento para las diferentes superficies tanto en el entorno real como en el simulado.

Tabla 13. Comparación entre los parámetros de ambos entornos para el mismo movimiento para la superficie de madera. Fuente: Elaboración propia.

Parámetros	Entorno real	Entorno simulado
Velocidad de ambas ruedas en el movimiento recto	10	20
Velocidad de la rueda izquierda en el giro	0	-20
Velocidad de la rueda derecha en el giro	10	20
Tiempo necesario para realizar cada giro	3,8	3,5
Coeficiente de rozamiento	0,4	0,4

Tabla 14. Comparación entre los parámetros de ambos entornos para el mismo movimiento para la superficie de vinilo. Fuente: Elaboración propia.

Parámetros	Entorno real	Entorno simulado
Velocidad de ambas ruedas en el movimiento recto	10	20
Velocidad de la rueda izquierda en el giro	0	-20
Velocidad de la rueda derecha en el giro	10	20
Tiempo necesario para realizar cada giro	3,8	2,7
Coeficiente de rozamiento	0,09	0,09

Podemos observar que tanto en la Tabla 13 como en la Tabla 14, la velocidad durante la trayectoria recta es el doble en el entorno simulado que en el real, y la diferencia entre la velocidad en la rueda izquierda y la derecha el cuádruple. Además, podemos observar que el tiempo para realizar el giro en el simulador para la superficie de madera es xx y para la de vinilo es 2,7. Por estas razones, vemos que existe diferencia entre ambos entornos para realizar la trayectoria, aunque el resultado final sea el mismo.

Además, también podemos observar que el coeficiente de rozamiento influye mucho más en el entorno simulado que en real, pues el valor de los parámetros es el mismo en las tres superficies en las que se han realizado las pruebas para el entorno real, pero el tiempo requerido para el giro en las tres superficies es distinto en el entorno simulado.

6.4.2 Cámara

Para caracterizar el sensor de la cámara se han realizado las mismas pruebas que en el apartado de pruebas básicas, pero en este caso se va a estudiar cómo afecta el entorno y cómo influye la luz tanto en su detección cuando el robot permanece estático, como en su detección cuando el robot permanece en movimiento. Como se ha comentado anteriormente, el método de detección de ArUcos del entorno simulado devuelve un área. Por ello se ha creado un programa para que el detector de ArUcos en el entorno real también devuelva el área.

❖ Entorno real

La realización de estas pruebas ha comenzado con el estudio del robot cuando este permanece estático y se ha ido colocando el marcador ArUco a las distancias consideradas, variando para cada medida los niveles de luz. Para medir el nivel de luz en la escena lo ideal sería contar con un luxómetro profesional. Al no disponer de uno se ha optado por utilizar una aplicación móvil que usa el sensor de luz del smartphone. Esta aplicación requiere ser calibrada previamente. El sensor de luz se encuentra a 1,80 m del foco principal y a esa distancia se obtuvieron los siguientes valores de luminosidad en el ambiente: 390 lux y 41 lux. En la Tabla 15 se muestra el valor del área obtenida para cada valor de luminosidad.

Tabla 15. Área que devuelve el ArUco en las pruebas en entorno real con 390 lux y 41 lux. Fuente: Elaboración propia.

Distancia del ArUco (cm)	Área que devuelve (px2) con 390 lux	Área que devuelve (px2) con 41 lux
30 cm	10181,20	10240,4305
40 cm	5873,35	5717,209
60 cm	2324,70	2294,69
80 cm	1211,48	1233,92
100 cm	766,03	752,83
120 cm	497,96	501,17

Es posible obtener una relación entre el área calculada por el detector de ArUcos y la distancia a la que se encuentra este. A continuación, en la Figura 64 y la Figura 65 se pueden observar los posibles polinomios de interpolación para la luminosidad de 390 lux y 41 lux respectivamente. Para ambas la tendencia que mejor se ajusta a nuestros puntos es la potencial para una función polinómica que se muestra a continuación, pues en los dos casos el coeficiente de determinación es del 0,9994 en el caso de la primera y del 0,9998 en el caso de la segunda.

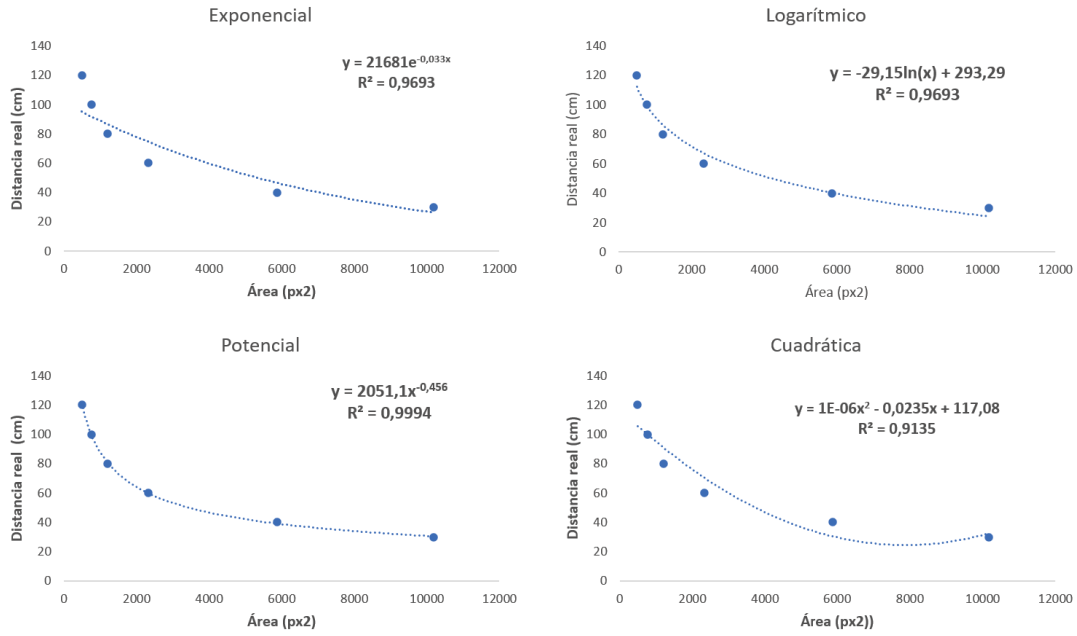


Figura 64. Polinomios de interpolación con tendencia exponencial, logarítmico, potencial y cuadrática para las pruebas realizadas en el entorno real a 390 lux. Fuente: Elaboración propia.

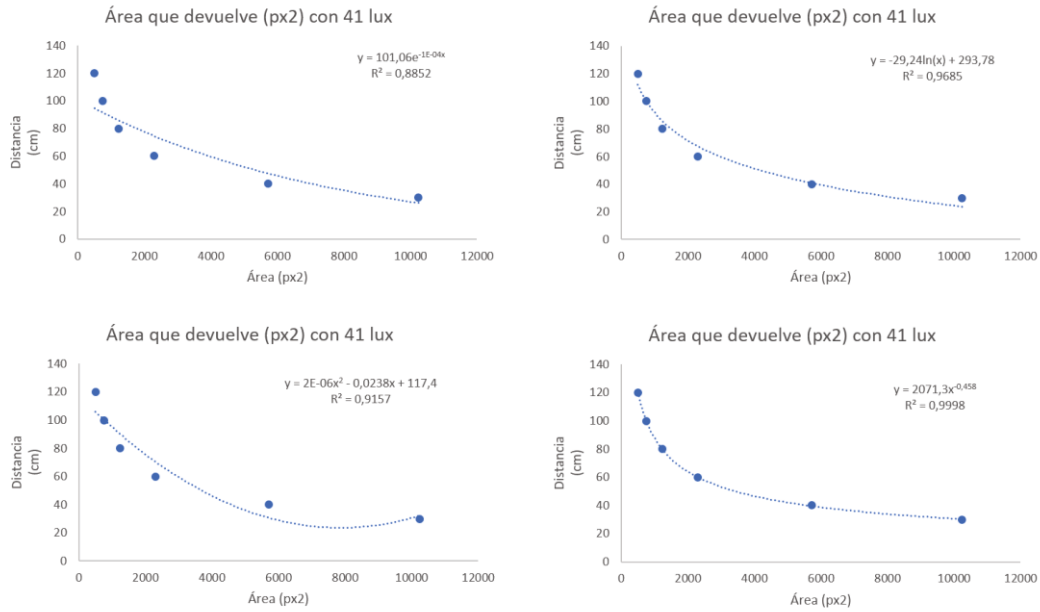


Figura 65. Polinomios de interpolación con tendencia exponencial, logarítmico, potencial y cuadrática para las pruebas realizadas en el entorno real a 41 lux. Fuente: Elaboración propia.

También se muestran los polinomios obtenidos para ambas luminosidades. La Ecuación (3) corresponde a las medidas tomadas a 390 lux y la Ecuación (4) a las tomadas a 41 lux.

$$y = 2051{,}1\ x^{(-0456)} \tag{3}$$

$$y = 2071{,}3\ x^{(-0458)} \tag{4}$$

A continuación, se muestra una tabla con las distancias calculadas con los distintos polinomios de interpolación. Como podemos observar, las distancias obtenidas a 41 lux se acercan más a la realidad que las obtenidas con 390 lux. Esto quiere decir, que el Robobo lee mejor los marcadores ArUco cuando menos luminosidad hay en el ambiente.

Tabla 16. Distancia obtenida después de la interpolación en las pruebas en entorno real para 390 lux y 41 lux. Fuente: Elaboración propia.

Distancia del ArUco (cm)	Distancia obtenida a 390 lux (cm)	Distancia obtenida a 41 lux (cm)
30 cm	30,509178	30,1659678
40 cm	39,208017	39,396052
60 cm	59,830622	59,8454866
80 cm	80,536847	79,5121611
100 cm	99,259198	99,7047427
120 cm	120,79979	120,129549

A continuación, se han realizado las pruebas para evaluar el tiempo de respuesta del robot con diferente iluminación una vez leído el ArUco para que este se detenga. Para la realización de estas pruebas se ha tomado como referencia el esquema de la Figura 38, dónde el Robobo debe realizar un circuito con distintas velocidades y detenerse ante el STOP. Los datos obtenidos se muestran en la Tabla 17 .

Tabla 17. Distancia a la que se detiene el Robobo respecto a la coordenada 0 al leer la señal de STOP para 390, 310 y 55 lux. Fuente: Elaboración propia.

Velocidades	Distancia respecto a la coordenada 0 para 390 lux (cm)	Distancia respecto a la coordenada 0 para 310 lux (cm)	Distancia respecto a la coordenada 0 para 55 lux (cm)
5	13,5	16	17
	14	15,5	26
	12	17,5	24
	14,5	16	20
10	13	15,5	21,5
	14	17	10,5
	11,5	16	19,5
	11,5	16,5	14
15	11	13,5	11,5
	13	12,5	11
	9,5	15	9
	11,5	14,5	10,5
20	8,5	9,5	9
	6	9	8,5
	10	9,5	9,5
	7,5	11,5	9
25	4,5	7	7,5
	6	8,5	7
	4,5	7	8
	5,5	5,5	7
30	2	5,5	9
	3,5	4	2,5
	2,5	5	6
	4	5,5	5

A continuación, se ha realizado una media de las medidas obtenidas para cada velocidad y se han graficado para poder ser comparadas con mayor claridad. Esto se muestra en la Figura 66. En ella podemos observar que, para velocidades muy pequeñas, la media de las respuestas a las diferentes iluminaciones es más grande que cuando son velocidades muy

elevadas, pues independientemente de la luz que haya en el ambiente, el tiempo de respuesta es menor debido a la inercia. Esto quiere decir que, a mayor velocidad, más tarda en detenerse el Robobo una vez detectado el ArUco. Además, se puede observar que, para 55 lux, el robot se detiene antes, pues la distancia que hay entre este y las coordenadas cero es mayor. De esta manera llegamos a las mismas conclusiones que en las pruebas con el robot estático, el Robobo detecta mejor los ArUcos cuando menos luz hay en el ambiente.

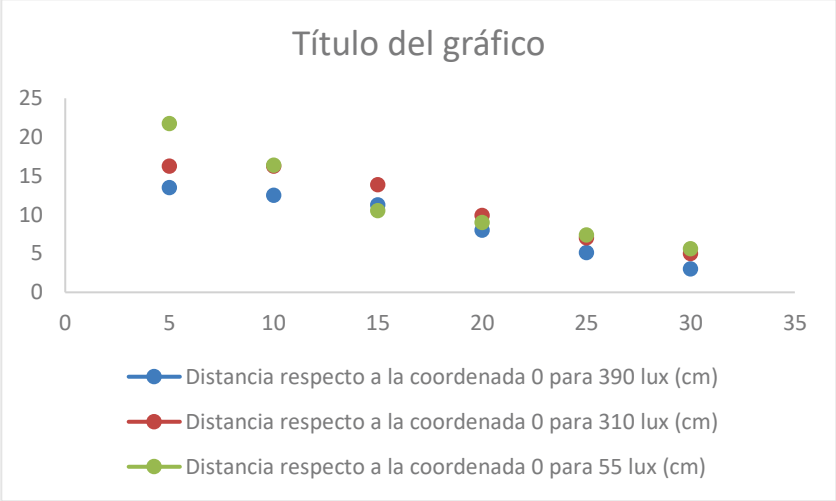


Figura 66. Comparación de los valores obtenidos para diferentes lux. Fuente: Elaboración propia.

❖ Entorno simulado

Por su parte al tratar de recrear las mismas condiciones de la prueba en el entorno simulado de Gazebo se ha encontrado la imposibilidad de cuantificar la cantidad de luz presente en el ambiente. El simulador permite generar focos o puntos de luz, sin embargo, no permite medir la luminosidad que generan. Existe la posibilidad de cuantificar esta información, sin embargo, ante la necesidad de que un mundo ya creado no sea modificado por parte del usuario se ha optado por la no realización de esta prueba ya que sería imposible realizar a posteriori la comparación entre ambos entornos.

Por otro lado, como no se ha podido determinar el nivel de luz para la realización de las pruebas en el Gazebo, se ha optado por realizar la prueba del Robobo en movimiento con el robot colocado a distintas distancias. El esquema seguido para su realización ha sido el seguido en la Figura 38 y en la Figura 67 se muestra una imagen de la elaboración de la prueba en la que el dispositivo se encuentra a una distancia de 40 cm de los marcadores ArUco y otra a una distancia de 80cm. En la Tabla 18 se muestran los valores obtenidos.

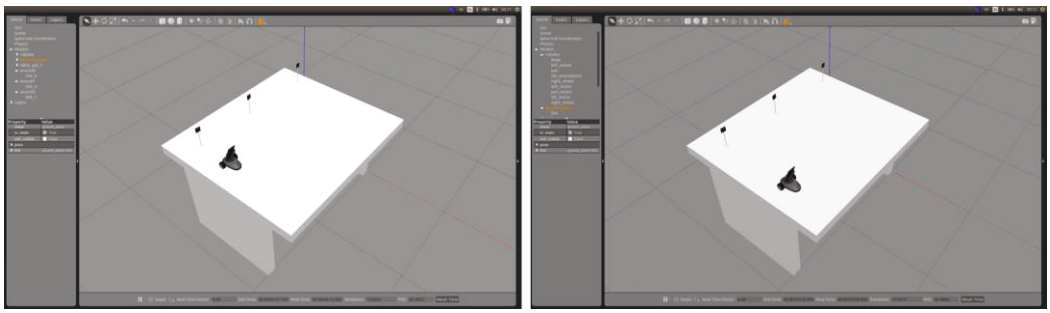


Figura 67. Simulación de la prueba de detección de ArUcos a distintas velocidades y distintas distancias en el simulador. Fuente: Elaboración propia.

Tabla 18. Comparativa de los valores obtenidos a una distancia de 40 cm y otra de 80 cm en el simulador. Fuente: Elaboración propia.

Velocidades	Distancia respecto a la coordenada 0 a 40 cm (cm)	Distancia respecto a la coordenada 0 a 80 cm (cm)
5	30,7088	47,1142
	32,1784	55,5680
	32,9454	43,8224
	34,1634	40,3121
10	30,1362	41,0905
	30,7785	38,1769
	32,1286	35,6718
	30,5479	9,9499
15	31,0144	38,1372
	30,4279	33,6172
	30,1254	32,1418
	29,0449	41,0133
20	29,8443	31,4218
	30,3589	13,3555
	30,5966	20,7827
	29,9141	12,9062
25	29,7202	19,9225
	28,4072	32,0052
	28,8587	9,2739
	30,3384	9,0312
30	30,7146	9,1854
	31,2127	9,7925
	31,1823	9,3107
	29,5276	6,7890

Para poder comparar mejor ambos resultados se ha optado por graficarlos como se muestra en la Figura 68. Para una distancia de 40 cm, el Robobo detecta el marcador ArUco con unos centímetros de distancia de diferencia para distintas velocidades, mientras que, a una distancia de 80 cm , hay una diferencia mucho mayor entre la distancia a la que se para el robot con una velocidad de 5 y una velocidad de 30. Cuanto más lento va el Robobo, antes detecta la señal ArUco. Además, su espectro visual es mucho más amplio, por ello detecta el marcador mucho antes que si se encuentra a una distancia menor.

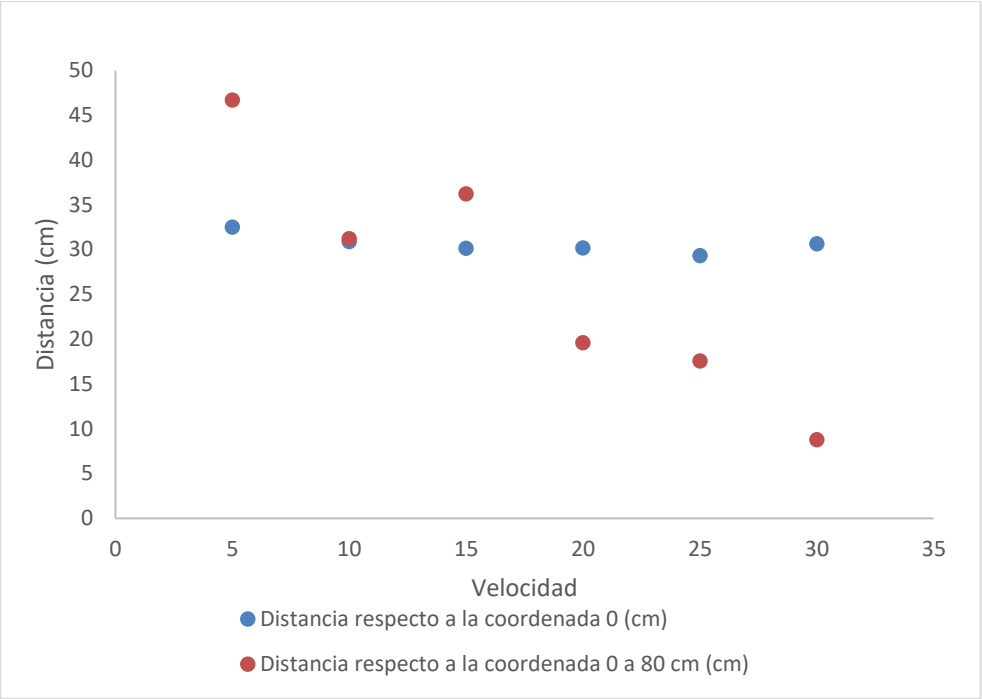


Figura 68. Representación de la velocidad frente al espacio respecto a la coordenada cero en las pruebas obtenidas con el simulador. Fuente: Elaboración propia.

❖ Comparación y conclusiones

Como se ha observado en estas pruebas, cuando hay menos luminosidad el Robobo detecta mejor los marcadores ArUco, aunque no los detecta con una oscuridad total. Para futuros trabajos sería interesante poder comparar estos resultados con un entorno simulado de manera que este, como se ha comentado anteriormente, tuviese la posibilidad de modificar los parámetros de luz y poder medir la luminosidad en cada momento.

Además, también es importante que a la hora de montar un escenario real en el que se van a realizar unas pruebas, se eviten zonas de sombras marcadas que dificulten la lectura del ArUco. Para ello, en interiores es aconsejable disponer de suficientes fuentes de iluminación, preferentemente difusa, que consigan una iluminación razonablemente homogénea sobre el escenario, de modo que se facilite el funcionamiento de los detectores basados en visión artificial.

6.4.3 Infrarrojos

Las pruebas realizadas para validar los infrarrojos son las mismas que se han explicado en el apartado anterior, pero en este caso, se han variado los diferentes fondos con el objetivo de comparar los valores que devuelve el sensor de infrarrojos, en función del color o la textura de la superficie. Las pruebas han sido realizadas en ambos entornos y a su vez también han sido divididas en pruebas estáticas y pruebas dinámicas.

En ambos entornos se han estudiado tres superficies distintas: una superficie blanca, una superficie de madera y una superficie negra.

❖ Entorno real

En estas pruebas los casos de estudio han sido la superficie de madera y la superficie negra (Figura 69.), ya que la superficie blanca ha sido evaluada en el apartado de pruebas básicas, pero sus resultados se compararán con los obtenidos en este punto. El procedimiento ha sido el mismo que con la superficie blanca. Se ha situado al robot a distintas distancias y se han anotado los valores obtenidos por el sensor del Robobo.

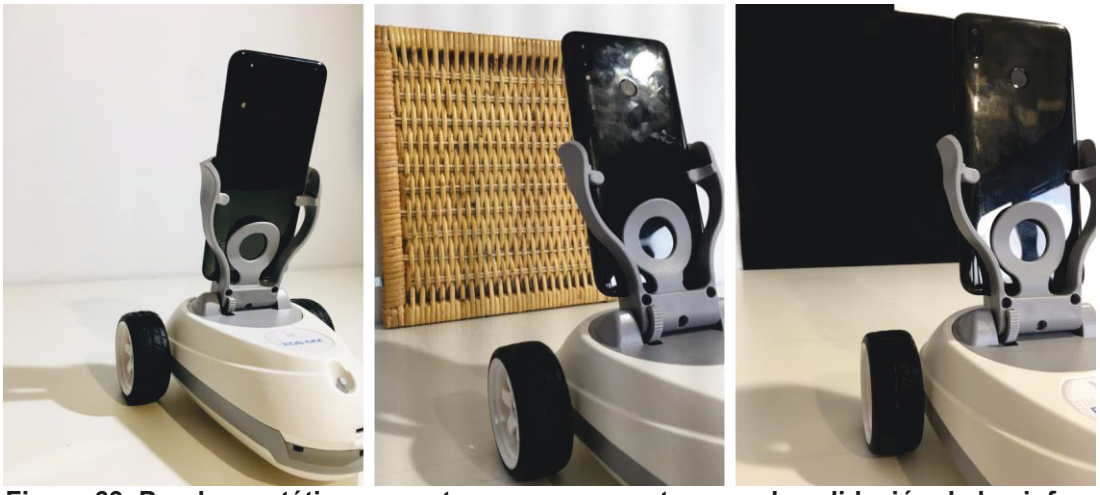


Figura 69. Pruebas estáticas en entornos no concretos para la validación de los infrarrojos en el entorno real. Fuente: Elaboración propia.

En la Tabla 19 y se pueden observar los valores obtenidos en la superficie de madera y la superficie negra y comparados con la superficie blanca. En la superficie de madera podemos observar que como en la superficie blanca, cuanto más lejos se encuentra el robot, más pequeño es el valor que devuelve el sensor.

Tabla 19. Valores del sensor de infrarrojos obtenidos en pruebas estáticas para distintas superficies en un entorno real. Fuente: Elaboración propia.

Distancia del robot (cm)	Valores del sensor de infrarrojos sobre la superficie blanca	Valores del sensor de infrarrojos sobre la superficie de madera	Valores del sensor de infrarrojos sobre la superficie negra
11 cm	149	112	14
14 cm	70	58	12
19 cm	44	37	11
24 cm	32	27	11
34 cm	21	19	10
44 cm	17	15	10
54 cm	14	13	10
69 cm	8	6	7

En cuanto a la superficie negra observamos que, para diferentes distancias, el valor es exactamente el mismo, debido a que la superficie negra absorbe prácticamente toda la radiación de la luz. Es por esta razón por la que se ha decidido que la superficie negra sea uno de los casos de estudio. Tal vez en estas pruebas no sea algo relevante, pero su análisis en las pruebas dinámicas sí que puede ser interesante.

Por lo tanto, comparando todas las superficies podemos observar que la superficie blanca es para la que el sensor devuelve mayores valores, mientras que para la negra es para la que los valores son más pequeños. En las pruebas dinámicas estos datos tendrán mayor relevancia.

Con estos datos es posible sacar una relación entre la distancia a la que se encuentra el robot de la superficie y el valor de infrarrojos que arroja para cada superficie, y comparar las funciones de interpolación que se obtienen para cada una de ellas. En la Figura 70 se muestran las tendencias seleccionadas en función del coeficiente de determinación que mejor se ajusta para cada polinomio de interpolación.

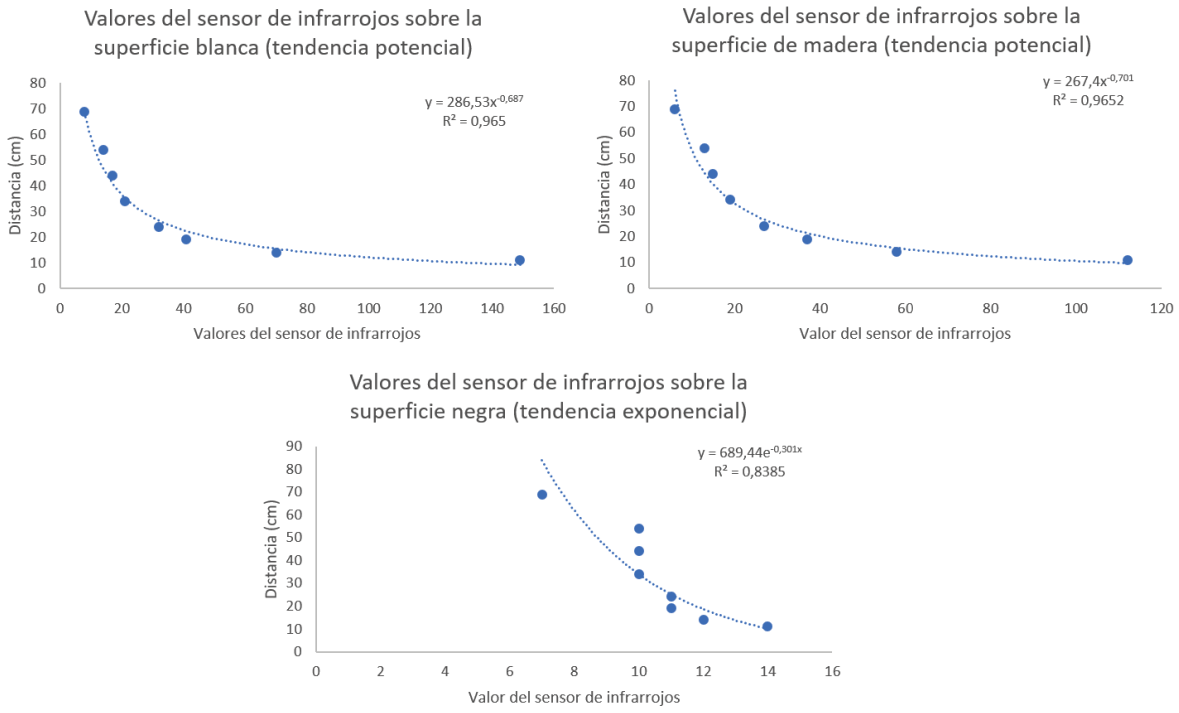


Figura 70. Polinomios de interpolación obtenidas para cada superficie para las pruebas realizadas en el entorno real. Fuente: Elaboración propia.

En la Figura 70, podemos ver que la superficie blanca sigue una tendencia potencial, al igual que la superficie de madera. Sin embargo, la superficie negra no presenta ninguna tendencia, aunque en la figura se ha representado una tendencia exponencial por ser la que mayor coeficiente de determinación presenta. Tiene lógica que la superficie blanca y la superficie de madera sigan ambas la misma tendencia pues la diferencia de los valores que devuelve el sensor de infrarrojo entre ellas es muy pequeña. La superficie negra como podemos ver no sigue ninguna tendencia clara en función de la distancia porque como comentamos anteriormente esta absorbe prácticamente toda la radiación reflectante. Por ello no existe ninguna relación entre la distancia real y los valores obtenidos.

A continuación, también se muestran las funciones de interpolación obtenidas para la superficie blanca y la de madera. La Ecuación (5) corresponde a la superficie blanca y la Ecuación (6) a la de madera.

$$y = 286{,}53\, x^{(-0{,}687)} \tag{5}$$

$$y = 267{,}4\, x^{(-0{,}701)} \tag{6}$$

Tras la realización de las pruebas estáticas, se han abordado las pruebas dinámicas. Estas pruebas también son las mismas realizadas en el apartado de pruebas básicas con una superficie blanca. El robot debe estar en movimiento hasta que el sensor detecte un objeto que devuelva un valor inferior a 30, entonces debe de detenerse.

En la Tabla 20 se vuelven a mostrar los valores obtenidos para la superficie blanca para poder compararlos con el resto.

Tabla 20. Valores del sensor de infrarrojos obtenidos en pruebas dinámicas para una superficie blanca. Fuente: Elaboración propia.

Valores del sensor de infrarrojos	Distancia del robot (cm)
32	15 cm
34	17 cm
35	19 cm
34	15 cm
33	17 cm

En la Tabla 21, se muestran los resultados para la superficie de madera. Podemos observar que los valores que devuelve el sensor son un poco inferiores a los que devolvía la superficie blanca para las mismas distancias del robot. Esto tiene sentido, pues ya se ha comprobado en las pruebas estáticas, que la superficie blanca arroja valores superiores que la de madera.

Tabla 21. Valores del sensor de infrarrojos obtenidos en pruebas dinámicas para una superficie de madera. Fuente: Elaboración propia.

Valores del sensor de infrarrojos	Distancia del robot (cm)
31	13 cm
33	15 cm
36	13 cm
33	15 cm
32	18 cm

En cuanto a la superficie negra, en la Tabla 22 podemos ver que los valores que devuelven los infrarrojos se corresponden con una distancia real de 1 o 2 cm. Como ya se ha explicado

anteriormente, esto es debido a que la superficie negra absorbe casi toda la radiación de la luz, y sumado a la inercia del robot, hace que este se detenga a tan solo un par de centímetros de la pared.

Tabla 22. Valores del sensor de infrarrojos obtenidos en pruebas dinámicas para una superficie negra. Fuente: Elaboración propia.

Valores del sensor de infrarrojos	Distancia del robot (cm)
35	2 cm
45	1 cm
36	2 cm
47	1 cm
31	2 cm

De esta forma, podemos demostrar que, si el objeto que tiene delante el Robobo es de color blanco, este se detendrá antes que si es de otro color debido a que las superficies blancas reflejan más luz que otras. Por el contrario, si el objeto es de color negro el Robobo se detendrá a escasos centímetros del mismo y es importante tener esto en cuenta para futuras pruebas.

❖ Entorno simulado

Para realizar las pruebas en el entorno simulado se ha utilizado el Mundo 1. En él se ha dispuesto unas cajas a modo de superficie de distintos colores (blanca, negra y marrón) en las coordenadas (0,0,0,941833) (Figura 71). Una vez colocada la caja, el Robobo ha sido situado a distintas distancias respecto a la caja para poder tomar las medidas que se muestran en la Tabla 23 .

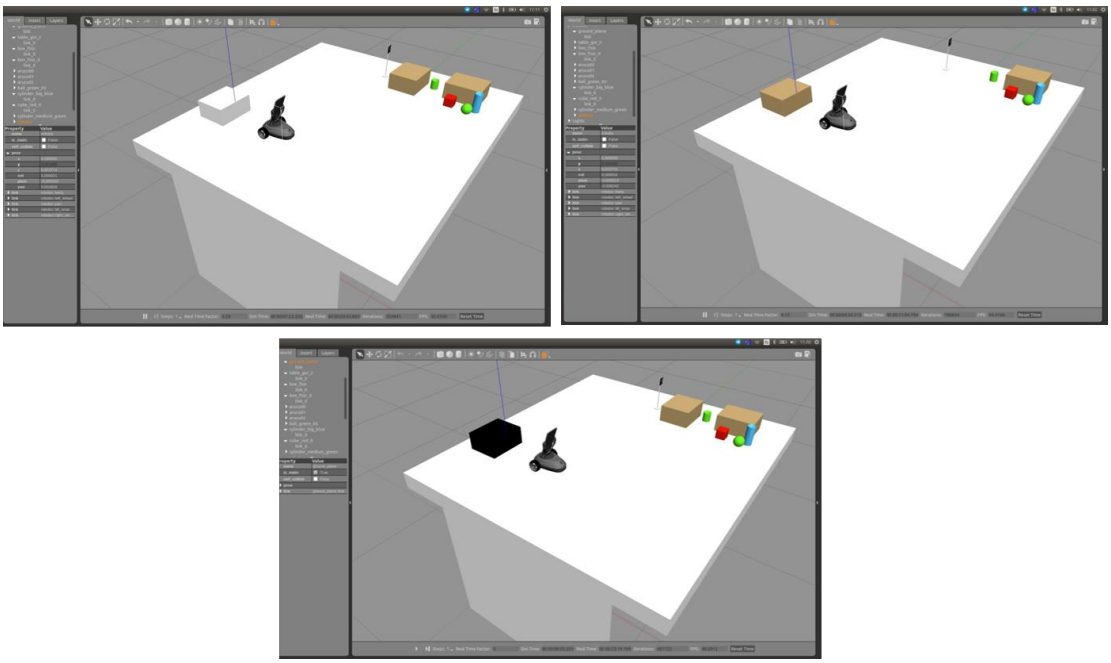


Figura 71. Obtención de datos con el sensor de infrarrojos de Gazebo en el entorno simulado. Fuente: Elaboración propia.

Tabla 23. Valores del sensor de infrarrojos obtenidos en pruebas estáticas para distintas superficies en un entorno simulado. Fuente: Elaboración propia.

Distancia del robot (cm)	Valores del sensor de infrarrojos sobre la superficie blanca	Valores del sensor de infrarrojos sobre la superficie de madera	Valores del sensor de infrarrojos sobre la superficie negra
11 cm	0	0	0
14 cm	89,466827	89,504560	89,462646
19 cm	47,29581	47,295312	47,297211
24 cm	29,599653	29,609365	29,595001
34 cm	17,300091	17,299957	17,298412
44 cm	13,627504	13,628924	13,640050
54 cm	13,634351	13,637605	13,639219
69 cm	13,635187	13,632968	13,639831

Si comparamos los datos de las tres columnas de la tabla anterior, podemos observar que para una distancia menor a 13 cm ninguna de las tres es capaz de detectar un objeto. Entre 14 cm y 44 cm los valores son muy similares para las tres superficies, variando apenas unas décimas o centésimas. Por último, todas las superficies a una distancia mayor de 44 cm dejar de modificar los valores.

Además de comparar los diferentes valores del sensor de infrarrojos para una misma distancia, también es necesario comparar como varían estos datos cuando el robot se encuentra en movimiento. Para la realización de estas pruebas, se han usado las mismas cajas que las usadas para las pruebas en estático. La prueba es la siguiente: el robot debe permanecer en movimiento hasta que encuentre un objeto para el que el sensor devuelva un valor mayor a 30. Esta prueba se ha repetido cinco veces para cada caja de color. En la Tabla 24, Tabla 25 y Tabla 26 se muestran los valores obtenidos para cada caja.

Tabla 24. Valores del sensor de infrarrojos obtenidos para la caja blanca en el simulador. Fuente: Elaboración propia.

Valores del sensor de infrarrojos	Distancia del robot (cm)
30,01775	29,749 cm
30,01793	29,812 cm
30,04976	29,641 cm
30,05144	29,778 cm
30,02647	29,795 cm

Tabla 25. Valores del sensor de infrarrojos obtenidos para la caja marrón en el simulador. Fuente: Elaboración propia.

Valores del sensor de infrarrojos	Distancia del robot (cm)
30,00026	29,837 cm
30,0390	29,811 cm
30,0097	29,827 cm
30,0132	29,824 cm
30,0294	29,819 cm

Tabla 26. Valores del sensor de infrarrojos obtenidos para la caja negra en el simulador. Fuente: Elaboración propia.

Valores del sensor de infrarrojos	Distancia del robot (cm)
30,02903	29,512 cm
30,05559	24,521 cm
30,05502	29,805 cm
30,09595	29,781 cm
30,15725	29,751 cm

Como se puede observar en las tres tablas anteriores, los valores obtenidos son similares. No existe ninguna diferencia apreciable entre el color de la caja que detecta, algo que ya se había demostrado con las pruebas estáticas.

❖ Comparación y conclusiones

A la hora de comparar los datos obtenidos en ambos entornos, podemos comprobar que en el entorno simulado no se aprecian modificaciones entre ninguna de las tres superficies, a diferencia del entorno real, que entre la superficie de color negro y las otras dos daban datos muy dispares.

Con estos resultados se puede determinar que el sensor de infrarrojos del entorno simulado no funciona igual que el del entorno real ya que a distancias inferiores a 14 cm el sensor no devuelve ningún valor, y para distintas distancias superiores a 44 cm arroja el mismo resultado. Además de que los valores son prácticamente los mismos en las tres superficies, cosa que como queda demostrado con las pruebas realizadas en el entorno real, no debería ser así ya que la superficie negra absorbe casi toda la radiación de la luz.

6.5 Pruebas en escenarios específicos

Una vez validados y caracterizados algunos de los sensores del Robobo en diferentes ambientes y con la influencia de distintos factores externos, el objetivo de esta nueva fase es realizar una prueba en la que se pueda comparar cómo es la respuesta de estos cuando actúan todos en común sobre un escenario de aplicación realista, y en la que se implementen los resultados obtenidos hasta el momento.

Anteriormente se hablaba de que el sensor de la cámara del robot real poseía diferentes funciones, una de ellas es la detección de líneas y el control de carriles, por lo que se ha querido comprobar cómo funciona esta en un entorno urbano en conjunto con el resto de sensorización del robot. Para ello, se ha utilizado una maqueta de una ciudad en el entorno real, comentada al principio del capítulo, y una maqueta de una ciudad en el entorno simulado al que hemos denominado Mundo 2.

En la Figura 72 se puede observar un esquema de la ciudad. Como se muestra, la ciudad cuenta con dos carriles para circular en los dos sentidos, cuatro cuadrados que corresponden a cuatro edificios, una rotonda y dos pasos de cebra. El objetivo de esta prueba es que el Robobo realice el circuito marcado en color rojo de forma autónoma, en el que debe realizar un tramo recto con una posterior curva y adentrarse en la ciudad atravesándola con su respectiva rotonda hasta detenerse en un STOP.

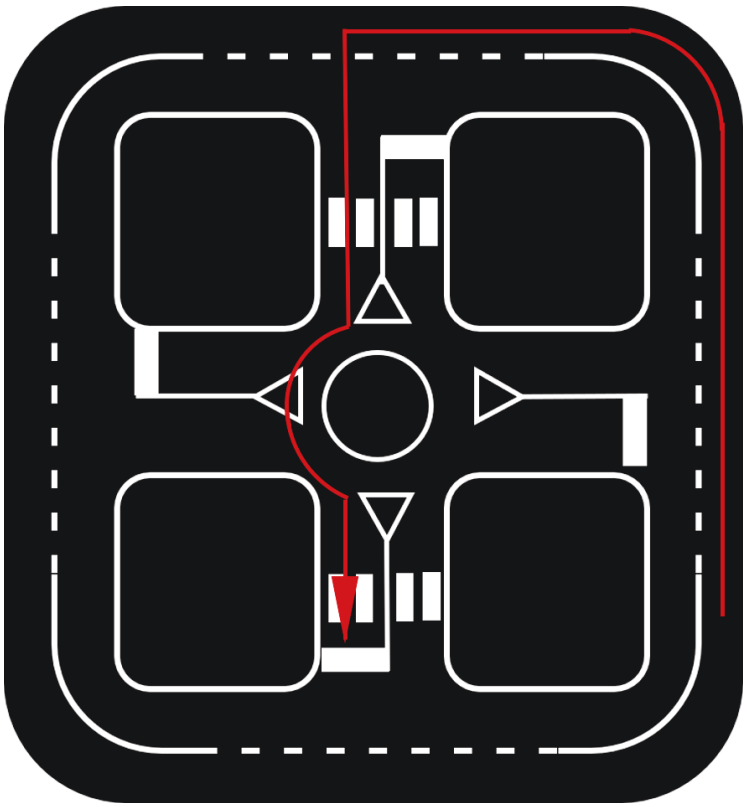


Figura 72. Esquema del circuito que debe realizar el Robobo en la ciudad. Fuente: Elaboración propia.

Hay que tener en cuenta que en el robot real se usa un método de detección de carriles escrito en java para ser ejecutado desde el smartphone, pero este método no ha sido adaptado para ser ejecutado en ROS en el simulador. Por ello, para la realización de este trabajo se ha utilizado un detector de líneas escrito en Python del Grupo Integrado de Ingeniería de la Universidade da Coruña y se ha rehecho el control de movimiento usando un control proporcional para que sea lo más parecido posible al que se usa en el robot real que es un control PID.

Además, se ha podido comprobar que el detector de carriles que ha sido adaptado y utilizado en las pruebas realizadas en el simulador no detecta de forma adecuada las líneas curvas en este escenario, por ello las pruebas llevadas a cabo en el entorno real se han adaptado a esta condición colocando marcadores ArUco en su defecto. A continuación, se mostrarán y compararán los resultados obtenidos tanto en el entorno real como en el simulado.

❖ Entorno real

A continuación, se describirá el circuito que debe seguir el Robobo en el entorno real. En la Figura 73 se muestra una imagen del circuito entero en el que se desarrollará la siguiente prueba. El Robobo comienza el circuito desde la esquina inferior derecha de la ciudad por el carril exterior. El objetivo durante este tramo es que el robot realice un movimiento sin determinar el tiempo en el que este debe estar moviéndose y que mediante el detector de líneas y el control de carriles el robot se mantenga en su carril.

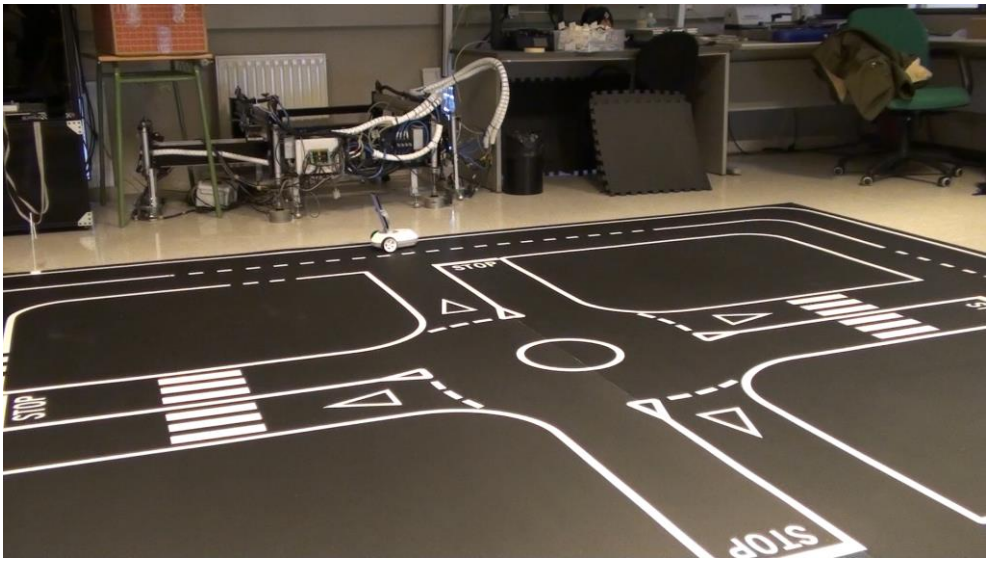


Figura 73. Inicio de la trayectoria del Robobo en la maqueta de la ciudad en el entorno real. Fuente: Elaboración propia.

Una vez llegado a la curva (Figura 74), debido al problema encontrado en la detección de carriles para los tramos curvos de este escenario, se ha optado por colocar una señal de giro con su correspondiente marcador ArUco para superar esta deficiencia, y que el robot pueda realizar sin demasiadas complicaciones un giro de 90º que le permita seguir la trayectoria. En fases anteriores del trabajo se había conseguido obtener y calibrar un cálculo de la distancia a la que se encontraba el marcador ArUco a partir del área en pixeles que calculaba el robot, así que este dato ha sido utilizado en este punto, de manera que el robot ahora es capaz de calcular a qué distancia debe realizar las ordenes encomendadas por la señal ArUco.

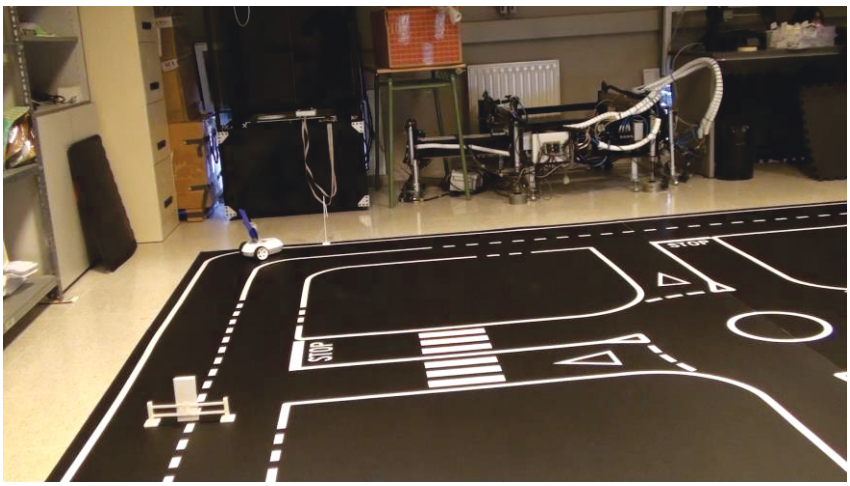


Figura 74. Circuito del Robobo en el entorno real en la maqueta de la ciudad. Fuente: Elaboración propia.

Una vez que el Robobo pasa la curva, este debe seguir recto y se encontrará con una señal de obras y calle cortada. Tras esto, el robot toma otro camino atravesando la ciudad para llegar hasta una señal de STOP (marcador ArUco) que le indicará que debe detenerse. Esto puede verse en la Figura 75.



Figura 75. Circuito que realiza el Robobo en el entorno real en la maqueta de la ciudad. Fuente: Elaboración propia.

El método de detección de carriles que se ejecuta de forma nativa en el smartphone del Robobo devuelve los coeficientes de un polinomio de segundo orden para cada uno de los carriles que detecta, el izquierdo y el derecho. Este método detecta líneas blancas y amarillas sobre fondo negro o viceversa. Para poder mantener al robot dentro de los carriles en todo momento, se ha utilizado un control de movimiento escrito en Python y que se ejecuta por encima del detector antes nombrando. Este utiliza un control PID (proporcional, integral, derivativo) logrando corregir la trayectoria del robot a cada intervalo de tiempo.

Este control utiliza los polinomios de segundo orden de ambos carriles para calcular el punto medio entre los dos a una cierta distancia *h* desde la cámara del Robobo. A continuación, se calcula el error como la diferencia entre este punto y el punto medio de la imagen, el cual es un valor fijo que depende del tamaño en pixeles de la cámara que use nuestro smartphone.

Como la cámara frontal se encuentra en el eje central del robot, el punto medio de la imagen indica siempre la dirección que el robot lleva en cada momento. Por lo tanto, el error antes comentado, muestra cómo es la desviación en ambas direcciones. Para corregir este error en la medida de lo posible se utiliza el control PID que trata de minimizarlo en cada iteración de tiempo.

Durante las pruebas realizadas para lograr que el Robobo completase el circuito establecido, se ha encontrado la existencia de factores que influyen en el resultado del circuito. Por un lado, tenemos el tema de la iluminación antes comentado. La localización en la que se encuentra la maqueta de la ciudad no cuenta con una buena iluminación, es decir una iluminación homogénea en todo el circuito. Además, la habitación no dispone de un medio para bloquear la luz externa por lo que se crean muchas sombras en el escenario de trabajo. Este hecho confunde al detector de carriles y por lo tanto dificulta de sobremanera la correcta ejecución del control de movimiento.

Por otra parte, si en el entorno existen objetos de color blanco o amarillo o el suelo es de estos colores, el método tendrá dificultades para su correcta detección. Además, durante la realización de estas pruebas se ha comprobado que los brillos o reflejos en las superficies captadas por la cámara han sido en muchas ocasiones tomadas erróneamente como líneas del carril.

Estos problemas se deben en gran medida a la implementación actual del algoritmo de detección de carriles, el cuál después de un breve análisis se considera que podría ser mejorado en un futuro, como se comenta en las conclusiones de este apartado.

❖ Entorno simulado

Para el entorno simulado, se ha planteado el mismo circuito descrito en la Figura 72 y que se analizará a continuación. En la Figura 76 se puede ver la maqueta completa de la ciudad en el entorno de simulación y el Robobo iniciando su trayectoria. Como se ha comentado anteriormente, el Robobo debe seguir una trayectoria recta hasta llegar a la curva, donde una señal de giro con su correspondiente marcador ArUco le indicará la próxima acción. Una vez realizado el giro, el Robobo se encontrará con un tramo cortado por obras, por lo que deberá iniciar otro camino circulando por dentro de la ciudad como se puede ver en la Figura 77 y en la Figura 78.

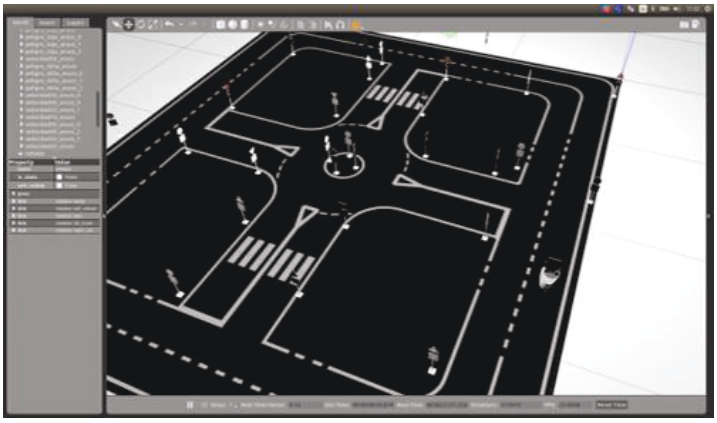


Figura 76. Inicio de la trayectoria del Robobo en la maqueta de la ciudad en el entorno real. Fuente: Elaboración propia.

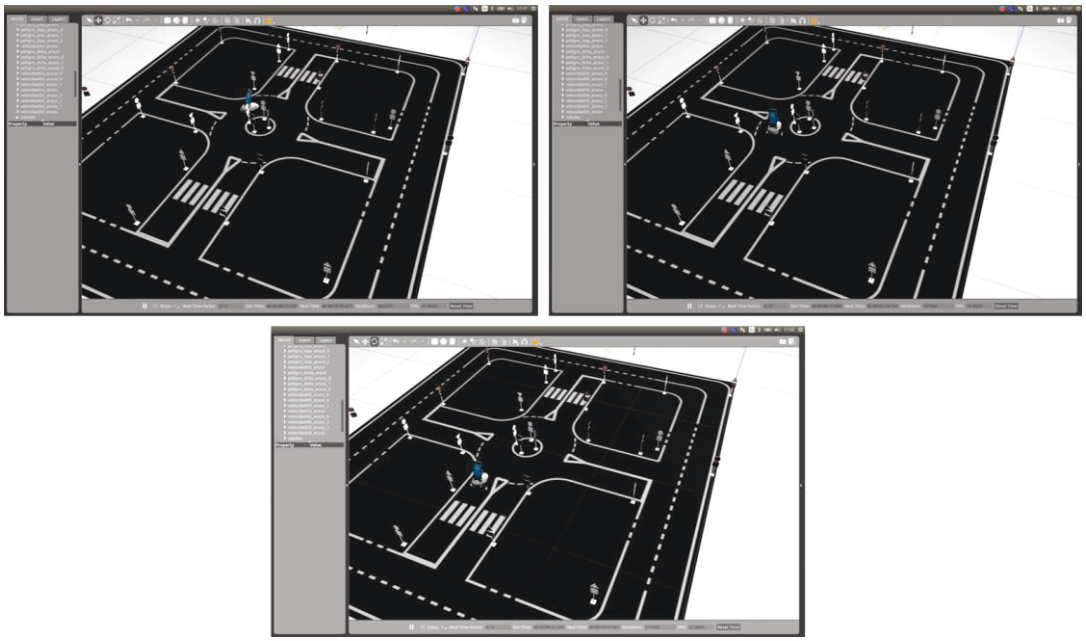


Figura 77. Circuito del Robobo en el entorno real en la maqueta de la ciudad. Fuente: Elaboración propia.

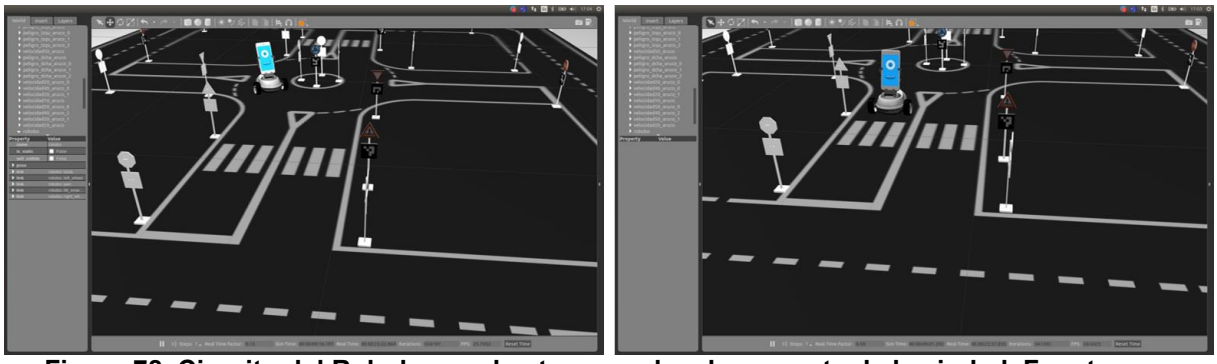


Figura 78. Circuito del Robobo en el entorno real en la maqueta de la ciudad. Fuente: Elaboración propia.

Como se ha comentado al principio de este apartado, el detector de líneas usado en el robot real no se encuentra implementado en ROS para ser usado en el simulador y por lo tanto se ha utilizado un script escrito en Python el cual utiliza OpenCV para realizar un filtrado gaussiano de la imagen.

A continuación, se ha hecho uso del método *HoughtLineP*. Este método devuelve las coordenadas (x, y) de ambos extremos de las rectas de los carriles. Haciendo uso de estas coordenadas se ha realizado un control de movimiento proporcional para lograr que el robot circule manteniéndose dentro de ambos carriles. Para ello, se ha calculado el punto medio del carril por el que circula haciendo uso de estas coordenadas.

Antes de seguir con la explicación del control de carriles, se va a exponer brevemente lo que es el control PID. El algoritmo PID (proporcional, integral, derivado) está formado por la suma de tres componentes, Proporcional, Integral, y Derivado. Matemáticamente, un controlador PID tiene la siguiente formulación.

$$Output\ (t) = Kp \cdot e(t)\ +\ Ki \int_0^t e(t) \cdot dt\ +\ Kd \cdot \frac{de}{dt} \tag{7}$$

Cada componente del PID es "independiente" de las demás, y en concreto para este trabajo interesa controlar el componente Kp, que es el primero de la Ecuación (7). El factor proporcional tiene una gran influencia en la velocidad de respuesta del sistema. Si tenemos una Kp pequeña, el sistema va a tardar mucho en alcanzar la consigna y si lo aumentamos, conseguimos disminuir el tiempo de respuesta.

En este caso el error se calcula de la misma manera que en el control real, se toma el punto medio del carril por el que circula y se compara con el punto medio de la imagen. Este error se multiplica por *kp* (que tiene valor 0,1) para obtener un valor al que hemos denominado incremento y que por lo tanto será proporcional al error obtenido. El robot lleva una velocidad fija en ambas ruedas y a estas se le añadirá este incremento para que gire a la derecha o a la izquierda en función de la desviación.

Además, a este control se le ha añadido una serie de limitadores del valor de salida al incremento, con el objetivo de evitar que el robot pueda acelerar excesivamente en un momento dado. Por un lado, se ha incorporado un limitador de la velocidad donde el incremento no puede superar el valor de 15. Si en algún momento este valor es mayor, pasa a ser sustituido directamente por 15. Por otro lado, se ha implementado otra limitación para que no exista la posibilidad de que se genere un incremento negativo. De esta manera, si el incremento da un valor por debajo de cero, este pasa a ser directamente cero.

En cuanto al rango de operación de este control, se ha tomado un rango de ±20 pixeles. Este rango ha sido tomado de esta manera, para no tener como referencia solo un pixel que se considere el centro del carril y ha sido creado basándose en el control realizado en el entorno real. Dentro de este rango, el robot debe mantener el mismo rumbo y si el error fuese menor, debería girar a la izquierda y si fuese mayor debería girar a la derecha.

En este entorno, no existen sombras que puedan afectar al detector de carriles, sin embargo, el movimiento del robot es muy lento respecto al del real porque el tiempo de simulación también es más lento.

❖ Comparación y conclusiones

Si comparamos ambos escenarios, vemos que el entorno en el que se encuentre el modelo con el que se va a trabajar es muy importante, porque como se ha comentado en anteriores ocasiones, las luces son un factor muy importante a tener en cuenta. También, el hecho de que el simulador vaya más lento implica cambios en los parámetros del movimiento para que el robot realice el circuito de una manera similar. A continuación, se muestra una tabla con el valor de los parámetros tanto para el entorno real como para el simulado.

Tabla 27. Comparación de los parámetros que intervienen en el movimiento del Robobo en los distintos entornos. Fuente: Elaboración propia.

Parámetros	Entorno real	Entorno simulado
Velocidad base de ambas ruedas en el movimiento recto	15	20
Velocidad de la rueda izquierda en el giro cuando detecta un ArUco	10	-20
Velocidad de la rueda derecha en el giro cuando detecta un ArUco	20	20
Tiempo necesario para realizar cada giro en la detección del ArUco	3,8	2,7
Coeficiente de rozamiento	0,09	0,09

En cuanto a los problemas asociados a la detección de carriles basada en imagen, podrían paliarse en un futuro trabajando en la algoritmia actualmente implementada. Por ejemplo, en escenarios reales de conducción autónoma, algunos algoritmos utilizan diferentes métodos de procesamiento de imagen para zonas claras y zonas sombrías de la vía, una vez segmentadas dichas zonas. Este tipo de aproximación podría también tratar de añadirse al algoritmo de detección de carriles actualmente implementado. Después de analizar los fundamentos del algoritmo actual, también se cree que sería adecuado mejorar el recorte automático que se hace del área de interés de la imagen, en el que se trata de eliminar la información visual que no pertenece a la carretera, ya que ahora mismo únicamente se consigue hacer el recorte a partir de la línea del horizonte, pero no un recorte de la superficie lateral que no pertenece al área de interés.

6.6 Pruebas multi-robot

En escenarios de conducción autónoma la comunicación de información potencialmente relevante en tiempo real entre diferentes vehículos es una línea de desarrollo e investigación interesante, al igual que para otros escenarios de robótica autónoma en los que distintos robots trabajan en tareas colaborativas en un mismo entorno.

Esta comunicación está presente en el robot real ya que este es capaz de soportarla, sin embargo, en el simulador de Gazebo todavía no se habían hecho las adaptaciones necesarias. En el robot real hay un modo multi-robot en el que se puede configurar cada Robobo (en ROS) para que todos se conecten a un mismo máster. En este caso, uno de ellos actúa como máster y los nodos de los demás se conectan a este para poder así comunicarse entre ellos.

Para solucionar este problema en el simulador, lo primero que se hizo fue adaptar la implementación de Gazebo para que pudiese soportar la operación de múltiples instancias del mismo tipo de robot. Con la implementación existente, esto era imposible debido a que los componentes ROS a través de los que se accedería a la funcionalidad de cada robot (nodos, topics y servicios) entrarían en conflicto con los de los demás.

En el siguiente apartado se explicará la adaptación necesaria de la implementación en Gazebo para poder utilizar más de un robot en el entorno simulado y así llevar a cabo las pruebas planteadas.

6.6.1 Implementación en Gazebo de multi-robot

Para posibilitar la ejecución de múltiples instancias del robot Robobo en Gazebo se utiliza el concepto de *namespace* de ROS. Estos son espacios jerárquicos de nombres dentro de los que se pueden definir los robots simulados con todos sus *nodos* y todos sus componentes. Esto nos permitirá posteriormente, que en tiempo de ejecución los recursos de un robot (nodos, topics y servicios) cambien su nombre de acceso al añadirles el prefijo indicado por el namespace del que cuelga la definición de cada robot, consiguiendo el efecto de mantener identificadores únicos para los recursos de varios robots que son del mismo tipo.

Por lo tanto, de cara al usuario, la estrategia simplemente implica que se configure un grupo con *namespace* diferente para cada uno de los Robobos que se definan para ejecutar en el modelo de simulación. En la Figura 79, se muestra un ejemplo en el que se definen dos robots Robobo según esta aproximación. Se define el grupo raíz, rodeado con un círculo en rojo, con el nombre “robot” (prefijo obligatorio para mantener la compatibilidad con los robots Robobo reales). Dentro de este grupo al que podemos considerar “grupo raíz”, se define un subgrupo para cada robot con un *namespace* diferente (Robobo1 y Robobo2 en este ejemplo). De esta manera se crea una jerarquía de *namespaces* que nos permite evitar conflictos en el acceso a los recursos.

En este punto, para facilitar la transición transparente de un programa ejecutado en simulación a un programa ejecutado con robots reales, el usuario deberá únicamente proporcionar el mismo nombre a los robots reales que el *namespace* definido para cada robot en Gazebo.

```
8
9       <!--create a new robobo-->
10      <group ns="robot">
11      <group ns="robobo1">
12
13              <!--define parameters-->
14              <param name="tf_prefix" value="robobo_tf" />
15              <arg name="robobo_name" default="robobo1"/>
16              <param name="robot_description" command="$(find xacro)/xacro --inorder '$(find robobo_gazebo)/urdf/robobo.urdf.xacro'
17                      pusher:=false
18                      camera_front:=true
19                      emotion:=NORMAL
20                      visualize_irSensor:=false
21                      visualize_camera:=false" />
22
23              <!--launch robobo model-->
24              <node name="robobo_model" pkg="gazebo_ros" type="spawn_model" respawn="false" output="screen" args="-urdf -x 0.5 -y 0.5 -
robot_description"/>
25
26              <!--launch infrared sensors-->
27              <node name="robobo_irs" pkg="robobo_gazebo" type="robobo_irs" args="-n $(arg robobo_name)"/>
28
29              <!--launch image node for light sensor-->
30              <node name= "image_view" pkg="image_view" type= "image_view" respawn= "false" output ="screen">
31                      <remap from="image" to="/robot/$(arg robobo_name)/light_sensor/image_raw"/>
32                      <param name="autosize" value="true"/>
33                      <param name="image_transport" value="compressed"/>
34              </node>
35      </group>
36      </group>
37
38      <!--create a new robobo-->
39      <group ns="robot">
40      <group ns="robobo2">
41
42              <!--define parameters-->
43              <param name="tf_prefix" value="robobo_tf" />
44              <arg name="robobo_name" default="robobo2"/>
45              <param name="robot_description" command="$(find xacro)/xacro --inorder '$(find robobo_gazebo)/urdf/robobo.urdf.xacro'
46                      pusher:=false
47                      camera_front:=true
48                      emotion:=NORMAL
49                      visualize_irSensor:=false
50                      visualize_camera:=false" />
51
```

Figura 79. Ejemplo de jerarquía de *namespaces* en el fichero de lanzamiento (*launch*) adaptado para ser implementado en Gazebo. Fuente: [21]**.**

Una vez realizado el cambio descrito en el fichero *launch* de Gazebo, también ha sido necesario modificar los *plugins* de Gazebo que implementan la conexión con ROS de cada uno de los controladores, para que utilicen los nuevos *namespaces* que se han definido.

Estos *plugins* están implementados en C++. En la Figura 80 se muestra uno de los segmentos de código C++ que se ha sido necesario añadirse al controlador *Encoder,* a modo de ejemplo, aunque este paso es necesario realizarlo en todos los demás controladores. Lo que se consigue con el cambio introducido es que, al inicio de la ejecución, cuando se crea un *nodo* de ROS, se obtiene del SDF del modelo el namespace que el usuario ha definido allí para un robot concreto. A continuación, se puede crear el *nodo* con este *namespace*, consiguiendo que todos los *topics* y servicios de ese nodo del robot pasen a tener como prefijo el nuevo espacio de nombres definido por el usuario.

```
std::string robot_namespace = "";
if (_sdf->HasElement("robotNamespace")) {
  robot_namespace = _sdf->GetElement("robotNamespace")->Get<std::string>();
}
// Create node handler
rosNode.reset(new ros::NodeHandle(robot_namespace));
```

Figura 80. Modificación del *plugin* de los *encoders* en el simulador de Gazebo. Fuente: [22]

6.6.2 Ejemplo multi-robot

Como se ha comentado al principio de este apartado, poder trabajar en un escenario en el que varios robots puedan comunicarse resulta de gran interés, especialmente cuando es posible programar estos comportamientos multi-robot pasando de una manera transparente del programa simulado al real. Esto resulta muy útil, por ejemplo, en un contexto de robótica educativa cuando trabajar desde el inicio con varios robots en un entorno real puede resultar tedioso o puede haber dificultades de acceso a varias unidades del robot real por parte de todos los alumnos.

Otro aspecto importante a destacar es que el modelo de comunicación publicador/subscriptor, en el que se basan las comunicaciones multi-robot del Robobo, resulta muy adecuado y cómodo para programar comportamientos colectivos en contextos como el de una ciudad inteligente. Así, por ejemplo, un dispositivo puede avisar a todos los interesados sobre eventos que se encuentran en la carretera, estado del tráfico, etc. sin necesidad de una interacción directa explícita. Con esta idea en mente, se diseñó y programó un experimento en el que los robots publican en uno o varios *topics* alertas o información relevante que son capaces de sensorizar y a los que cualquier robot en cualquier momento temporal puede subscribirse y recibir dicha información, y en su caso ejecutar algún comportamiento adecuado.

En el caso del ejemplo planteado (Figura 81), uno de los robots detecta un objeto en la carretera que impide la circulación y mediante el modelo publicador/subscriptor, publica en un *topic* un aviso de carretera cortada (Figura 82), para que el resto de robots que están circulando por la ciudad puedan subscribirse a esta información y cambiar la ruta antes de llegar a la zona conflictiva (Figura 83).

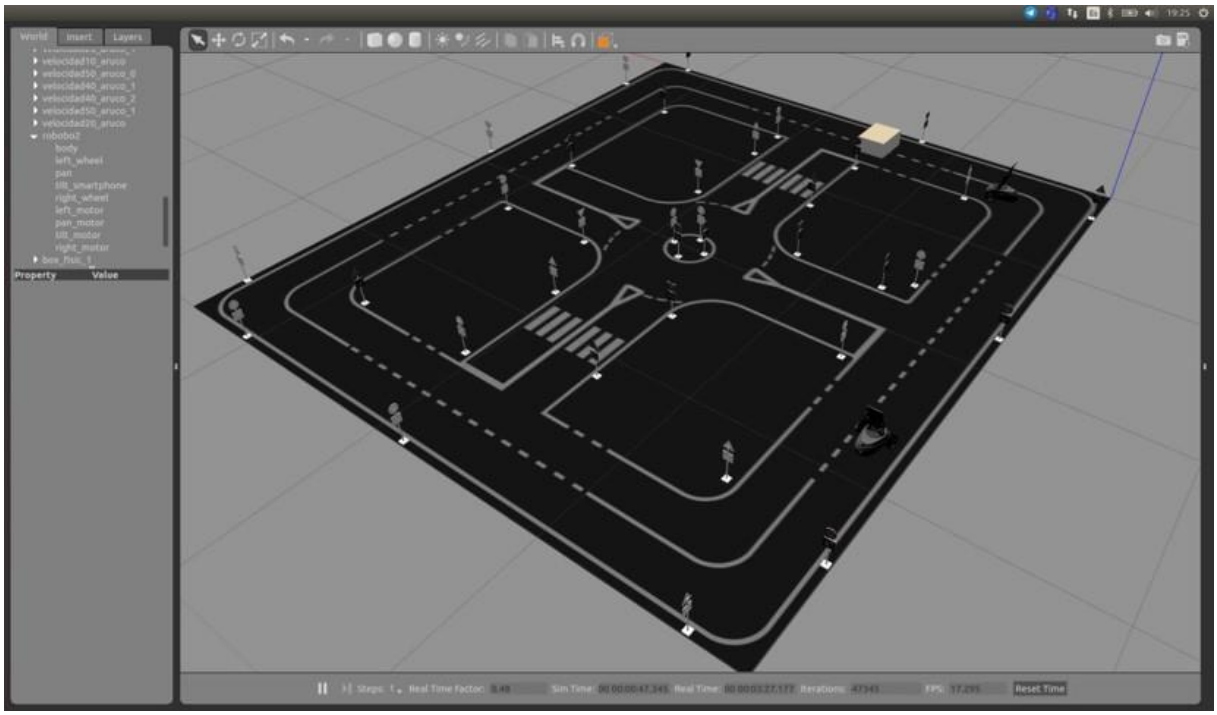


Figura 81. Circuito planteado para llevar a cabo las pruebas multi-robot. Fuente: Elaboración propia.

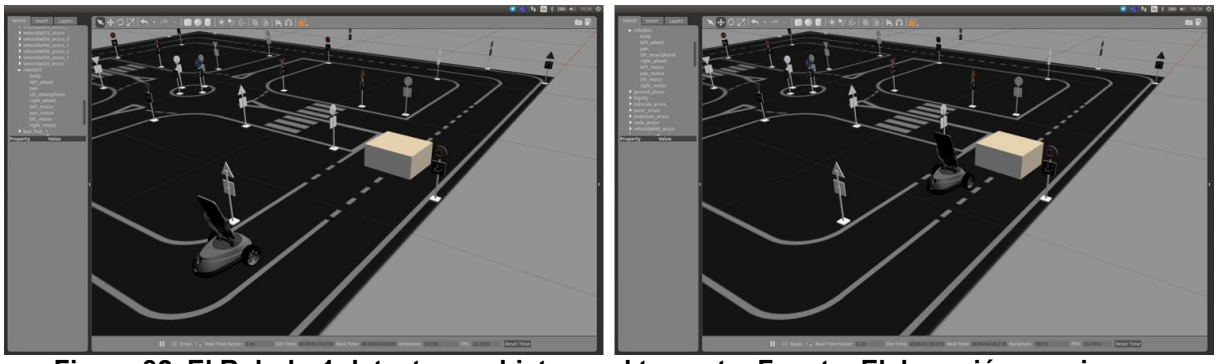


Figura 82. El Robobo1 detecta un objeto en el trayecto. Fuente: Elaboración propia.

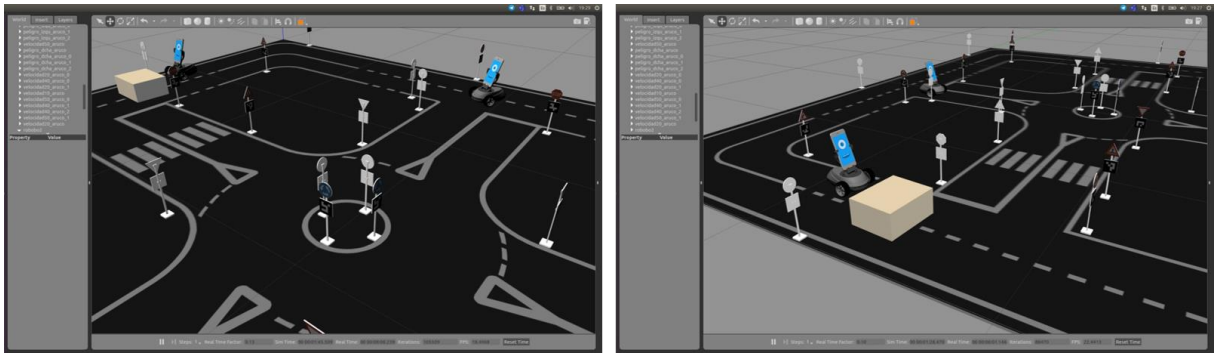


Figura 83. El Robobo1 publica la información y el Robobo2 se subscribe y recibe el aviso a través de esa subscripción. Fuente: Elaboración propia.

El escenario de este ejemplo sería la circulación de varios robots por las calles de la ciudad. Todos ellos disponen de sensores para detectar obstáculos de forma que, podrían reducir la velocidad o detenerse para evitar una colisión y avisar a los demás robots de los obstáculos que encuentren en su camino. Este aviso, se realizaría a través de un topic en el que uno o varios de los robots publicarían notificaciones de la ubicación de los obstáculos. Los robots que quisiesen recibir estas notificaciones se subscribirían a este topic.

Sobre este escenario se ha realizado un ejemplo con dos robots, donde uno de ellos, Robobo1, actúa de coordinador, es decir, es el encargado de notificar cualquier obstáculo que detecte en medio de la vía a través de la publicación de información el topic definido en este ejemplo para este tipo de notificaciones. El otro robot, Robobo2, actúa de “seguidor”, subcribiéndose a dicho topic para recibir los avisos que publique Robobo1.

La ejecución de este ejemplo comienza con ambos robots circulando por la ciudad, hasta que, Robobo1 detecta un obstáculo en su camino a través de los sensores de infrarrojos, detiene su movimiento, parando los motores de las ruedas y publica en el topic la detección de este obstáculo. Dado que Robobo2 está subscrito a este topic, recibe la notificación de aviso de obstáculo en la calzada por lo que, en lugar de seguir su camino por esa misma vía, realiza un giro a la izquierda evitando la vía cortada y poder, de esta forma, continuar su camino.

❖ Comparación y conclusiones

Al comparar el experimento en ambos entornos, han aparecido problemas en los propios sensores/actuadores ya comentados en anteriores pruebas pero que se vuelven a poner de manifiesto en este apartado. Factores como la iluminación, el rozamiento y la sensorización de infrarrojos son, en definitiva, variables que se deben tener en cuenta a la hora de realizar pruebas con robots. En el caso de nuestro ejemplo, en el entorno real la luz que exista en el ambiente es importante ya que puede influir en el color del objeto y en cómo lo detecta el Robobo, además de las sombras que se crean por no disponer de una luz homogénea en toda la habitación.

Esto afecta según se ha visto al sensor de infrarrojos que devuelve un valor diferente dependiendo del color y del tipo de material del objeto en cuestión. Sin embargo, en el entorno simulado este suceso no ocurría de la misma manera porque el valor para distintas superficies era prácticamente el mismo. También es importante recordar que el movimiento del robot es más lento en el entorno simulado, y por lo tanto los valores de los parámetros que afectan a este deben de modificarse respecto al real.

En cuanto a las nuevas capacidades del simulador para programar comportamientos con múltiples robots, los experimentos realizados ponen de manifiesto el potencial de este modelo para ser utilizado para enseñar a alumnos la programación de comportamientos colectivos autónomos en los que los robots no necesitan conocerse unos a otros de antemano y pueden incorporarse al colectivo dinámicamente. Además, como se ha mencionado previamente, ahora también resulta muy sencillo el paso de estas aplicaciones multi-robot desde entorno simulado a entorno real, simplemente cambiando las direcciones IP a las que se conectarán las aplicaciones que implementen los experimentos.

Asimismo, esta aproximación también tiene el potencial de ser una arquitectura base para aplicar técnicas de IA que analicen la información obtenida del colectivo para obtener comportamientos inteligentes de alto nivel en la ciudad.

7 CONCLUSIONES Y TRABAJO FUTURO

Una vez realizadas todas las pruebas y obtenidos todos los resultados, se pretende elaborar una conclusión global que permita esclarecer si los objetivos preestablecidos al inicio del proyecto han sido logrados y que propuestas de mejoras se podrían aportar para futuros trabajos.

A continuación, se muestran los objetivos planteados en un principio:

1. Participar en la definición y construcción de un escenario real y simulado adecuado para la realización de múltiples experimentos que permitan introducir conceptos de robótica autónoma e inteligencia artificial.

2. Analizar las capacidades y limitaciones actuales del modelo simulado del robot en Gazebo en relación con su utilización en un escenario como el propuesto. Se tendrán en cuenta las condicionantes externos del uso del robot en contextos educativos.

3. Conceptualizar y desarrollar aplicaciones experimentales que permiten evaluar el grado de fidelidad entre el comportamiento en el escenario simulado y en el escenario real. Dichas aplicaciones implicarán la programación del control autónomo del robot utilizando las plataformas software ROS y Python.

4. Proponer medidas correctoras en el entorno simulado y/o en la programación/ hardware del robot real.

En el punto número uno se establece participar en la definición y en la construcción de un modelo con el que poder realizar actividades que ayuden a elaborar una conclusión sobre la relación de fidelidad entre el entorno real y el simulado. Este punto se ha abordado con éxito puesto que se ha colaborado en la construcción y simulación de la maqueta de una ciudad en la que se han realizado diferentes pruebas de validación y comunicación multi-robot que han permitido obtener unos resultados que ayuden a elaborar unas conclusiones para mejorar su comportamiento.

En cuanto al punto dos y tres, se han analizado las fortalezas y debilidades del modelo simulado en Gazebo en relación con el modelo real mediante la aplicación de programación en las plataformas software ROS y Python. De este análisis se pueden extraer una serie de conclusiones generales acerca del funcionamiento de los sensores y actuadores de manera individual. Los movimientos en la simulación son más lentos que en la realidad y por ello los parámetros usados son diferentes en ambos entornos necesitando, de un valor de velocidad o tiempo más elevado para la obtención del mismo movimiento.

Por otro lado, la detección de los marcadores ArUcos funciona de manera similar en ambos entornos, sin embargo, esta detección depende en gran medida de la luminosidad ambiental siendo esto un parámetro que no puede ser modificado ni cuantificado en el simulador. Además, para una misma distancia en ambos entornos el área del Aruco detectado varía más cuanto menor es la distancia respecto al robot, por ello cada entorno tiene un polinomio de interpolación distinto que relaciona ambas variables.

Para los sensores infrarrojos, el funcionamiento en el simulador está restringido a un rango de distancias donde funciona correctamente, entre 15 y 50cm. Además, el simulador no parece hacer distinción según el color y la superficie reflejada.

Todos estos sensores y actuadores se han analizado individualmente con el objetivo de utilizar toda la información obtenida de su funcionamiento para ser usado de forma conjunta en un entorno realista, como es la ciudad. Dado que el robot real contaba con un detector de carriles que se ejecutaba de forma nativa en el smartphone, fue necesario la creación de un control similar en el simulado que permitiese realizar las mismas funciones.

Para finalizar, como se ha dicho anteriormente, era necesario que en Gazebo existiese un comportamiento multi-robot que permitiese una comunicación entre distintos dispositivos para poder ser empleado en labores de investigación en robótica educativa. Gracias a la implementación realizada, esto es posible.

Por lo tanto, las labores realizadas en el presente trabajo fin de grado han aportado una serie de información acerca del funcionamiento de los distintos sensores y actuadores del robot simulado, que permiten tener un simulador con un control más robusto y cercano a la realidad. Además, se ha añadido un control de carretera y se ha adaptado el uso de un detector de carriles al entorno de ROS para poder ser usado en Gazebo, junto con la implementación del comportamiento multi-robot desde simulador.

En cuanto al punto cuatro, como trabajo futuro, se propone mejorar la algoritmia del detector de carriles actual en ambos entornos implementado un sistema más robusto capaz de diferenciar entre zonas claras y oscuras de la carretera y que utilice un área de detección mejorada con la capacidad de discriminar de mejor manera las partes no pertenecientes al carril.

También se propone, como se ha comentado durante el análisis de las pruebas, incorporar la posibilidad de cuantificar la luminosidad en el propio simulador, para poder analizar si la detección de ArUcos sigue siendo buena dependiendo de este factor.

Por último, teniendo como base de los experimentos realizados en este proyecto resultaría interesante plantear nuevos experimentos multi-robot con comportamientos colectivos en los que participen un mayor número de robots y que permitan continuar probando todas las capacidades de sensorización presentes en el robot de forma simultánea.

8 REFERENCIAS

[1] “Fundación telefónica | Guía práctica: Nosotros Robots,” 2011.

[2] “Wikipedia, the free encyclopedia.” https://es.wikipedia.org/wiki/Robótica_educativa#Orígenes_e_historia.

[3] “Wikipedia, the free encyclopedia.” https://es.wikipedia.org/wiki/Lego_Mindstorms#Tercera_Generación:_Bloque_EV3.

[4] “Lego | Lego mindstorms Ev3.” [Online]. Available: https://www.lego.com/es-es/product/lego-mindstorms-ev3-31313.

[5] “Robotic | e-manual, Turtlebot3.” [Online]. Available: http://emanual.robotis.com/docs/en/platform/turtlebot3/overview/.

[6] “k team | khepera-iv.” [Online]. Available: https://www.k-team.com/khepera-iv.

[7] “Soft Bank Robotics | NAO.” [Online]. Available: https://www.softbankrobotics.com/emea/en/nao.

[8] “Kickstarter| Romo the smartphone robot for everyone.” [Online]. Available: https://www.kickstarter.com/projects/peterseid/romo-the-smartphone-robot-for-everyone?lang=es.

[9] J. Ramón and N. Vera, “Carreteras, territorio y utopía moderna,” pp. 31–40, 2013, [Online]. Available: http://ropdigital.ciccp.es/pdf/publico/2013/2013_febrero_3540_04.pdf.

[10] “CEA | Los niveles de la conduccion autonoma.” https://www.cea-online.es/blog/213-los-niveles-de-la-conduccion-autonoma.

[11] C. S. [DOT CSV], “Www.Youtube.Com,” *¿Por qué aun no tenemos conces autónomos?* 2019.

[12] “Robobo | TheRoboboproject.” [Online]. Available: https://therobobоproject.com/.

[13] L. Llamas, “Artificial Intelligence Teaching through Embedded Systems: A Smartphone-Based Robot Approach,” Ferrol, A Coruña, 2019.

[14] L. Llamas, “Adapting Computer Vision Algorithms to Smartphone based Robot for Education.” Ferrol, A Coruña, 2020.

[15] “ArUco tutorial | Open Cv.” [Online]. Available: http://docs.opencv.org/3.1.0/d5/dae/tutorial_aruco_detection.html.

[16] “Gazebo | Gazebosim.” [Online]. Available: http://gazebosim.org/.

[17] “viewpage wiki.nps.edu.” [Online]. Available: https://wiki.nps.edu/pages/viewpage.action?pageId=989954095.

[18] T. F. De Mestrado, “Escola Politécnica Superior,” 2019.

[19] R. Ragel De la Torre, “Entorno de simulación ROS/Gazebo,” pp. 65–118, 2010, [Online]. Available: http://bibing.us.es/proyectos/abreproy/5139/fichero/PFC-+Por+capítulos%252F3-Entorno+de+simulación.pdf.

[20] “Robot Operating System (ROS).”, Robótica, Grado en Tecnologías Industriales, UDC, Ferrol, 2019.

[21] “Github | launch.” [Online]. Available: https://github.com/mintforpeople/robobo-gazebo-simulator/blob/master/launch/robobo.launch.

[22] “Github | Encoders.” [Online]. Available: https://github.com/mintforpeople/robobo-gazebo-simulator/blob/master/plugins/encoders.cpp.

AENXOS

ANEXO I: Presupuesto

C.I CAPÍTULO I. SOFTWARE					
N/P	CONCEPTO	Uds.	Cantidad	Precio (€)	Importe (€)
2.1	Licencia Gazebo (open source)		1	0,00 €	0,00 €
	IMPORTE TOTAL CAPÍTULO I				0,00 €

C.II CAPÍTULO II. MANO DE OBRA					
N/P	CONCEPTO	Uds.	Cantidad	Precio (€)	Importe (€)
3.1	Desarrollo del modelo	Horas	360 h	40,00 €	14 400,00 €
	IMPORTE TOTAL CAPÍTULO II				14 400,00 €

Resumen por capítulos	
Capítulo I. Software	0,00 €
Capítulo II. Mano de obra	14 400,00 €
IMPORTE DE EJECUCIÓN MATERIAL	**14 400,00** €

IMPORTE DE EJECUCIÓN MATERIAL	14 400,00 €
13% de Gastos Generales	1872,00 €
6% de Beneficio Industrial	864,00 €
IMPORTE DE EJECUCIÓN	17136,00 €
21% de IVA	3 598,56 €
IMPORTE DE CONTRATA	**20 734,56 €**